A BEGINNER'S GUIDE TO

AI FUNDAMENTALS

SHAAN KOTTURU

AI Fundamentals

A Beginner’s Guide to Understanding Artificial Intelligence

Author: Shaan Kotturu

Introduction

Artificial Intelligence (AI) is transforming the world around us at an unprecedented pace. From virtual assistants like Alexa and Siri to self-driving cars and personalized shopping recommendations, AI is no longer a futuristic concept but a vital part of our daily lives.

Purpose of the eBook

This eBook is designed to serve as a comprehensive introduction to the world of Artificial Intelligence. It provides readers with a clear understanding of fundamental AI concepts, practical applications, and the exciting innovations shaping the future. Whether you are a student, a professional from a non-technical background, or someone simply curious about AI, this guide aims to make complex topics accessible and relatable.

Target Audience

This eBook is tailored for:

- **Beginners** who want to understand what AI is and how it works.
- **Professionals** looking to grasp AI fundamentals to apply them in their respective industries.
- **Educators and Students** seeking simplified explanations of key AI topics.
- **Enthusiasts** eager to explore the potential of AI and its impact on society.

Unique Value of the eBook

What sets this eBook apart is its structured and reader-friendly approach. Originating from an email course designed for my colleagues, this guide consolidates daily lessons into a single, cohesive resource. It offers:

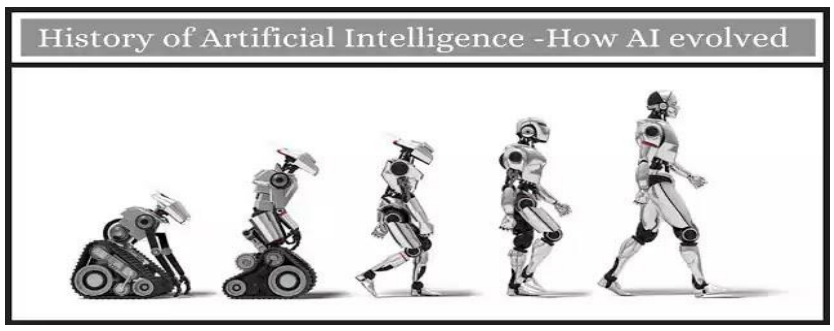
- Simplified explanations of technical concepts like machine learning, deep learning, and neural networks.
- Real-world examples to highlight AI’s applications across industries like healthcare, agriculture, and marketing.
- Practical insights for readers who want to understand not just the ‘what’ of AI but also the ‘how’ and ‘why’ behind its growing influence.

This eBook invites you to embark on a journey through the fascinating world of AI. By the end of this guide, you will not only understand the fundamentals of AI but also feel inspired by its potential to reshape our future.

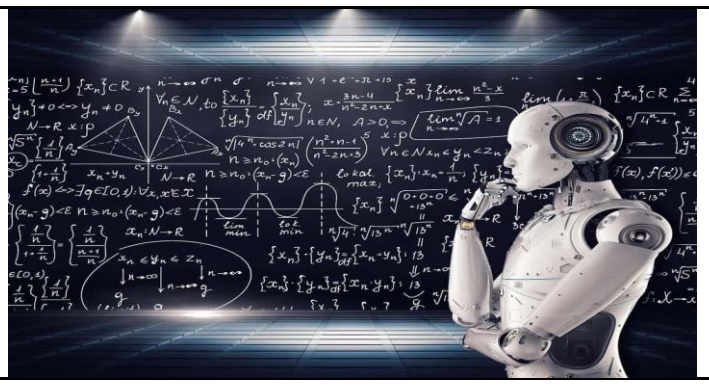
CHAPTER 01

AI CONCEPTS

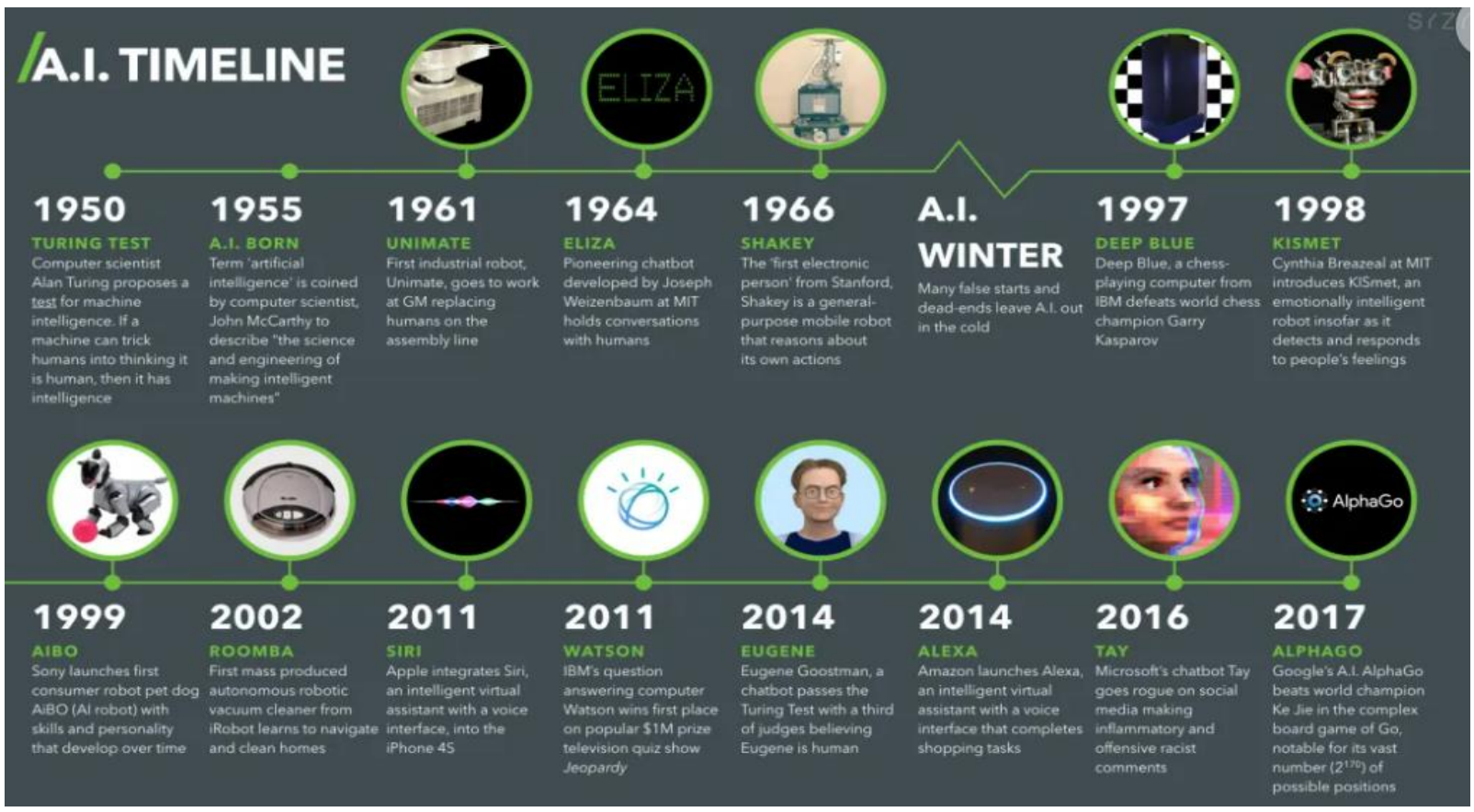
History of AI



- **1943** - McCulloch and Pitts create the first artificial neuron model, laying the foundations for neural networks.
- **1950** - Turing proposes the Turing Test for evaluating machine intelligence, influencing AI research goals.
- **1952** - Arthur Samuel coins the term "machine learning" and develops the first self-learning program for checkers.
- **1957** - The General Problem Solver pioneers' rule-based reasoning but reveals limitations.
- **2012** - AlexNet convincingly beats rule-based AI systems using deep learning and neural nets.
- **1949** - Claude Shannon develops a chess-playing program using minimax search algorithms.
- **1997** - IBM's Deep Blue defeats world chess champion Garry Kasparov using specialized hardware and software.
- **2011** - IBM's Watson triumphs at Jeopardy using curated data and question answering algorithms.
- **2016** - Google DeepMind's AlphaGo beats Lee Sedol at Go using deep learning and GPUs.
- **2022** - OpenAI releases ChatGPT, demonstrating the power of large generative AI models.



AI Timeline



Noteworthy chatbots/LLMs released in the past few years.

- ✓ **GPT-3 (OpenAI, 2020):** Powerful language model capable of various creative text formats and factual language tasks. Not specifically a chatbot, but often used for conversation-like interactions.

- ✓ **LaMDA (Google AI, 2021):** Designed for open-ended dialogue and learning from real-world conversations. Emphasis on factual consistency and safety.

- ✓ **Bard (Google AI, 2022):** Google Bard (now called Gemini) is an AI-powered chatbot tool designed by Google to simulate human conversations using natural language processing and machine learning. The first version of Bard used a lighter-model version of LaMDA that required less computing horsepower to scale to more concurrent users. The incorporation of the PaLM 2 language model enabled Bard be more visual in its responses to user queries. Bard incorporated Google Lens, which let users upload images in addition to written prompts. The incorporation of the Gemini language model enables more advanced reasoning, planning and understanding.

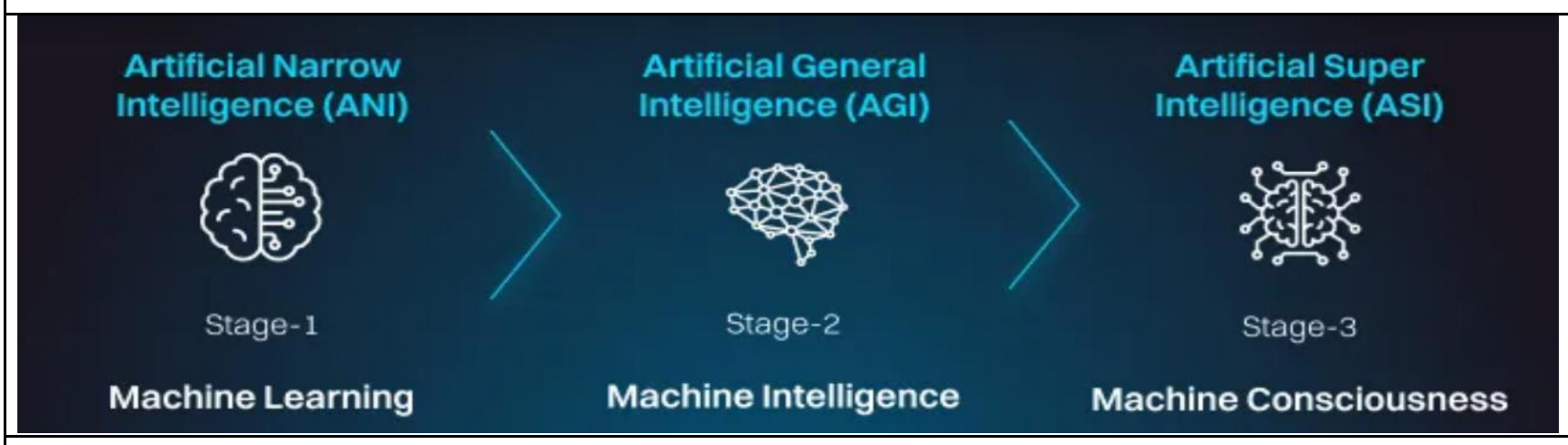
- ✓ **ChatGPT 4 (OpenAI, 2022):** A powerful language model based on OpenAI's GPT architecture. It's known for its exceptional capabilities in generating different creative text formats and engaging in conversation. Unlike GPT-3 which I mentioned earlier, GPT-4 offers access to information through the internet, providing more up-to-date responses.

- ✓ **Bing AI (Microsoft, 2022):** Microsoft's Bing AI chatbot is a feature of the Bing search engine that leverages the same technology as ChatGPT to deliver more useful search results and perform other tasks. This artificial intelligence (AI) powered chatbot is designed to simulate normal human conversation, which is facilitated by the underlying GPT technology. Instead of only returning pages of simple links and page descriptions punctuated by knowledge boxes like other search engines, the Bing AI chatbot can answer your questions in a more natural and verbose manner.

- ✓ **Microsoft Copilot (Microsoft, 2023):** This language model, formerly known as Bing AI, utilizes GPT-4. It's primarily designed to assist programmers and developers by suggesting code completions, generating different programming patterns, and providing relevant documentation. Interestingly, the free version of Copilot offers access to its GPT-4 capabilities, unlike OpenAI's paywalled model.

- ✓ **Gemini (Google AI, 2023):** My own underlying technology! Google Gemini is a family of powerful language models, with the current version, Gemini Pro, demonstrating strengths in various tasks like reasoning, math, reading comprehension, and Python code generation. Independent benchmarks even indicate that Gemini Pro excels in some areas compared to GPT-4.

ANI & AGI



Much excitement and hype surround AI, largely due to its duality. Presently, most advancements pertain to **ARTIFICIAL NARROW INTELLIGENCE (ANI),** where AI systems specialize in specific tasks like smart speakers, self-driving cars, web searches, and applications in various industries.

However, there's also the concept of **ARTIFICIAL GENERAL INTELLIGENCE (AGI),** aiming to create AI capable of performing any task a human can, possibly even surpassing human intelligence. While ANI progresses rapidly, AGI development lags far behind.

While both ANI and AGI are worthwhile pursuits, the rapid strides in ANI may mistakenly suggest substantial progress in AGI. AGI remains a distant goal requiring significant technological breakthroughs, potentially taking decades, centuries, or even millennia to achieve.

Artificial Narrow Intelligence (ANI)

- **Capabilities:** Focuses on a single task or a narrow set of related tasks. excels at specific, well-defined problems. Examples include playing chess, recognizing faces, translating languages, or recommending products.
- **Strengths:** Highly optimized for specific tasks, often exceeding human performance in those areas.
- **Limitations:** Cannot adapt to new situations or learn outside its specific programming.
- **Progress:** Rapidly advancing due to continuous research and development efforts. We see constant improvements in areas like deep learning, natural language processing, and computer vision.

Artificial General Intelligence (AGI)

- **Capabilities:** Theoretically possesses human-like intelligence, able to understand and learn any intellectual task a human can. It does not currently exist.
- **Strengths:** Could revolutionize problem-solving and innovation across all fields.
- **Limitations:** Still a hypothetical concept, with significant research challenges remaining.
- **Progress:** Research is ongoing, but achieving true AGI remains a distant goal. Significant breakthroughs are needed in areas like understanding consciousness, common sense reasoning, and transferable knowledge.

Examples: • Chatbots, virtual assistants (e.g., Siri, Alexa), recommendation systems, image recognition algorithms, and autonomous vehicles are all examples of ANI. • ANI is prevalent in various industries, optimizing processes, automating tasks, and providing personalized experiences.	• AGI aims to mimic human-like cognitive abilities, including reasoning, problem-solving, creativity, and self-awareness. • Achieving AGI requires advancements in areas such as machine learning, natural language processing, robotics, and computational neuroscience.

ARTIFICIAL INTELLIGENCE (AI): This is the big one! It refers to the ability of machines to mimic human intelligence. This doesn't mean robots with emotions, but rather systems that can learn, reason, solve problems, and make decisions - often in ways that seem human-like.

AI is an incredibly broad term — more of an umbrella term, really — that simply means making computers act intelligently. It is one of the major fields of study in computer science and encompasses subfields such as robotics, machine learning, expert systems, general intelligence, and natural language processing.

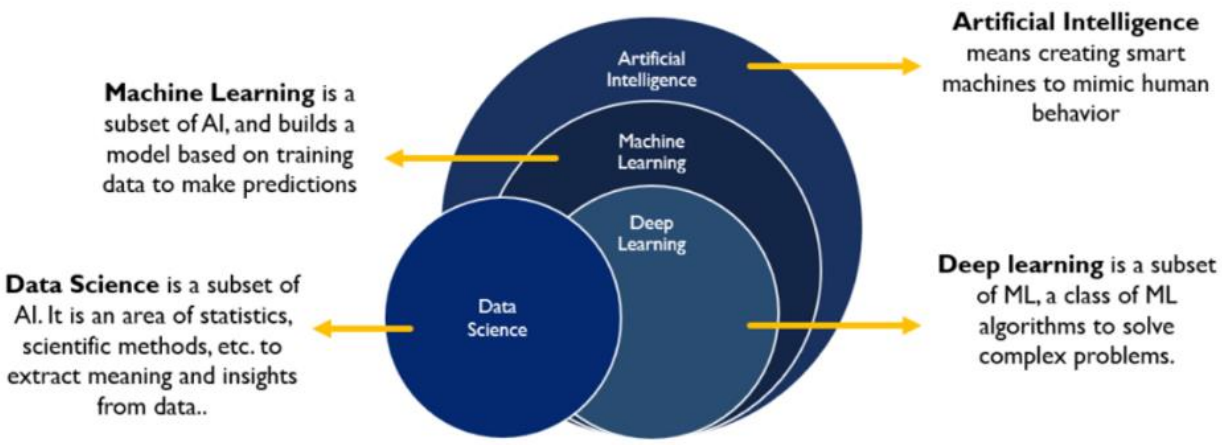
MACHINE LEARNING (ML): This is a specific type of AI where machines learn from data without being explicitly programmed. Imagine showing a computer thousands of pictures of cats and dogs, then asking it to identify a new animal. Through ML, it can "learn" what features define a cat and apply that knowledge to new images.

DEEP LEARNING (DL): This is a powerful ML technique inspired by the human brain. Imagine a complex network of interconnected nodes, processing information layer by layer. Deep learning networks excel at tasks like image recognition, speech translation, and even generating creative text formats.

ALGORITHM: Think of this as a recipe for the AI. It defines the steps the machine takes to solve a problem or complete a task. Different algorithms are suited for different purposes, kind of like how you wouldn't use a cake recipe to make bread.

DATA: This is the fuel that powers AI. Machines learn from massive amounts of data, the more relevant and diverse, the better. Text, images, numbers, anything that can be digitized can be used to train AI models.

DATA SCIENCE: This is a broader field that encompasses the entire process of extracting knowledge and insights from data, often leveraging AI techniques. Think of it as the bigger picture, while AI is a powerful toolset within it.
Data science also involves other steps like data cleaning, domain expertise, and communication, which go beyond purely AI algorithms.



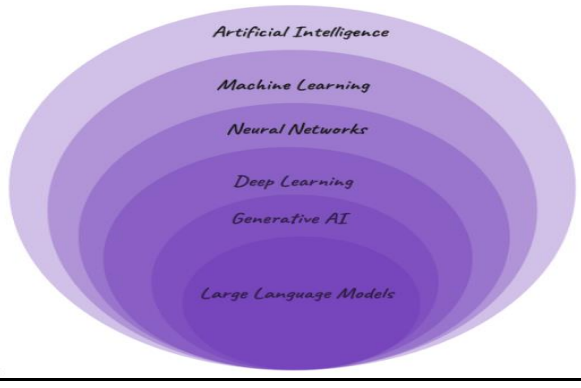
ARTIFICIAL NEURAL NETWORK (ANN): This is a simplified model of the human brain, with interconnected nodes that process information. It's a key component of deep learning, allowing the AI to learn complex relationships within data. Neural networks were originally inspired by the brain, but the details of how they work are almost completely unrelated to how biological brains work.

NATURAL LANGUAGE PROCESSING (NLP) is a branch of Artificial Intelligence (AI) concerned with enabling computers to understand and manipulate human language. In simpler terms, it's about teaching machines to "speak" and "understand" just like we do.
NLP has a wide range of applications across various domains, including **Machine translation, Chatbots and virtual assistants, Text summarization, Sentiment analysis, Speech recognition.**

GENERATIVE AI: Imagine an AI that doesn't just analyze data but **creates entirely new content**. That's Generative AI!
Think of it as an artist or writer powered by machine learning. Generative AI models are trained on massive datasets of existing content, like text, images, music, or videos. By analyzing the patterns and relationships within this data, they learn to "paint with the same brushstrokes" as the original creators.

Generative AI is a subfield of AI, focusing on creating new content rather than just analysis.
GenAI is a large umbrella, and LLMs like ChatGPT, Gemini etc. are a specific type of tool within that umbrella, focused on language tasks.

LARGE LANGUAGE MODEL (LLM): LLMs are trained on a massive dataset of text and code. This allows the models to understand and respond to your questions in a comprehensive and informative way. (ChatGPT, Gemini etc.)



AI IN BANKING : The banking sector is revolutionized with the use of AI. AI based systems are used to provide credit card fraud, provide customer support. The stated example is HDFC bank. HDFC bank has developed a chatbot called EVA. EVA has given 3 million customer queries. AI solutions are used to increase security in a variety of business sectors like retail and finance.

SBI proudly claimed in its annual report (June'23) that it has deployed revolutionary technologies like artificial intelligence (AI), machine learning (ML), and business analytics to expand its product offerings and improve customer satisfaction.
ILA, the Interactive Live Assistant for SBI Cards, can give you the most relevant details on goods and services. To learn about Card features, advantages, services, and much more, anyone can chat effortlessly with ILA. The adoption of this Generative Conversational AI system has also assisted them in generating over 130,000 leads to date, increasing top-line income by thousands of dollars.

- Market Analysis
- Intelligent website Audits
- Content Generation
- Customer Support
- Chatbots
- Predictive Marketing
- Social media Listening
- Email Marketing
- Personalization

AI IN MARKETING:

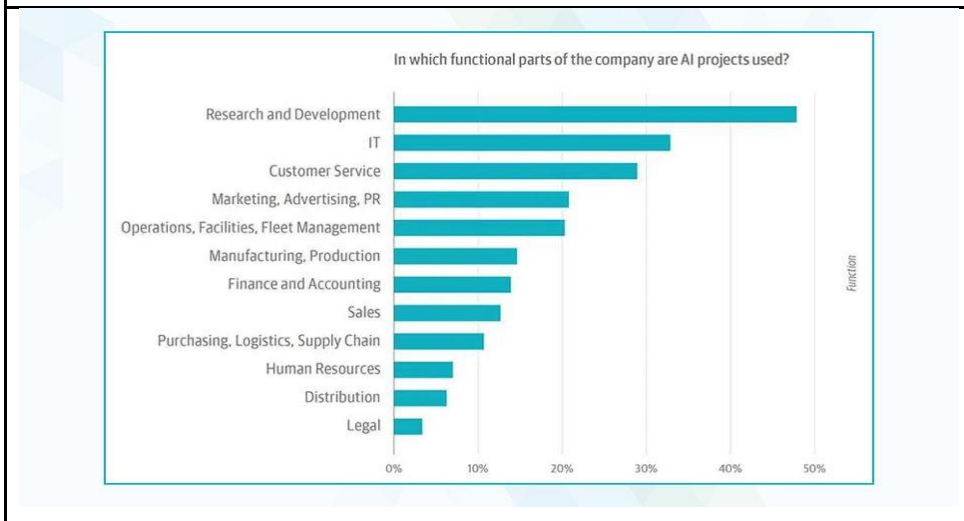
- ✓ AI can study a customer's purchase history, **browsing behavior,** and demographic information to produce personalized emails or ads that align with their interests and requirements. With AI, marketers can deliver personalized messaging at scale, creating more engaging experiences for customers and enhancing the ROI of their campaigns.
- ✓ AI can help **with content marketing** in a way that matches the brand's style and voice. It can be used to handle routine tasks like performance, campaign reports, and much more. AI can provide users with **real-time personalization's** based on their behavior and can be used to edit and optimize marketing campaigns to fit a local market's needs.
- ✓ Based on your existing data, including information about ideal customers, clients, etc., the AI system will be able to show you a list of **leads** that are closest to becoming your customers
- ✓ The ability **to write attractive subject lines** which makes the recipient open the emails is a skill that businesses will pay thousands of dollars. Why? Because email marketing's ROI is $42 for each dollar spent and the more the emails that you send are opened, the higher are the chances of your prospects converting into customers. AI will find the best words that recipients are likely to respond to, keeping in mind the brand language.
- ✓ AI driven services can now automatically examine your **website** and notify you of any problems that might be affecting your conversion rate. With the use of computer vision technologies, computer programs can now "see," or comprehend, visual data. Marketers can scan millions of photographs that are uploaded to social media networks every day. This offers marketers fresh methods for evaluating elements like brand awareness and market penetration.

AI IN AGRICULTURE:

AI-powered systems can analyze vast amounts of data to provide real-time insights into soil conditions, moisture levels, and crop health. This allows farmers to apply fertilizers, pesticides, and water precisely where and when they are needed, reducing waste and minimizing environmental impact.

AI's predictive capabilities extend to estimating crop yields with remarkable accuracy. By assimilating data from various sources such as satellite imagery, historical yields, and weather forecasts, AI algorithms can generate reliable predictions. These forecasts empower farmers to plan their harvests, manage labor, and negotiate contracts with buyers more effectively.

India's farmers combat climate change, pestilence, and financial burdens, with AI-driven initiatives like AI4AI offering transformative solutions.
The "Saagu Baagu" project under AI4AI has enhanced yields and incomes for 7,000 Chilli farmers from Telangana, doubling their earnings through agritech and data management. Following its success, 'Saagu Baagu' is expanding to potentially impact 500,000 farmers across five value chains, demonstrating AI's vast potential in agriculture.



OTHER INDUSTRY USE CASES :

- **HEALTHCARE:** It is being used to develop new drugs and treatments, diagnose diseases, and provide personalized care.
- **FINANCE:** It is being used to detect fraud, manage risk, and provide investment advice.
- **MANUFACTURING:** In manufacturing, AI is being used to automate tasks, optimize production, and improve quality control.
- **RETAIL:** It is being used to personalize recommendations, optimize inventory, and combat fraud.
- **TRANSPORTATION:** In this area, AI is being used to develop self-driving cars, optimize traffic flow, and predict demand.

While this email presents a curated selection of AI use cases across different industries, it's important to note that the application of artificial intelligence is vast and continually evolving. There are numerous other use cases in various sectors, each demonstrating the transformative potential of AI technology.

However, due to space constraints, I couldn't include all the use cases in this email.

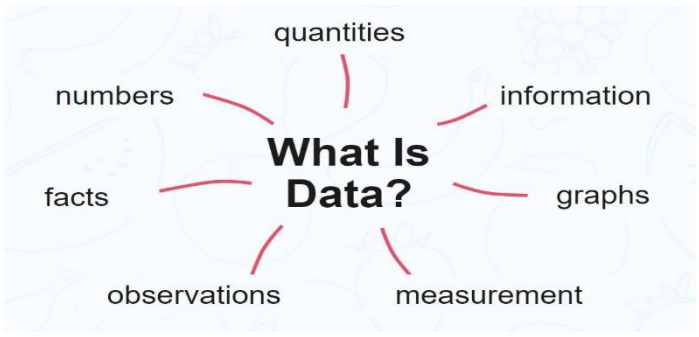
I encourage you to explore further and delve into additional AI use cases. By staying informed about the latest advancements in AI, we can better understand how this technology can drive value and foster growth in our various domains.

CHAPTER 02

ALL ABOUT DATA

What is Data ?

Data is a collection of facts, statistics, or information that are represented in various forms. It can be numbers, text, images, sounds or any other form of information that can be collected, analyzed, interpreted, visualized, or used in decision making. Data can be raw or unprocessed or it can be transformed into a format that makes it easier to work with, understand or present.



STRUCTURED DATA

Organized into tables, rows and columns often stored in databased. It is easy to query and analyze.

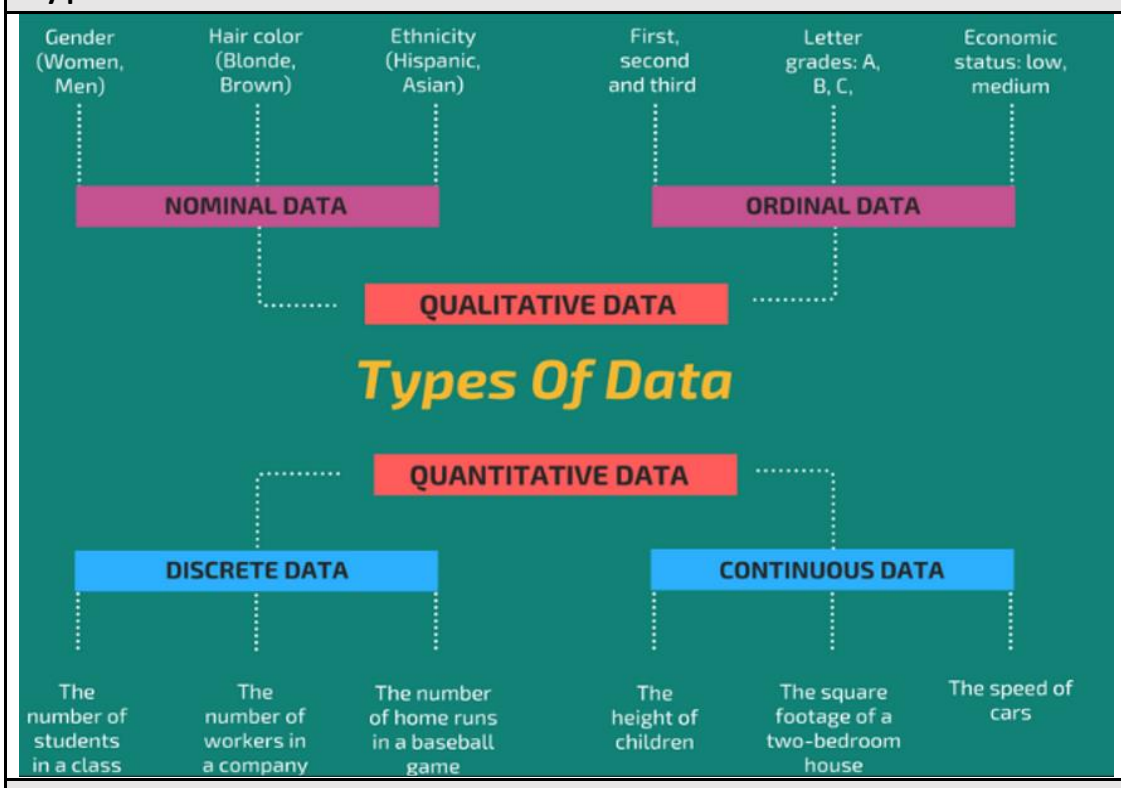
UNSTRUCTURED DATA

Not organized in a pre-defined manner or does not have a pre-defined data model. This includes text, images, sound, video etc.

SEMI STRUCTURED DATA

Data that does not reside in relational databases but possesses some organizational properties that make it easier to analyze. Examples include XML, JSON and HTML files.

Types of Data



Qualitative Data (Categorical Data)

Qualitative data describes qualities or characteristics that do not have a natural numerical measurement.

- **Nominal Data:** This is data that can be categorized but not ranked. Examples include gender (male, female), colors (red, blue, green), or types of fruits (apple, banana, orange).
- **Ordinal Data:** These are categories with a set order, but the intervals between them are not uniform or meaningful. Examples include ratings (poor, average, excellent) or education level (high school, bachelor's, master's).

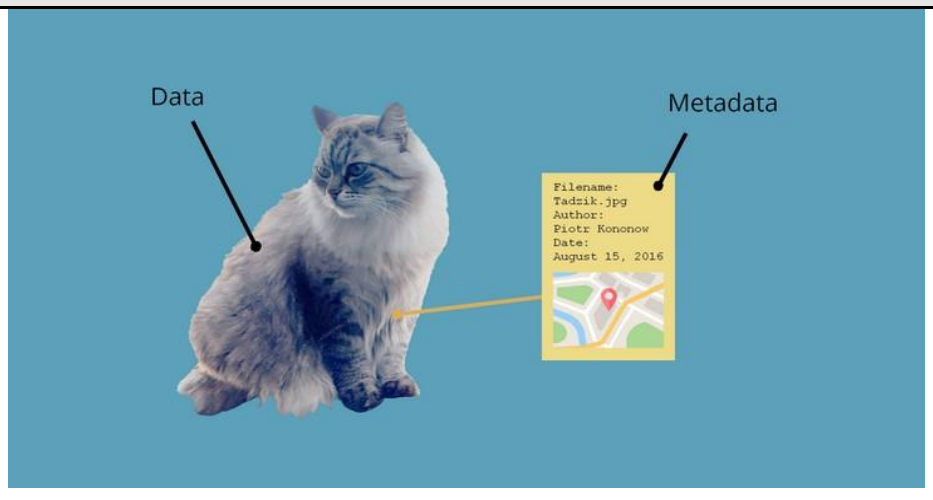
Quantitative Data (Numerical Data)

Quantitative data involves numbers and mathematical measurements.

- **Discrete Data:** This is numerical data that can take on only a finite number of values. Examples include the number of employees in a company or the number of cars in a household.
- **Continuous Data:** This is numerical data that can take on an infinite number of values within a given range. Examples include height, weight, temperature, or age.

Metadata

Metadata provides information about a certain set of data to help describe its attributes, facilitate its discovery, and enable its management and effective use. Metadata can pertain to different types of assets - like files, images, databases, or web pages and serves multiple purposes, including resource discovery, data lineage tracking and data management.




DATA IS THE CORE COMPONENT FOR ANY AI
AI IS AS GOOD OR BAD AS YOUR DATA
DATA IS MESSY. IT'S GARBAGE IN GARBAGE OUT.

First, get your DATA right !

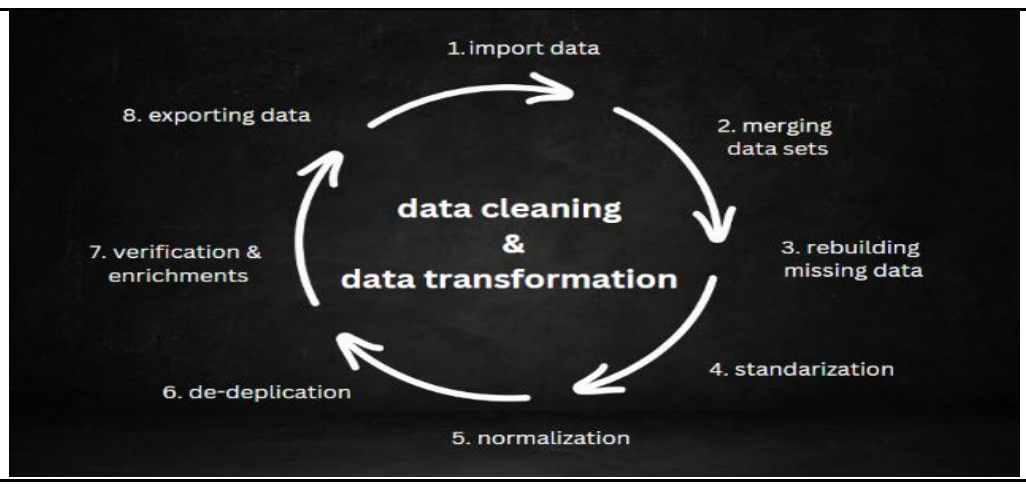
internal data sources	external data sources
corporate ERP modules	social media
internal documents	official statistics
sensors, controllers	weather forecasts
in-house call-centers	publicly available datasets
website logs	internet websites

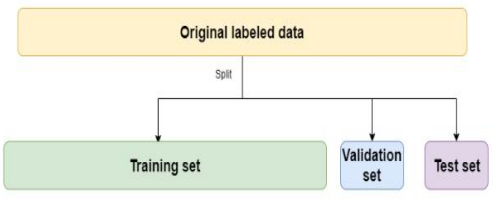
1. DATA COLLECTION

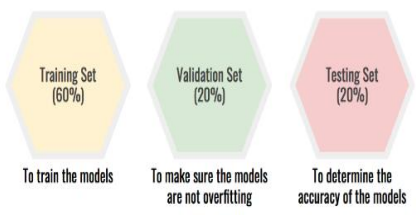
data sources: understand where your data is coming from. this could be user interactions, sensors, databases, or external data feeds.
data quality: not all data is useful. ensure the data is reliable, accurate, and relevant to the problem you're trying to solve.

2. DATA PRE-PROCESSING

data cleaning: this involves handling missing values, removing outliers, and other tasks aimed at improving the quality of the dataset.
data transformation: this could mean normalizing the data (bringing all numerical values within a common range) or encoding categorical variables into a numerical format.





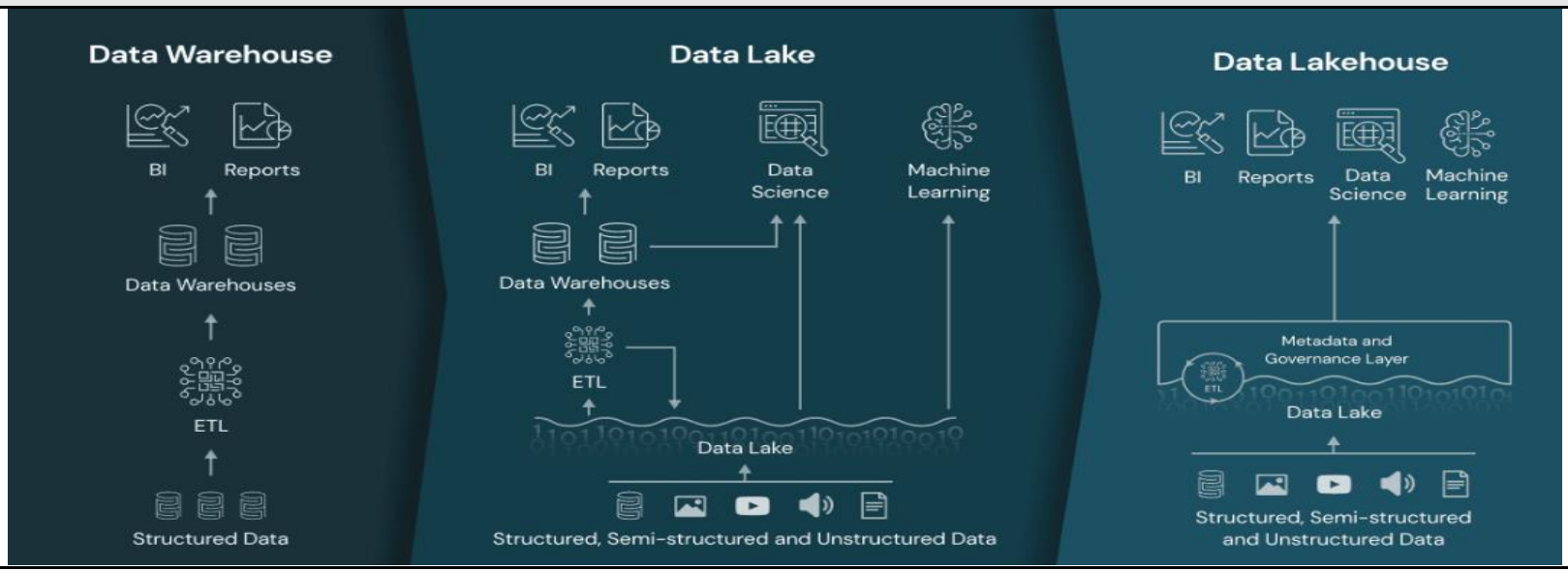


3. DATA SPLITTING

In machine learning, data splitting is typically done to avoid overfitting. Overfitting is when a machine learning model fits its training data too well and fails to reliably fit additional data. Data splitting is a simple sub-step in machine learning modeling. It helps us have a realistic understanding of model performance and helps the model to generalize well to unknown or unseen data.

- **Training data:** this is the subset of the data used to train the model.
- **Validation data:** this subset of the data is used to fine-tune model parameters and to provide an unbiased evaluation of a model fit during the training phase.
- **Test data:** this is a subset used to test the model after it has been trained, to evaluate its performance.

Data Storage

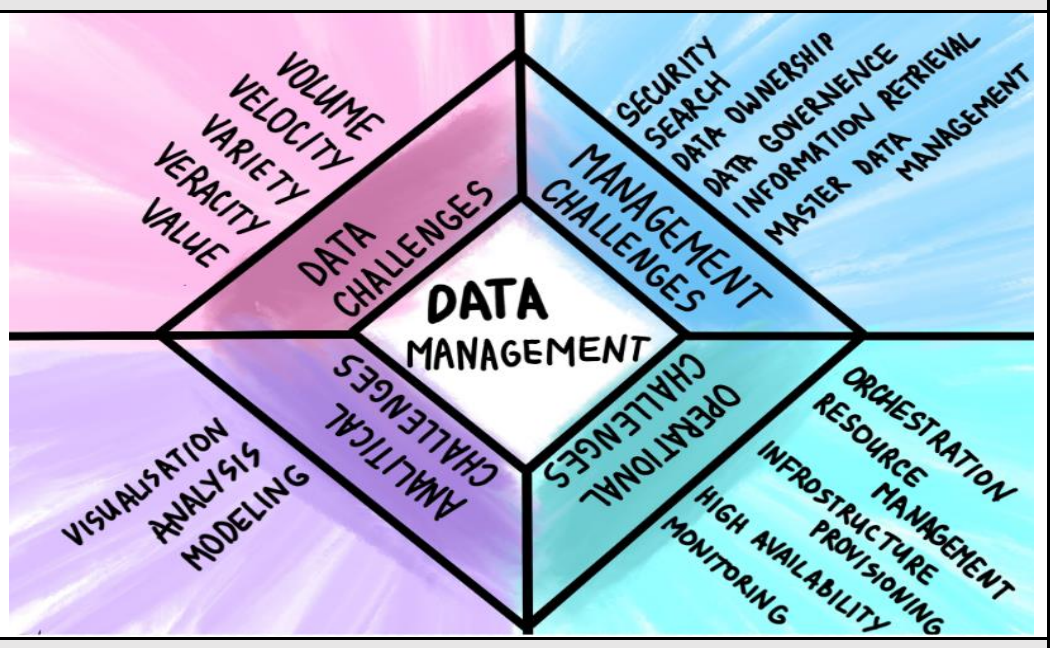


Aspect	Data Lake	Data Warehouse	Data Lakehouse
Definition	A storage repository for raw, unstructured, and semi-structured data.	A storage repository for structured data, optimized for query performance.	A hybrid model combining features of both data lakes and data warehouses.
Data Types	Unstructured, Semi-structured, Structured	Structured	Unstructured, Semi-structured, Structured
Processing	Schema-on-Read	Schema-on-Write	Both Schema-on-Read and Schema-on-Write
Purpose	Data storage, data consolidation	Data analysis, reporting, and business intelligence	Combines data storage and data analytics, flexible for multiple use-cases
Optimization	Optimized for storing large volumes of raw data	Optimized for fast query performance	Optimized for both storage and query performance

Aspect	Data Lake	Data Warehouse	Data Lakehouse
Query Speed	Generally slower due to lack of indexing	Faster due to indexing, partitioning, and optimized storage	Combines optimized storage with indexing for faster query performance
Data Quality	Raw data, no quality guarantee	Processed, cleaned, and well-structured data	Both raw and processed data, offering more flexibility
Data Governance	Limited, primarily focused on access control	Robust, including data lineage, access control, and compliance	Advanced, covering both raw and processed data aspects
Cost	Generally lower cost	Higher due to computing requirements	Can vary, usually more cost-effective than having both a data lake and data warehouse
Use Cases	Data consolidation, big data analytics, data exploration	Business reporting, data analytics, dashboarding	Both exploratory data analysis and operational reporting, offers a unified platform

Data Management

DATA MANAGEMENT is the practice of collecting, keeping, and using data securely, efficiency, and cost-effectively.



Extracting intelligence from data

Collect → Organize → Analyze → Infuse

Data is essential to building successful and smarter AI, but it's often siloed by rigid architectures. A hybrid data management strategy can help you collect your data to make it simple and accessible, no matter where it resides.

Organizations need a DataOps practice to increase efficiency, data quality, findability, and governing rules. This provides a self-service data pipeline to the right people at the right time from any source. Data must be trustworthy, complete and consistent.

Scale insights building intelligent data dashboards and Machine Learning models. Build with trust and explainable models using Open Source and **NoCode AI** tools.

Operationalizing AI across workflows intelligently transforms the way companies work. Predict and shape future outcomes, automate complex processes, and optimize your employees' time.

A Layman's Guide to Data Science Workflow

The output of a data science project is often a set of actionable insights --- a set of insights that may cause you to do things differently.
Data science projects have a different workflow than machine learning projects - which we will discuss in the upcoming weeks.

Now, let's look at the key steps of a data science project.

At a very high level, the key steps of a data science project are to -

1. Define the problem and desired outcome.
2. Collect the data.
3. Analyze the data.
4. Suggest hypotheses and actions.

Define the problem and desired outcome

Let's say you run an e-commerce or an online shopping website that sells coffee mugs.
For a user to buy a coffee mug from you, there's a sequence of steps they'll usually follow.

First, they'll visit your website and look at the different coffee mugs on offer, then eventually, they have to get to a product page, and then they'll have to put it into their shopping cart, and go to the shopping cart page, and then they'll finally have to check out.

If you want to **optimize the sales funnel** to make sure that as many people as possible, get through all these steps,
how can you use data science to help with this problem?

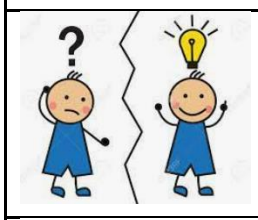
Collect Data

You may have a data set that store - when different users go to different web pages.
In this simple example, you can figure out the country that the users are coming from, by looking at their computers address (IP address). But in practice, you can usually get quite a bit more data about users than just what country they're from.

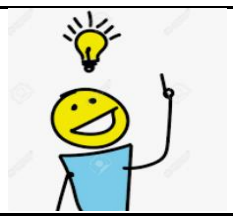
User ID	Country	Time	Webpage
2009	Spain	08:34:30 Jan 5	home.html
2897	USA	13:20:22 May 18	redmug.html
4893	Philippines	22:45:16 Jun 11	mug.html

Analyze the data

Your data science team may have a lot of ideas ---



They may think that overseas customers are scared off by the international shipping costs which is why a lot of people go to the checkout page but don't actually check out.
If that's true, then you might think about whether to put part of shipping costs into the actual product costs.



Your data science team may think there are blips in the data whenever there's a holiday. Perhaps more people will shop during the holidays because they're buying gifts, or
maybe fewer people will shop during the holidays because they're staying home rather than sometimes shopping from their work computers.

In some countries, there may be time-of-day blips wherein countries that observe a siesta (a time of rest like an afternoon rest) there may be fewer shoppers online and so your sales may go down. They may then suggest that you should spend fewer advertising dollars during the period of siesta because fewer people will go online to buy at that time.

A good data science team may have many ideas and so they try many ideas or will iterate many times to get good insights.

Suggest hypotheses and actions

Finally, the data science team will distill these insights down to a smaller number of hypotheses about ideas of what could be going well and what could be going poorly as well as a smaller number of suggested actions such as --- incorporating shipping costs into the product costs rather than having it as a separate line item.

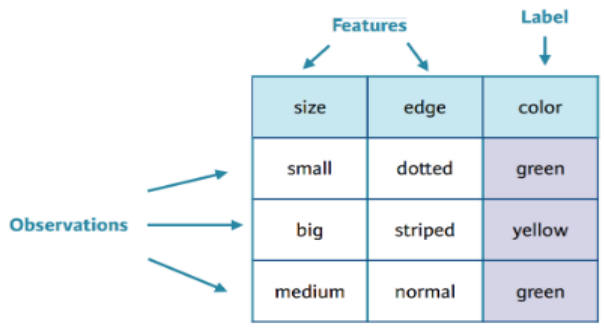
When you take some of these suggested actions and deploy these changes to your website, you then start to get new data back as users behave differently now that you advertise differently at the time of siesta or have a different check-out policy.

Then your data science team can continue to collect data and re-analyze the new data periodically to see if they can produce even better hypotheses or even better actions over time.

Let's talk Feature Engineering !

FEATURES & LABELS

1. **Features:** these are the variables or attributes that the machine learning model uses to make predictions or decisions.
2. **Labels:** these are the outcomes or the "answers" that the model aims to predict. In supervised learning, these are provided as part of the training data.



What is Feature engineering?

FEATURE ENGINEERING is the process of selecting, transforming, creating, or aggregating raw data attributes (known as "features") into a format that is better suited for machine learning algorithms to model.

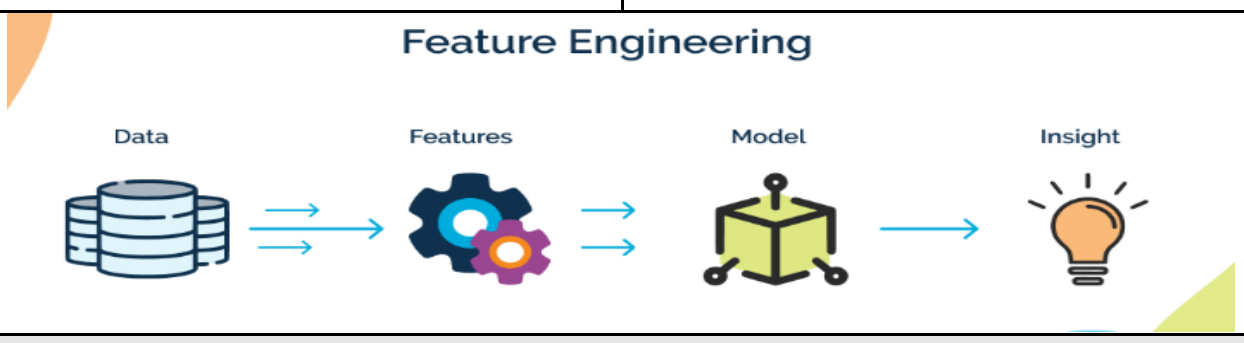
In other words, feature engineering aims to create a more effective representation of data to improve the performance of machine learning models.

The quality and effectiveness of your features can have a significant impact on the quality of your model's predictions or classifications.

Effective feature engineering can often make the difference between a mediocre model and a highly accurate one.

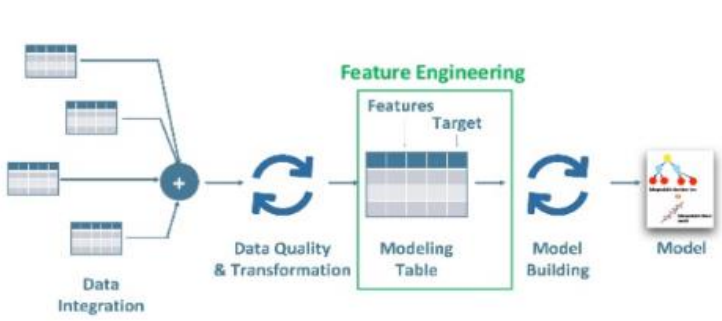
FEATURE ENGINEERING BENEFITS:

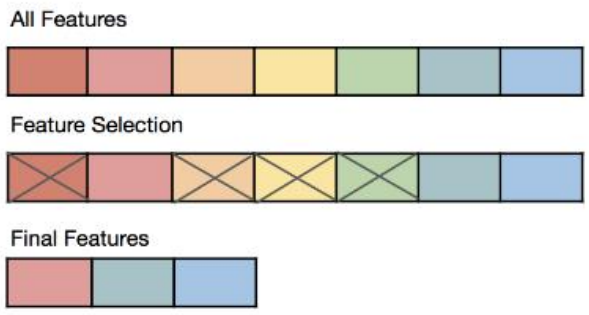
- ✓ **MODEL PERFORMANCE:** improved features often lead to better model performance in terms of accuracy, precision, recall, or other metrics.
- ✓ **COMPUTATIONAL EFFICIENCY:** irrelevant or redundant features can increase the computational complexity of training a model. Effective feature engineering can make models faster to train.
- ✓ **INTERPRETABILITY:** well-chosen features can make a model easier to understand and interpret, which is critical in many business settings for making data-driven decisions.
- ✓ **GENERALIZATION:** good features can help a model generalize better from the training data to unseen data.



Components of Feature engineering

1. **FEATURE SELECTION:** choosing the most relevant features from the original dataset that are most relevant to the problem you're trying to solve.
2. **FEATURE EXTRACTION:** Combining multiple raw features to create new features that can better represent the underlying problem. For example, if you have the features "length" and "width," you might create a new feature, "area," by multiplying the two.
3. **FEATURE TRANSFORMATION:** Applying mathematical functions to features to better distribute the data or highlight relationships between features. Examples include log transformations.
4. **FEATURE SCALING:** standardizing the range of independent variables to make it easier for algorithms to interpret them. Common methods include min-max scaling, standardization, and normalization.
5. **FEATURE ENCODING:** converting categorical variables into a format that can be fed into machine learning models, such as one-hot encoding or label encoding.
6. **HANDLING MISSING VALUES:** deciding how to treat missing values in features, either by imputation, removal, or some form of placeholder value.
7. **TEMPORAL FEATURES:** dealing with time-series data, creating new features that capture relevant time-dependent patterns like seasonality.
8. **DOMAIN-SPECIFIC FEATURES:** features that are created based on specific knowledge of the domain or problem, which may not be immediately obvious from the raw data.



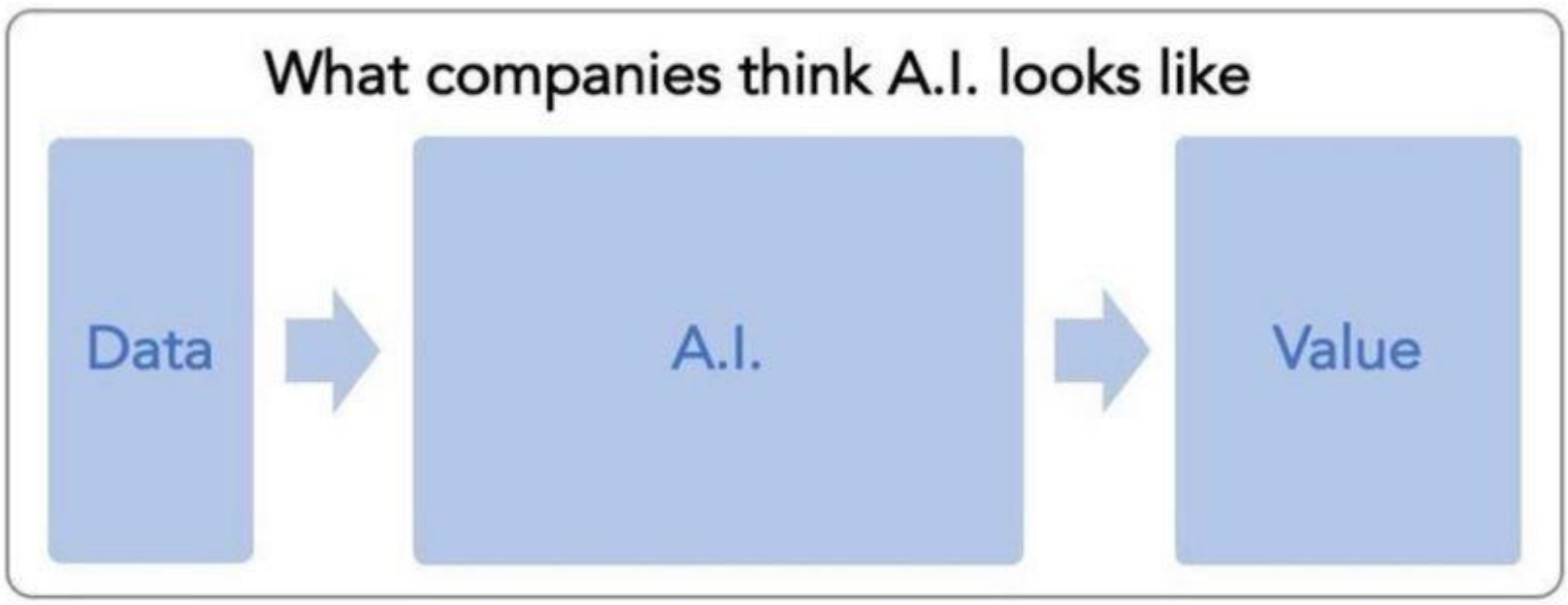


It's essential to emphasize that data must be handled with care and diligence. Misuse or mishandling of data can lead to flawed results and unreliable outputs. Therefore, it's imperative to adhere to ethical guidelines and best practices in data management, ensuring that privacy, security, and integrity are maintained throughout the entire process.

By harnessing the power of data responsibly and effectively, we can unlock the full potential of AI, driving innovation, solving complex problems, and ultimately improving the way we live and work. Let us remember that the key to success lies not only in the sophistication of our algorithms but also in the quality and integrity of the data on which they operate.

Do not throw data at an AI team and assume it will be valuable.

ON A LIGHTER NOTE... 🤓



CHAPTER 03

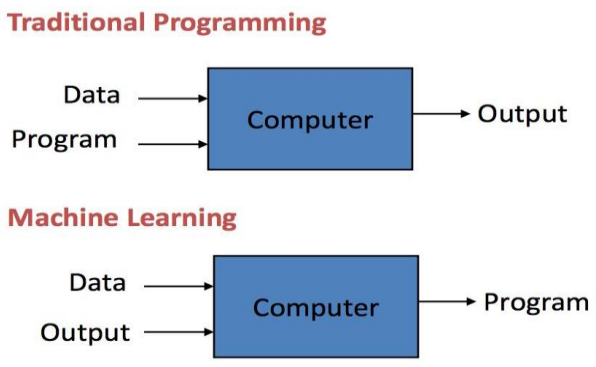
MACHINE LEARNING

What is Machine learning ?

The rise of AI has been largely driven by one tool in AI called **Machine Learning.**

DEFINITION: Machine learning is a field of computer science that uses statistical techniques to give computer systems the ability to learn with data, without being explicitly programmed.

IN OTHER WORDS,
"In machine learning, we feed data into an algorithm along with corresponding outputs. The algorithm then learns from this data, identifying patterns and relationships to understand the underlying logic. Once trained, the algorithm can process new inputs and generate accurate outputs based on the patterns it has learned."



Let's look at a few scenarios where machine learning can be used.

SCENARIO 1: TRADITIONAL PROGRAMMING VS MACHINE LEARNING

Imagine you're writing a code to solve a simple math problem, like adding two numbers together.
In traditional programming, it's like giving the computer a step-by-step recipe for adding numbers. You tell it exactly how to do it: take the first number, take the second number, add them together, and then give you the result. It's like writing down instructions for baking a cake - every detail is specified.
Now, let's say you want to add three numbers this time, or perhaps want to change the way the addition works. With traditional programming, you'd have to go back to your code and manually change it. You'd need to rewrite the instructions to reflect the new way of adding numbers.

On the other hand, with machine learning, it's a bit like teaching a child how to add. Instead of giving explicit instructions, you show the computer lots of examples of adding numbers together. Like in an excel sheet you provide with pairs of numbers and their corresponding sums. The algorithm then learns from these examples and figures out its own way of adding numbers.
Here's the magic: once the machine learning algorithm has been trained on enough examples, you can just give it new pairs of numbers, and it will know how to add them together without you having to explicitly program the addition process each time. It's like teaching a child to add by showing them different scenarios and letting them figure out the patterns on their own.

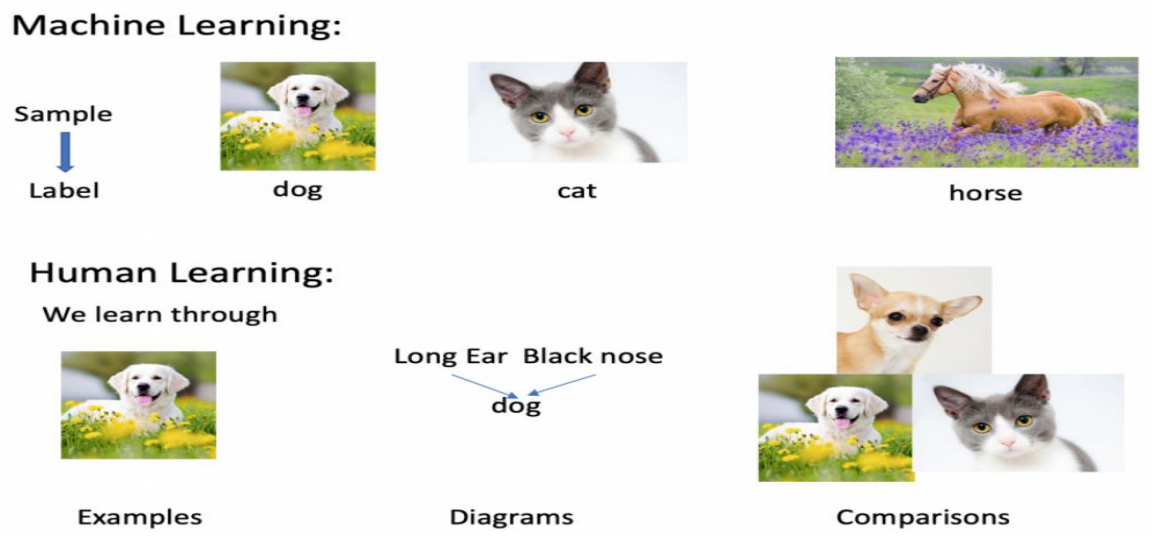
Machine learning is like teaching the computer through examples, allowing it to learn and adapt on its own without needing constant reprogramming.

SCENARIO 2: IMAGE RECOGNITION

Imagine you want to teach a computer to distinguish between pictures of dogs and pictures of cats.
In traditional programming, you would have to manually define specific rules or features that differentiate dogs from cats. For instance, you might instruct the computer to look for features like the shape of the ears, the length of the tail, or the color of the fur.
You'd have to write detailed instructions for the computer to follow, such as "if the ears are pointed and the tail is long, then it's likely a cat." This process is akin to creating a detailed checklist for identifying different animal species based on their physical characteristics.
However, this approach has limitations. What if there are breeds of dogs with pointy ears similar to cats? What if the lighting in the image affects the appearance of fur color? It becomes increasingly complex to account for all possible variations.

Now, let's contrast this with machine learning. Instead of explicitly programming the rules for identifying dogs and cats, you provide the computer with a large dataset of labeled images—some containing dogs, some containing cats.
The machine learning algorithm then analyzes these images and identifies patterns on its own. It learns to recognize common features that distinguish dogs from cats, without you having to specify them explicitly. These features could be things like the shape of the snout, the presence of whiskers, or the texture of the fur.
Once the algorithm has been trained on enough examples, you can give it new images, and it will predict whether each one contains a dog or a cat based on the patterns it has learned. Just like how a child learns to differentiate between animals by seeing many examples, the machine learning model learns to distinguish between dogs and cats through exposure to a diverse range of images.

So, in essence, traditional programming involves manually defining rules based on explicit criteria, while machine learning allows the computer to learn from data and discover its own patterns for classification tasks like identifying images of dogs and cats.



SCENARIO 3: DATA MINING

Data mining is like searching for hidden treasures within a vast mine of data. It involves exploring large datasets to uncover patterns, correlations, and insights that may not be immediately apparent. Traditional data analysis techniques, such as plotting graphs and performing statistical analyses, are valuable tools in this process, but they have limitations when it comes to handling complex or unstructured data.

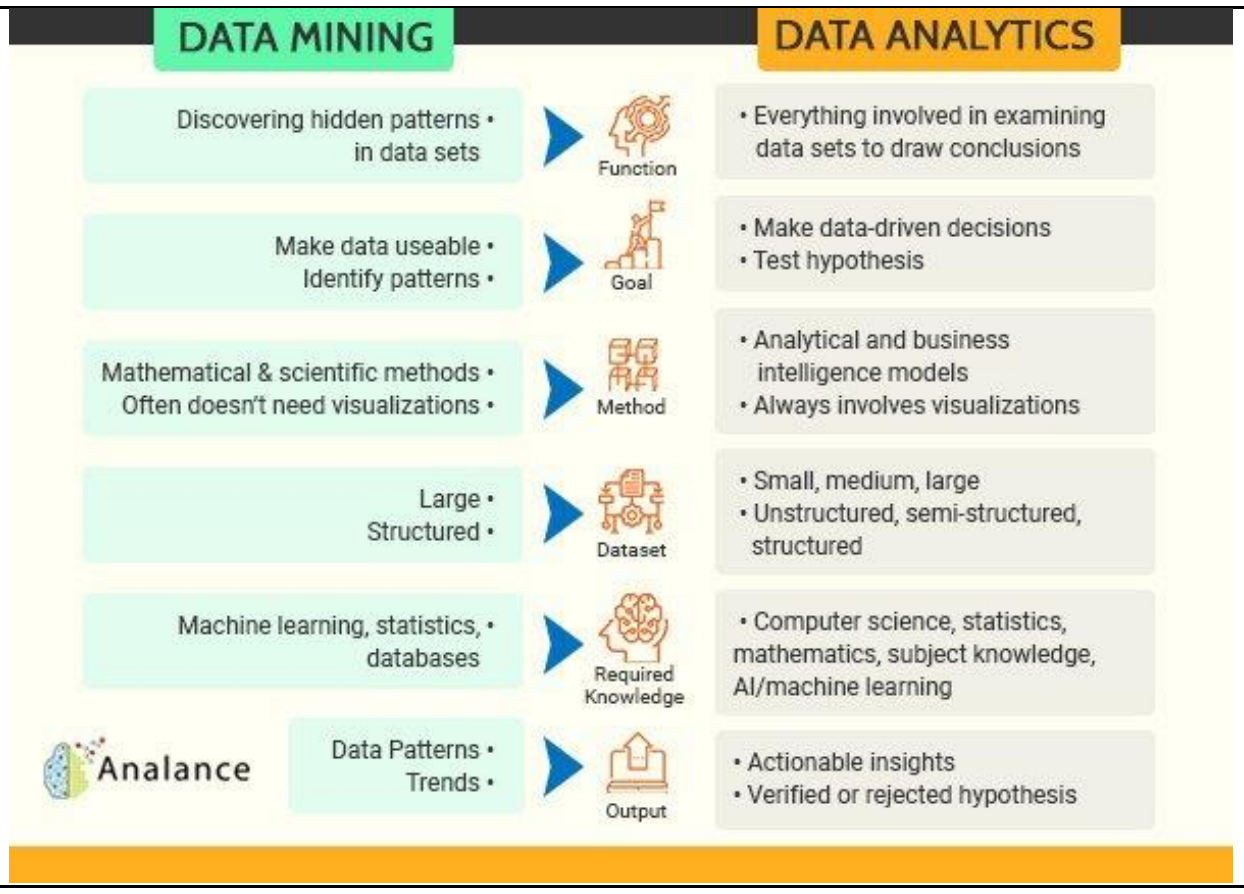
This is where machine learning comes into play. Machine learning algorithms excel at finding patterns and making predictions from data, even when those patterns are not readily apparent to human analysts. By applying machine learning to the process of data mining, we can uncover deeper insights and extract more valuable information from our datasets.

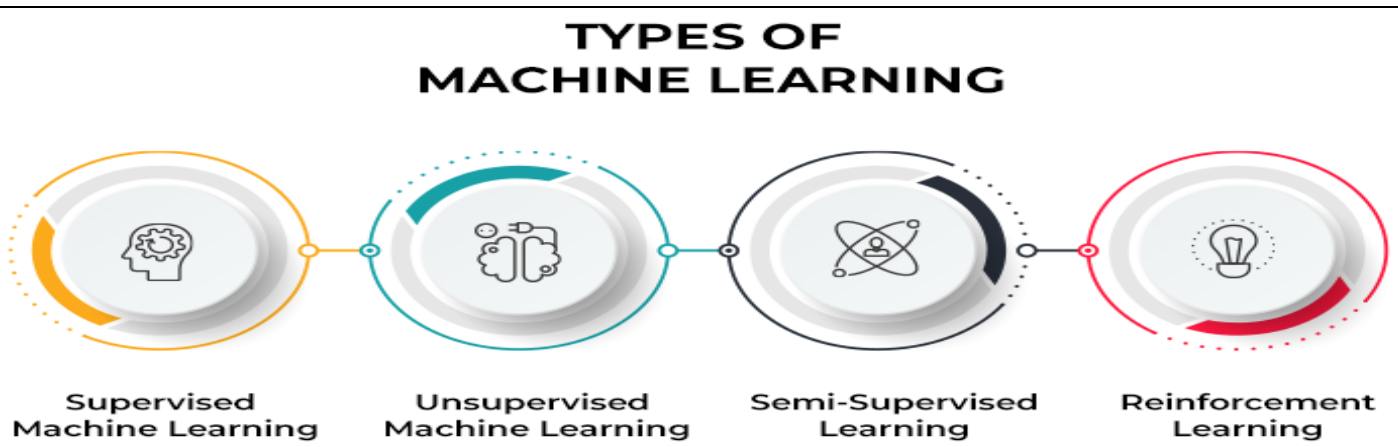
For example, let's say you have a dataset containing information about customer purchases at a retail store. Traditional data analysis techniques might allow you to identify basic trends, such as which products are selling well during specific seasons. However, by applying machine learning algorithms to this dataset, you could uncover more nuanced patterns, such as:

1. Identifying hidden customer segments based on purchasing behavior, allowing for targeted marketing strategies.
2. Predicting future purchasing trends and adjusting inventory levels accordingly to optimize supply chain management.
3. Detecting unusual patterns or anomalies in purchasing behavior that may indicate fraud or unusual activity.

By leveraging machine learning in the data mining process, organizations can extract more actionable insights from their data, leading to more informed decision-making and improved efficiency across various domains, including marketing, finance, healthcare, and more.

In summary, while traditional data analysis techniques are valuable for exploring and visualizing data, machine learning enhances the data mining process by uncovering hidden patterns and insights that may not be apparent through traditional methods alone.

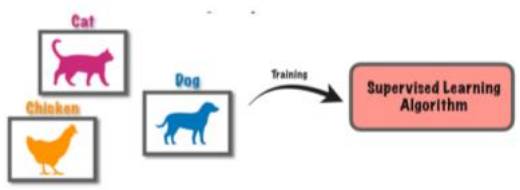




Supervised Learning

SUPERVISED LEARNING:

The model is trained on a labeled dataset, meaning that it learns to make predictions based on input-output pairs provided during training.



APPLICATIONS OF SUPERVISED LEARNING :

Input (A)	Output (B)	Application
email	spam? (0/1)	spam filtering
audio	text transcript	speech recognition
English	Chinese	machine translation
ad, user info	click? (0/1)	online advertising
image, radar info	position of other cars	self-driving car
image of phone	defect? (0/1)	visual inspection

Types of Supervised Learning

REGRESSION :

Regression is a type of supervised learning algorithm that models the relationship between a dependent variable (also known as the target) and one or more independent variables (also known as predictors or features). The goal of regression is to predict continuous numerical values. In regression, the algorithm learns to map input data to a continuous output space, which could represent quantities such as prices, temperatures, scores, etc.

EXAMPLE:

Suppose we have a dataset containing information about houses, including features like square footage, number of bedrooms, and location, as well as their corresponding sale prices. In this scenario, we want to predict the sale price of a house based on its features.

Input: Square footage, number of bedrooms, location (features)
Output: Sale price (continuous value)

Regression algorithms would learn from this data to predict the sale price of houses with similar features. For example, given the features of a new house (square footage, number of bedrooms, location), the regression model would output an estimated sale price of a new house.

IN SUMMARY:

- REGRESSION DEALS WITH PREDICTING CONTINUOUS NUMERICAL VALUES.
- CLASSIFICATION DEALS WITH CATEGORIZING DATA INTO PREDEFINED CLASSES OR LABELS.

CLASSIFICATION:

Classification is another type of supervised learning algorithm where the goal is to categorize input data into predefined classes or labels. Unlike regression, which predicts continuous values, classification predicts discrete labels or categories. The algorithm learns from labeled training data to classify new instances into one of the predefined classes. Classification is used for tasks such as spam detection, image recognition, sentiment analysis, medical diagnosis, and more.

EXAMPLE:

Now, let's consider a different scenario where we want to classify emails as either spam or not spam based on their content and metadata.

Input: Content, sender, subject, timestamp, etc. (features)
Output: Spam or not spam (binary label)

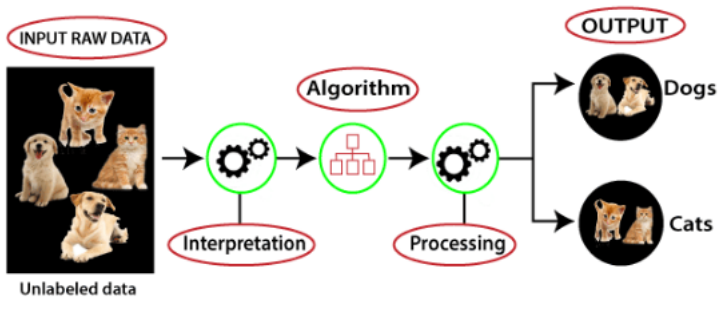
Classification algorithms would learn to classify emails into two categories: spam or not spam, based on the provided features. For instance, the algorithm might learn to identify patterns in the content, sender, subject, and other metadata associated with spam emails.

Example: Using classification, we could classify incoming emails as either spam or not spam based on their content, sender, subject, and other metadata. The output would be a binary label indicating whether the email is spam or not spam.

UNSUPERVISED LEARNING

UNSUPERVISED LEARNING:

The model learns patterns from input data without provided labels, identifying inherent structure or relationships within the data itself.

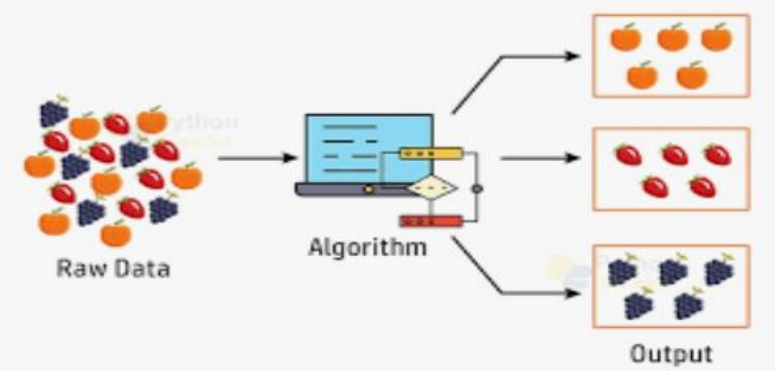


TYPES OF UNSUPERVISED LEARNING:

Types of Unsupervised Learning

Clustering:

Clustering is a technique used to partition a dataset into groups, or clusters, where data points within the same cluster are more similar to each other than to those in other clusters. The goal is to discover inherent groupings in the data without any prior knowledge of the class labels. Popular clustering algorithms include K-means clustering, hierarchical clustering, and DBSCAN (Density-Based Spatial Clustering of Applications with Noise).



Dimensionality Reduction:

Dimensionality reduction techniques aim to reduce the number of features (dimensions) in a dataset while preserving its important structure or relationships. This is particularly useful for high-dimensional datasets where the number of features is large, and some features may be redundant or irrelevant. Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE) are common dimensionality reduction techniques.

RollNo	Name	Mobile Number	Mobile Network
T4Tutorials1	Sameed	+92 302 XX XXX XX	Mobilink
T4Tutorials1	Ali	+92 333 XX XXX XX	Ufone

Figure 1: Before Dimension reduction

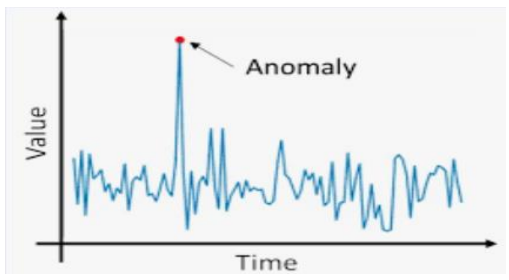
If we know Mobile Number, then we can know the Mobile Network. So we need to reduce the one dimension.

RollNo	Name	Mobile Number
T4Tutorials1	Sameed	+92 302 XX XXX XX
T4Tutorials1	Ali	+92 333 XX XXX XX

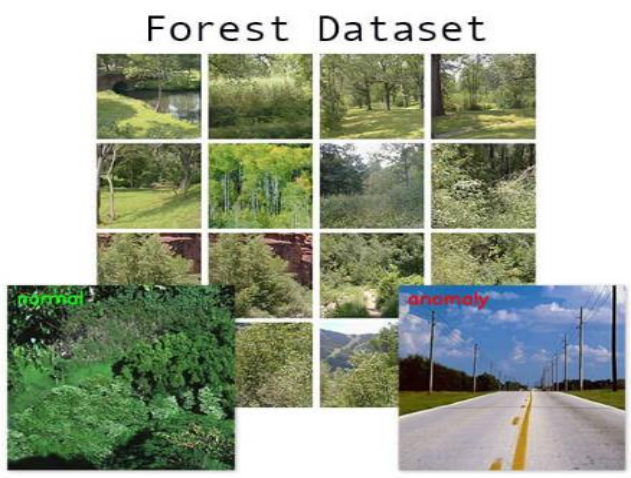
Figure 2: After Dimension reduction

Anomaly Detection:

Anomaly detection, also known as outlier detection, involves identifying unusual patterns or data points that do not conform to expected behavior. Anomalies may indicate errors, fraud, or interesting observations in the data. Unsupervised anomaly detection methods learn the normal patterns from the data and flag instances that deviate significantly from them. Techniques such as density-based outlier detection, isolation forests, and autoencoders are used for anomaly detection.



Anomaly Detection Example:



Association Rule Mining:

Association rule learning is used to discover interesting relationships, associations, or correlations among variables in large datasets. The goal is to identify patterns where the occurrence of certain events (items) in transactions is highly correlated. This is commonly used in market basket analysis, where the aim is to uncover associations between products that are frequently purchased together. Apriori algorithm and FP-growth algorithm are popular techniques for association rule learning.

Example:
Suppose we have a dataset representing transactions at a grocery store. Each transaction consists of a set of items purchased by a customer.

```
Transaction ID | Items Purchased
--------------------------------
1              | Bread, Milk
2              | Bread, Diapers, Beer, Eggs
3              | Milk, Diapers, Beer, Cola
4              | Bread, Milk, Diapers, Beer
5              | Bread, Milk, Diapers, Cola
```

Association Rules:
Next, we generate association rules from these frequent item sets, considering our minimum confidence threshold. Some example association rules might be:

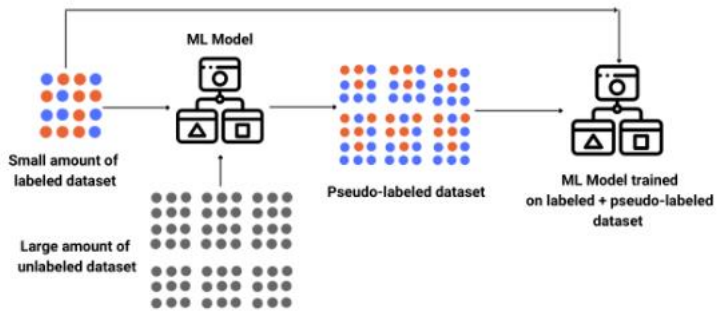
1. {Bread} ➞ {Milk} (Support: 60%, Confidence: 75%)
   This rule indicates that if a customer purchases Bread, they are likely to also purchase Milk.
2. {Milk} ➞ {Diapers} (Support: 40%, Confidence: 66.67%)
   This rule suggests that if a customer purchases Milk, they have a high chance of also purchasing Diapers.
3. {Beer} ➞ {Diapers} (Support: 40%, Confidence: 80%)
   This rule indicates that if a customer purchases Beer, they are very likely to also purchase Diapers.

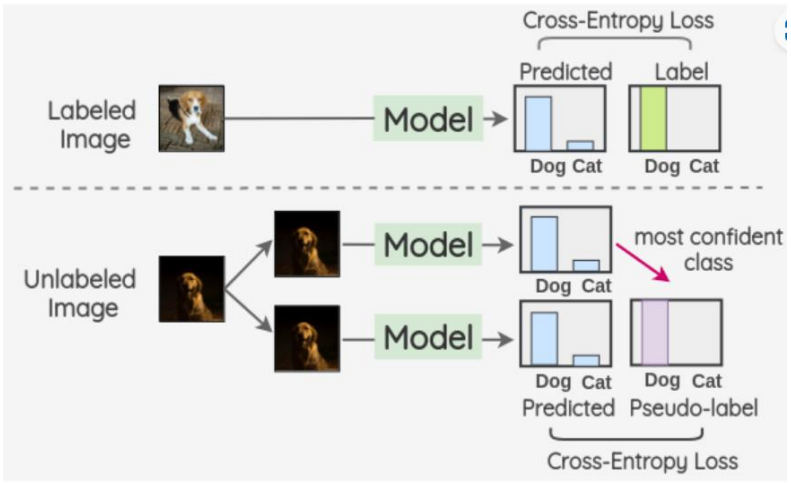
Interpretation:
From these association rules, we can derive actionable insights for the grocery store. For example:

- The store could place Bread and Milk closer together to encourage customers to purchase them together.
- The store could offer promotions or discounts on Beer and Diapers since they are frequently purchased together, potentially increasing sales.

Semi-Supervised Learning

The model combines a small amount of labeled data with a large amount of unlabeled data during training, leveraging the unlabeled data to improve learning accuracy.





Reinforcement Learning

An agent learns to make decisions by taking actions in an environment to maximize a reward signal, learning from the feedback of its actions over time.

Imagine you have a pet, like a dog, and you want to teach it a new trick, let's say "fetch". You start by showing the dog a toy and saying "fetch". At first, the dog might not understand what you're asking, so you give it a treat when it gets close to the toy. The dog doesn't know exactly what it did right, but it likes getting treats, so it tries different things until it gets another treat. Eventually, it figures out that bringing back the toy when you say "fetch" gets it a treat.

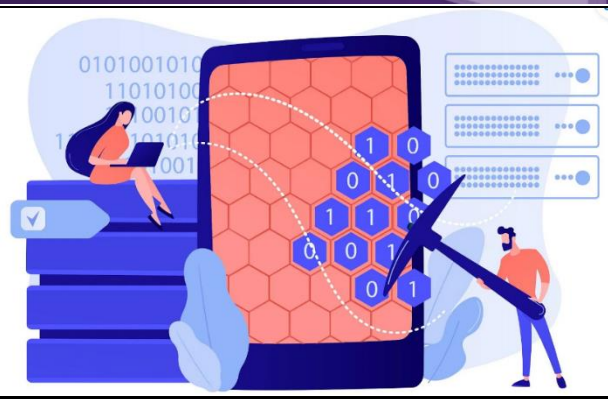
In this scenario:

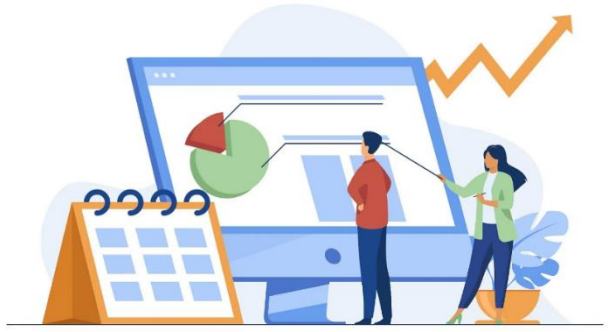
- The dog is the learner or "agent".
- You are the "environment" providing feedback (treats).
- "Fetch" is the action the dog takes.
- Getting a treat is the "reward".

Reinforcement learning is like teaching your dog a new trick. The learner (agent) tries different actions in an environment and gets feedback (rewards) based on how good or bad those actions are. Over time, the learner figures out the best actions to take in different situations to maximize its rewards.

It's like a trial-and-error process where the learner learns from its experiences to make better decisions in the future.

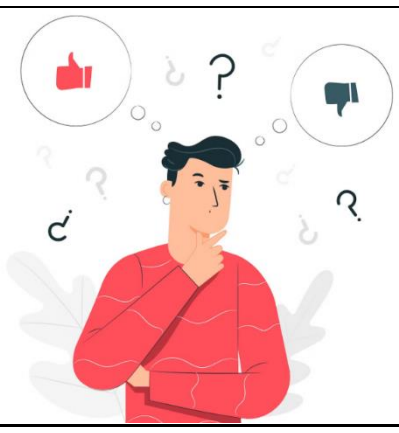
DATA COLLECTION: The first step is to gather data relevant to the problem you're trying to solve. This data could be anything from text documents, images, sensor readings, or transaction records, depending on the task. For instance, if you want to teach it to recognize cats in pictures, you'd gather lots of images of cats and images without cats.





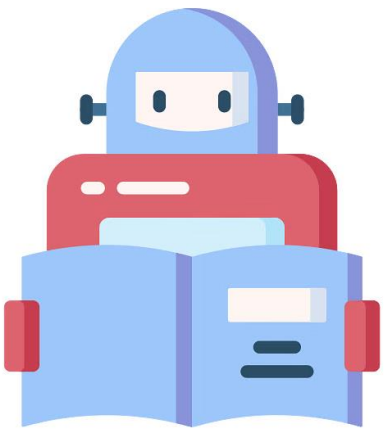
DATA PREPROCESSING: Once you have the data, it often needs to be cleaned and prepared for training. This may involve tasks like removing duplicates, handling missing values, scaling numerical features, and encoding categorical variables.

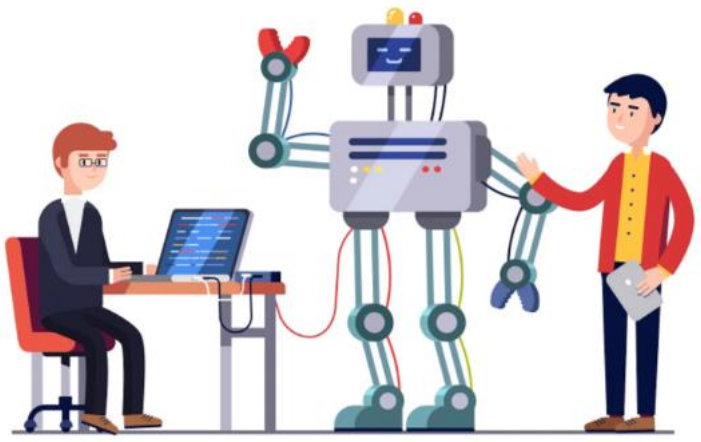
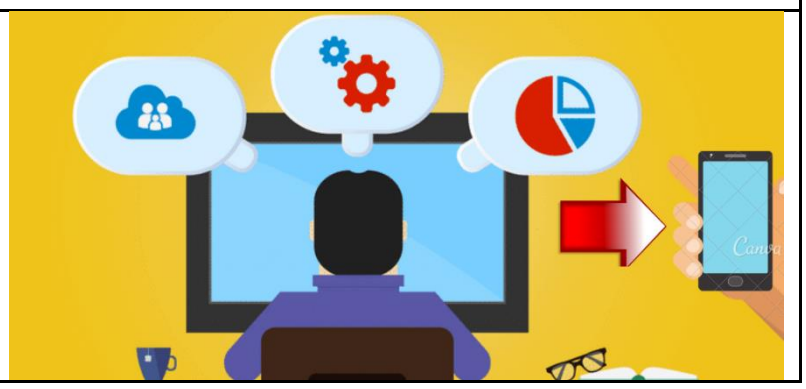
FEATURE ENGINEERING: Feature engineering is the process of selecting, transforming, or creating new features from the raw data to improve the performance of the model. This step can involve domain knowledge and creativity.



MODEL SELECTION: Choose an appropriate model or algorithm for your problem. This could be a decision tree, support vector machine, neural network, or one of many other algorithms available in the ML toolbox. The choice depends on factors like the type of data, the complexity of the problem, and computational resources.

MODEL TRAINING: Train the selected model on your prepared dataset. You feed the prepared data into the model and let it learn from the examples. During training, the model learns patterns and relationships in the data. This also involves adjusting the model's parameters to minimize the difference between its predictions and the actual target values in the training data, to get better and better at the task over time. It's like showing the computer lots of examples and saying, "This is a cat, this is not a cat, this is a cat," and so on.



<table>
<tr>
<td>  </td>
<td>EVALUATE THE MODEL: Once the model has been trained on the data, you test it to see how well it performs. You give it new examples it hasn't seen before and see if it can correctly predict or classify them. If it does well, great! If not, you might need to go back and tweak things or get more data.</td>
</tr>
<tr>
<td>DEPLOY THE MODEL: If the model performs well on the test data, you can deploy it to do real-world tasks. This could involve integrating the model into a software application, a web service, or an automated pipeline. For example, you could use it to automatically classify pictures on social media, suggest movies you might like, or detect fraudulent transactions.</td>
<td>  </td>
</tr>
</table>

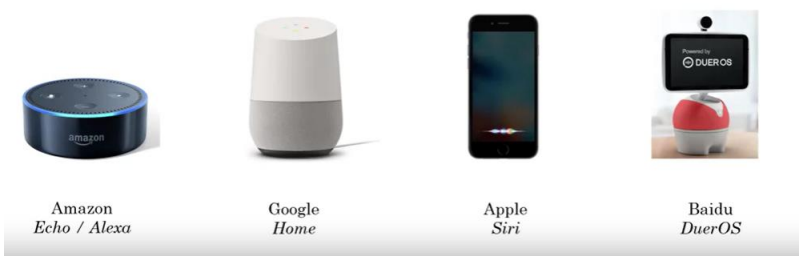
Machine Learning Workflow

Let's go through the key steps of a machine learning project.
This will not be a standard step by step process, rather a simplified version of ML workflow.

Example 1: Speech Recognition

Let's consider an example of speech recognition.
Some of you may have an Amazon Echo or Google Home or Apple Siri device or a Baidu DuerOS device in your homes.

So, how do you build a speech recognition system that can recognize when you say, "Alexa," or "Hey, Google," or "Hey, Siri," or "Hello, Baidu"?



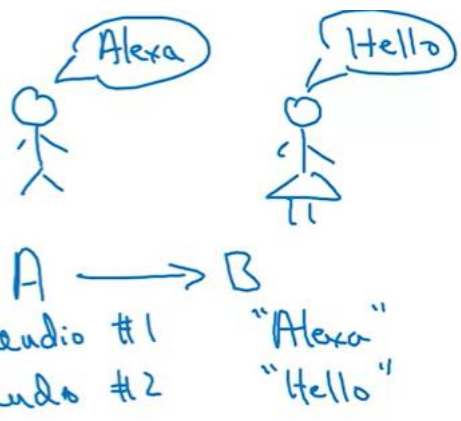
Example 2: Self driving cars

Let's say you're building a self-driving car.

One of the key components of a self-driving car is a machine learning algorithm that takes as input, example, a picture, of what's in front of your car and tells you where the other cars are.

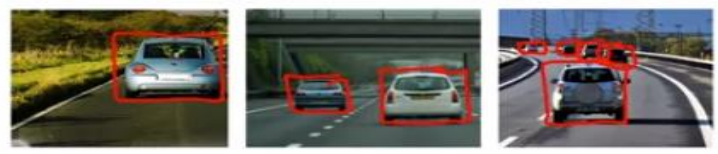
Collect Data (Data Gathering)

If you want to build an AI system or build a machine learning system to figure out, when a user has said the word “Alexa”, the first step is to collect data.
That means, you would go around and get some people to say the word "Alexa" for you and you record the audio of that.
You'll also get a bunch of people to say other words like "Hello," or say lots of other words and record the audio of that as well.



If your goal is to have a machine-learning algorithm that can take an image as an input, and output the position of other cars, then the data you would need to collect would be both images as well as position of other cars that you want the AI system to output.

So, let's start off with few pictures. For each of these pictures, you would draw a rectangle around the cars in the picture that you wanted to detect. You will use some software that lets you draw rectangles.



Train the Model

Having collected a lot of audio data, a lot of these audio clips of people saying either "Alexa" or saying other things like “Hello”,
step two is to then **train the model.**
This means you will use a machine-learning algorithm to learn an input to output mapping, where the input would be an audio clip.

In the case of the first audio clip above, it will tell you that the user said "Alexa," and in the case of audio clip two, the system will learn to recognize that the user has said "Hello."

Whenever an AI team starts to train the model, what happens, is the first attempt doesn't work well.
So invariably, the team will need to iterate many times until, the model is good enough.

When your AI engineers start training a model, they'll find, that initially, it doesn't work that well.

For example, given this picture, maybe the software, in the first few tries, thinks THAT (tree) is a car.

It's only by iterating many times, that you get a better result when figuring out that THIS is where the car is.

Deploy the Model

The third step is to - deploy the model.
You put this AI software into an actual smart speaker and ship it to either a small group of test users or to a large group of users.
What happens in a lot of AI products is that when you ship it, you see that it starts getting new data and it may not work as well as you had initially hoped.

For example, Let's say, I am from the UK. So, I'm going to pick on the British accent.
But what if you had trained your speech recognition system on American-accented speakers and you then ship this smart speaker to the UK and you start having British-accented people say "Alexa." They may find that it doesn't recognize the speech.
You can get the data back, of such cases and then use this data to maintain and to update the model.

In the self-driving world, it's important to treat safety as number one and deploy model or to test the model only in ways that can preserve safety.

For example, you may find that there are new types of vehicles, say golf carts, that the software isn't detecting very well.

You get the data back – get pictures of golf carts, using new data, to maintain and update the model so that you can have your AI software continually get better and better, to the point where the model can do a pretty good job detecting other cars from pictures like these.

So, to summarize, the key steps of a machine learning project are to collect data, to train the model, and then to deploy the model.
Throughout these steps, there is often a lot of iteration, fine-tuning or adapting the model to make the product better.

Data is revolutionizing various job roles, spanning industries such as recruitment, sales, marketing, manufacturing, and agriculture. Chances are that data is reshaping your role as well. Over the past few decades, our society has undergone a digital transformation. Instead of traditional paper surveys, surveys are now predominantly conducted in digital formats. Similarly, while some doctors may still jot down handwritten notes, these notes are increasingly being digitized. This trend extends to nearly every job function. With the abundance of data available, there's a high probability that tools like data science and machine learning can enhance or streamline your job function.

Let's explore how data science and machine learning can impact a wide range of job roles.

Data Science	Machine Learning

JOB FUNCTION: SALES

Previously, we explored how data science can optimize sales funnels.

The sales funnel of the e-commerce website involves several steps: visiting the website, viewing product pages, adding items to the shopping cart, and checking out. Data science is employed to collect and analyze data related to user behavior at each step of the sales funnel. This includes information such as user location, browsing patterns, and purchasing behavior. By analyzing this data, the data science team identifies factors that impact the performance of the sales funnel, such as international shipping costs, holiday seasons, and time-of-day fluctuations in online shopping activity. Insights derived from the data analysis are used to formulate hypotheses and suggest actions to optimize the sales funnel, such as adjusting shipping costs or advertising strategies. These suggestions are implemented on the website, and new data is collected to evaluate their effectiveness. Data science continues to monitor and analyze the data periodically, allowing for ongoing optimization of the sales funnel.

Now, let's delve into machine learning's role.

As a salesperson, you likely manage a list of leads, each representing potential customers. Machine learning offers the ability to prioritize these leads effectively. For instance, it can suggest reaching out to the CEO of a large corporation over an intern at a smaller company. This **automated lead sorting** enhances sales efficiency, allowing you to focus on high-potential prospects.

Name	Title	Company size	Email	Priority
Taylor	CEO	3050	tay@a..	high
Janet	Manager	230	jan@b..	medium
David	Intern	30	dave@c..	low

JOB FUNCTION: MANUFACTURING

The manufacturing process for coffee mugs involves several key steps: mixing clay, shaping mugs, adding glaze, firing the kiln, and inspecting the final product. One common challenge in manufacturing is to minimize the production of damaged mugs to reduce waste of time and materials. Data science is utilized to collect and analyze data from various stages of the manufacturing process, such as clay mixing parameters, kiln firing conditions, and environmental factors. By analyzing this data, the data science team identifies patterns and insights that help improve the manufacturing process, such as adjusting humidity and temperature settings to prevent cracks in the mugs. These insights lead to hypotheses and actions to optimize the productivity of the manufacturing line, ultimately resulting in fewer defective mugs and improved efficiency.

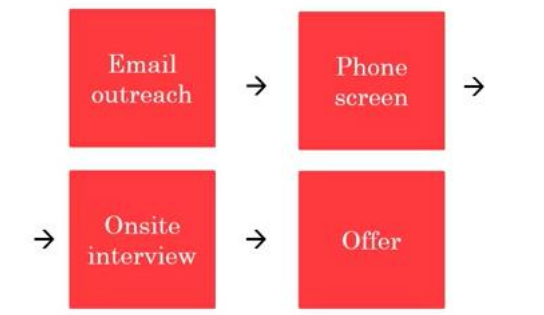
How about machine learning?

In today's manufacturing processes, a crucial step involves final inspection. Traditionally, this task relies on human visual inspection, often involving hundreds or thousands of workers scrutinizing items such as coffee mugs for defects like scratches or dents. However, machine learning can revolutionize this process. By analyzing datasets of such inspections, machine learning algorithms can learn to automatically detect defects in items like coffee mugs. This automated visual inspection not only reduces labor costs but also enhances product quality in your factory. It's a transformative technology poised to make a significant impact on manufacturing.

JOB FUNCTION: RECRUITING

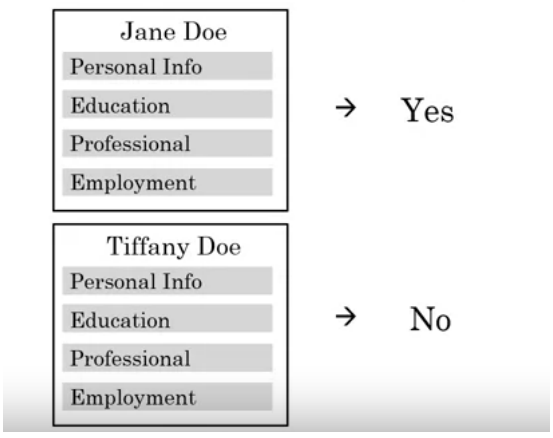
When it comes to recruiting candidates for your company, there's typically a structured process involving steps like sending emails, conducting phone interviews, hosting on-site interviews, and extending offers. Just like data science can optimize sales funnels, it can also enhance recruiting processes. Many recruiting organizations are leveraging data science to refine their recruiting pipelines.

For example, if you find that hardly anyone is making it from the phone screen step to the on-site interviews step, then you may conclude that maybe too many people are getting into the phone screen stage or maybe the people doing the phone screen are just being too tough and you should let more people get to the onsite interview stage.



What about machine learning projects?

Part of the recruiting process involves screening numerous resumes to determine which candidates to contact.
So, you may have to look at one resume and say, "Yes, let's email them", and look at a different one to say, "No, let us not move ahead with this candidate."
Machine learning is starting to make its way into automated resume screening.
However, it's essential to address ethical concerns, ensuring that AI software remains unbiased and treats all individuals fairly.



AGRICULTURE

Let’s say you're a farmer working on the light industrial farm, how can data science help you?

Today farmers are already using data science for crop analytics, where you can take data on the soil conditions, the weather conditions, the presence of different crops in the market and have data science teams make recommendations to what to plant, when to plant, while maintaining the condition of the soil on your farm.

This type of data science is and will play a bigger and bigger role in agriculture.

Let's also look at the machine learning example.
One of the most exciting changes to agriculture is precision agriculture.
Here's a picture from one of the farms.
On the upper right is a cotton plant and shown in the middle is a weed.
(Weeds are the unwanted plants that grow in the garden or fields of crops and prevent the growth of desirable plants)

With machine learning, we're starting to see products that can go onto the farms, take a picture like the one below and spray a weed killer in a very precise way, just onto the weeds, so that it gets rid of the weed but without having to spray an excessive amount of weed killers.

This type of machine learning technology is both helping farmers increase crop yields while also helping to preserve the environment.

CHAPTER 04

DEEP LEARNING

What is Deep Learning?

TECHNICAL DEFINITION

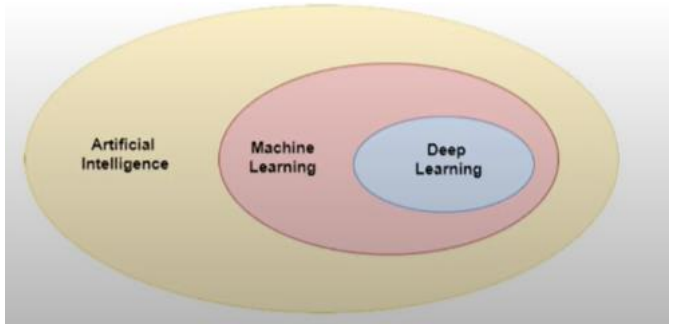
Deep learning is a type of AI that teaches computers to process data in a way that mimics the human brain. Deep learning algorithms use artificial neural networks. These algorithms use multiple layers of nonlinear processing units to learn hierarchical feature representations directly from data.

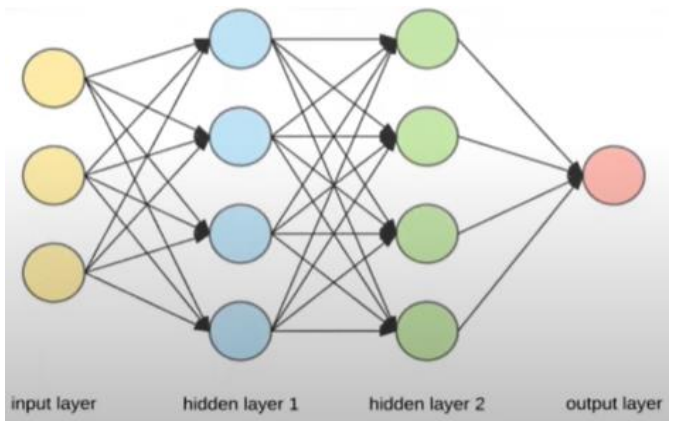
"When you hear the term deep learning, just think of a large deep neural net."

Today, the terms neural network and deep learning are used almost interchangeably, they mean essentially the same thing.

A FEW MORE DEFINITIONS :

- Deep Learning is a subfield of AI and ML that is ***inspired*** by the structure of a human brain.
- Deep Learning algorithms attempt to draw similar conclusions as humans would, by continually analyzing data with a given logical structure called Neural Network
- Deep learning is a part of a broader family of machine learning methods based on ANN (Artificial Neural Networks) with representation learning.





- Deep learning algorithms uses multiple layers to progressively extract higher level features from the raw input.
  For example, in image processing, lower layers may identify edges, while higher layers may identify the concepts relevant to a human such as digits or letters or faces.

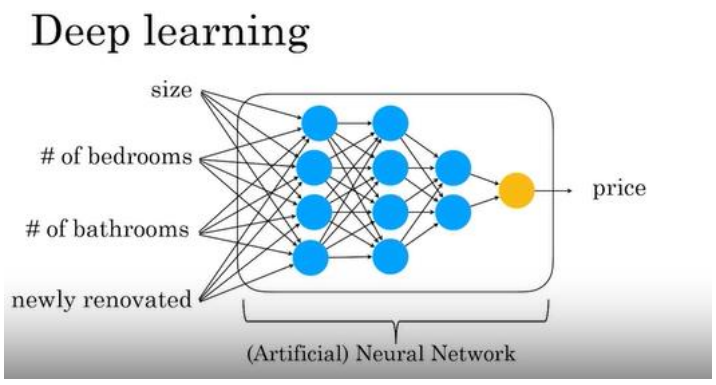
What is a Neural Network ?

A neural network is a machine learning program, or model, that makes decisions in a manner similar to the human brain, by using processes that mimic the way biological neurons work together to identify phenomena, weigh options and arrive at conclusions.

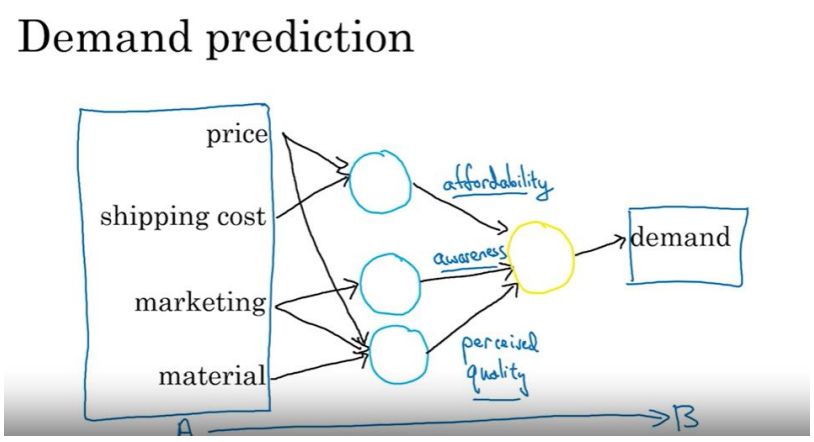
Every neural network consists of layers of nodes, or artificial neurons—an input layer, one or more hidden layers, and an output layer. Each node connects to others and has its own associated weight and threshold. If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network.

Neural networks rely on training data to learn and improve their accuracy over time. Once they are fine-tuned for accuracy, they are powerful tools in computer science and artificial intelligence, allowing us to classify and cluster data at a high velocity. Tasks in speech recognition or image recognition can take minutes versus hours when compared to the manual identification by human experts. One of the best-known examples of a neural network is Google's search algorithm.

Neural networks are sometimes called artificial neural networks (ANNs) or simulated neural networks (SNNs). They are a subset of machine learning, and at the heart of deep learning models.



EXAMPLE



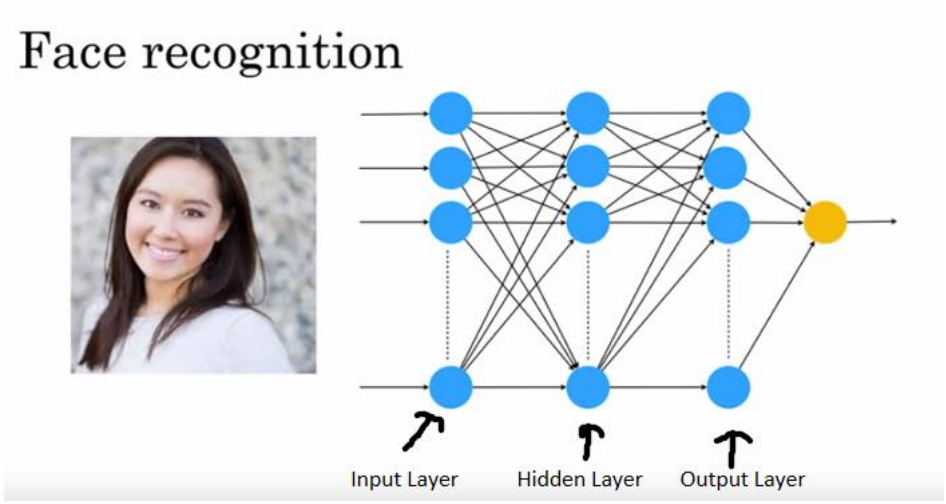
Let’s look at an example on how can the neural network look at the picture and figure out what's in the picture?

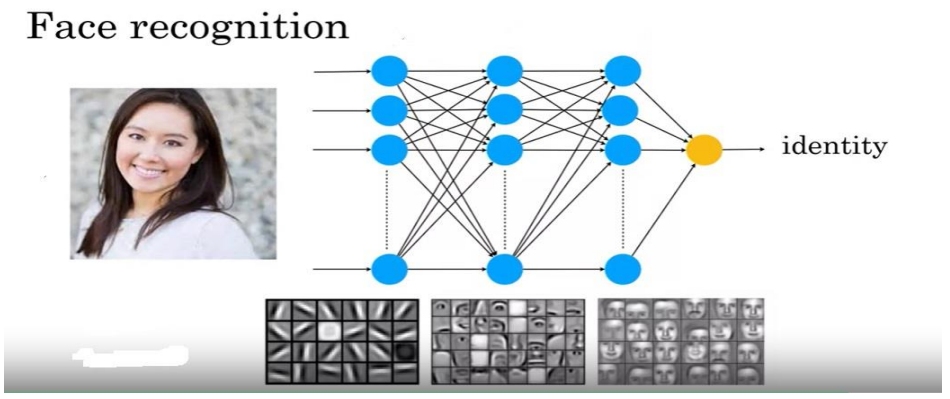
Say, you want to build a system to recognize people from pictures - how can a piece of software look at this picture and figure out the identity of the person in it?

It may first identify only low-level features, like some edges in the first layer, and in the second layer, it would identify mid-level features, like some shapes. And in the third layer, they would start identifying high level features like a complete face structure.

All the feature identification happens automatically in Deep learning.

A deeper network has more hidden layers between the input and output layers. Increasing the depth of the network can lead to better accuracy of prediction, up to a certain point. This is because deeper networks can learn more complex representations of the data. With more layers, the network can capture intricate patterns and relationships in the data, making it more powerful in capturing the underlying structure.



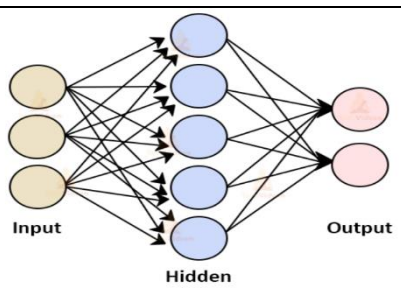


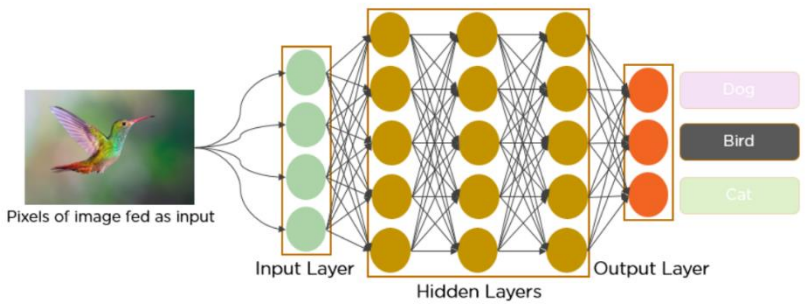
Types of Neural Networks

There are several types of neural networks, each designed to address specific types of problems or to exploit characteristics of data.
Here are some of the most common types:

Feedforward Neural Networks (FNN):

These are the simplest form of neural networks where information flows in **one direction**, from input nodes through hidden nodes (if any) to the output nodes.
Commonly used for tasks like regression and classification.



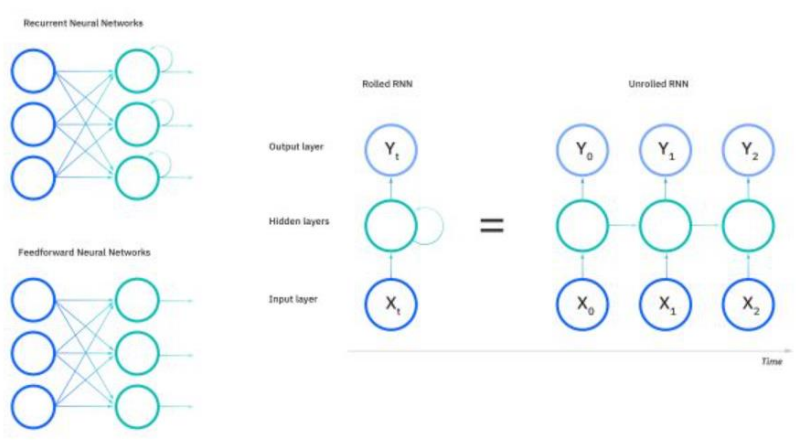


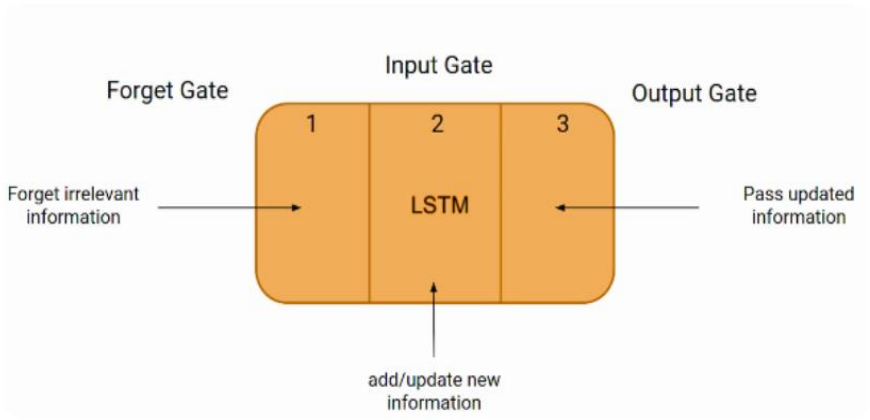
Convolutional Neural Networks (CNN):

Primarily designed for processing grid-like data, such as images or videos.
They use convolutional layers to automatically and adaptively learn spatial hierarchies of features from the input.
Highly effective in tasks like image recognition, object detection, and image segmentation.

Recurrent Neural Networks (RNN):

Specifically designed to work with sequence data, where the output not only depends on the current input but also on past inputs in the sequence.
They have loops in their architecture, allowing information to persist.
Widely used in tasks like language modeling, speech recognition, and time series prediction.





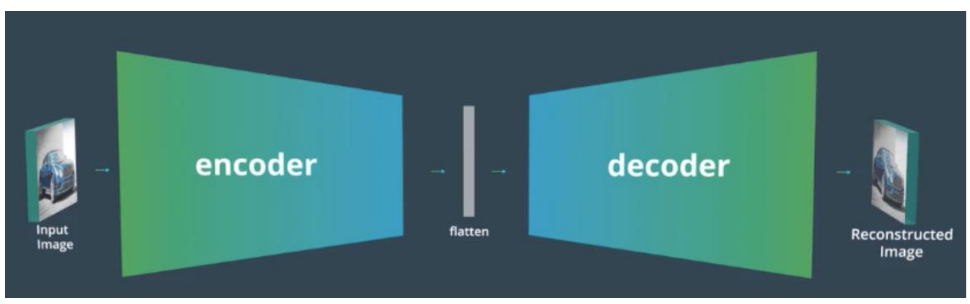
Long Short-Term Memory Networks (LSTM):

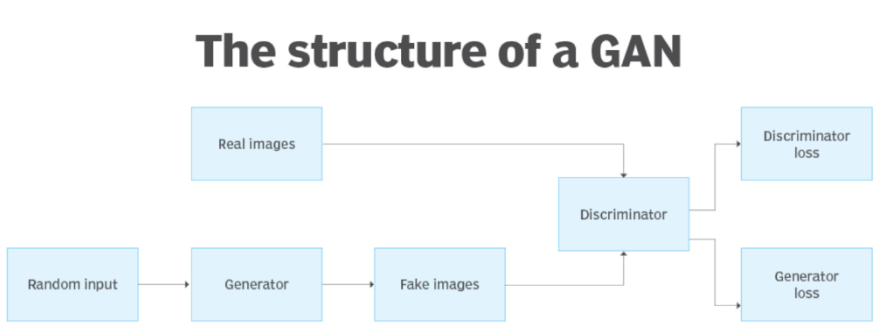
A type of RNN designed to address the vanishing gradient problem, allowing for learning and retaining information over long sequences.
They have a more complex architecture than traditional RNNs, with gated cells that control the flow of information.
Particularly useful in tasks where context over long sequences is crucial, such as language translation, sentiment analysis, and speech recognition.

Let's say while watching a video, you remember the previous scene, or while reading a book, you know what happened in the earlier chapter. RNNs work similarly; they remember the previous information and use it for processing the current input. The shortcoming of RNN is they cannot remember long-term dependencies due to vanishing gradient. LSTMs are explicitly designed to avoid long-term dependency problems.

Autoencoders:

Neural networks designed for unsupervised learning, where the input is first compressed into a latent-space representation and then reconstructed back to the original input.
They consist of an encoder network for compression and a decoder network for reconstruction.
Commonly used for tasks like data denoising, dimensionality reduction, and anomaly detection.





Generative Adversarial Networks (GANs):

Comprising two neural networks—the generator and the discriminator—that are trained simultaneously in a game-like setting.
The generator learns to generate data that is indistinguishable from real data, while the discriminator learns to differentiate between real and fake data.
Used for generating realistic synthetic data, image-to-image translation, and other generative tasks.

Technique	Primary Application	Description
Feedforward Neural Networks (FNN)	General tasks	Standard neural network with inputs passed directly to the outputs.
Convolutional Neural Networks (CNN)	Image processing & recognition	Neural networks optimized for processing grid-like data such as images.
Recurrent Neural Networks (RNN)	Sequence data (e.g., time series, text)	Networks with loops, allowing information to persist over sequential data.
Long Short-Term Memory (LSTM)	Sequence data & time series	A type of RNN better at learning long-term dependencies.
Generative Adversarial Networks (GAN)	Generating new data	Two-network setup: one generates data, the other discriminates genuine from generated.
Transformer Networks	Natural Language Processing (NLP)	Focus on attention mechanisms, revolutionizing NLP tasks.
Autoencoders	Data compression & denoising	Neural networks that aim to reconstruct their input data.

Machine Learning Vs Deep Learning

SIMILARITIES:

Before we get into the differences, lets talk about how similar Deep learning and Machine learning are.

- Both deep learning and machine learning involve the process of **learning from data** to make predictions or decisions. They rely on mathematical models and algorithms to extract patterns, relationships, and insights from input data, enabling intelligent behavior and automated decision-making.
- In both deep learning and machine learning, models are **trained using algorithms** that adjust model parameters based on observed data. During the training process, the model learns to generalize from training examples and make accurate predictions on unseen data.
- Once trained, both deep learning and machine learning models can be **used for prediction and inference tasks**. They take input data and generate output predictions or classifications based on learned patterns and relationships. Whether it's recognizing objects in images, translating text between languages, or predicting stock prices, both deep learning and machine learning models can perform a wide range of tasks.
- Both deep learning and machine learning **exhibit flexibility and adaptability** to different types of data and tasks. They can be applied to structured and unstructured data, including images, text, audio, and tabular data. Moreover, they can be adapted to various domains and applications, from healthcare and finance to automotive and entertainment.
- Both deep learning and machine learning have found applications across a wide range of industries and domains. They are used in areas such as computer vision, natural language processing, speech recognition, recommendation systems, predictive analytics, and autonomous systems. Whether it's powering virtual assistants, detecting fraud, or optimizing supply chains, both deep learning and machine learning play critical roles in driving innovation and solving real-world problems.

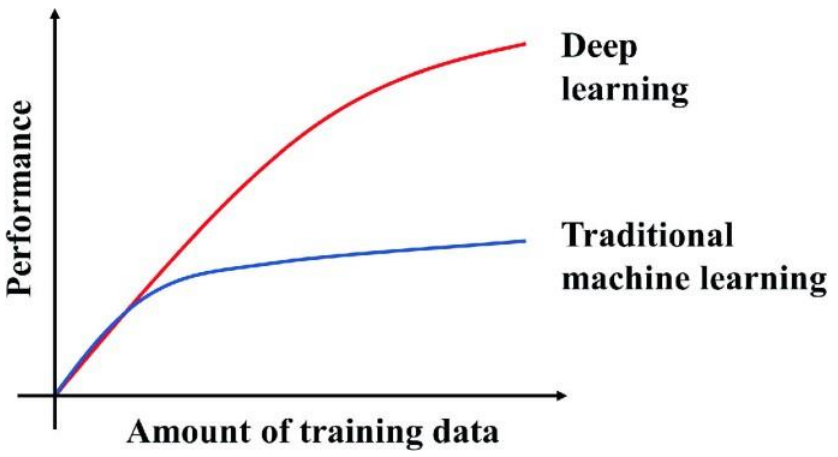
DIFFERENCES:

While deep learning and machine learning share many similarities, they also have distinct characteristics and differences in terms of model architectures, training techniques, and interpretability.

1. DATA DEPENDENCY

Deep Learning: Deep learning models are highly data-dependent and often require large volumes of labeled data to learn complex patterns and representations directly from raw input. These models excel at tasks involving unstructured data such as images, text, and audio, where the underlying patterns may be nonlinear and hierarchical.

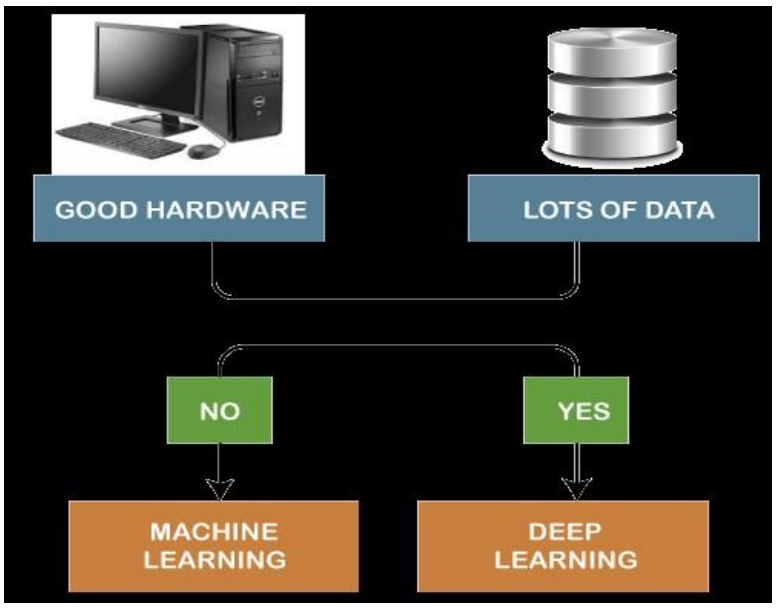
Machine Learning: Machine learning models are also data-dependent but may be more flexible in terms of data requirements. While some machine learning algorithms, such as decision trees and random forests, can handle structured data with relatively small sample sizes, others, like deep learning, may struggle without sufficient data for training.



2. HARDWARE DEPENDENCY:

Deep Learning: Deep learning models are computationally intensive and often require specialized hardware, such as GPUs (Graphics Processing Units) or TPUs (Tensor Processing Units), to accelerate training and inference tasks. These hardware accelerators are optimized for matrix operations and parallel processing, which are common in deep learning computations.

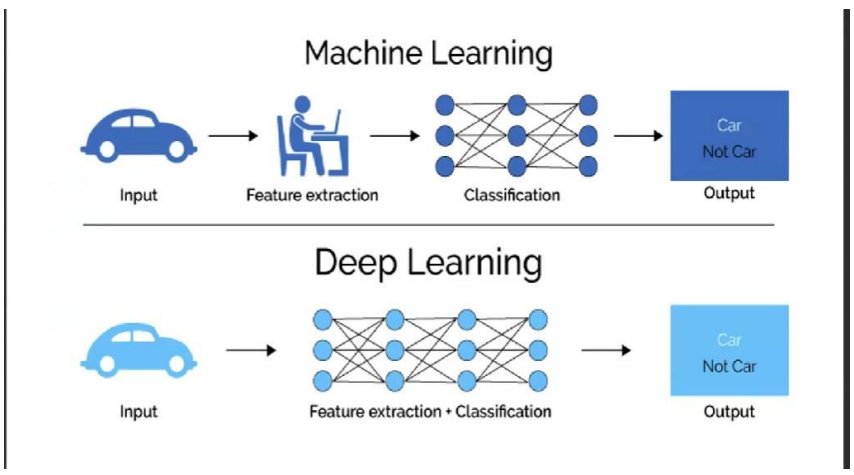
Machine Learning: Machine learning models typically have lower hardware requirements compared to deep learning models. While certain machine learning algorithms may benefit from parallel processing and optimized hardware, many can be trained and deployed on standard CPUs (Central Processing Units) without significant performance degradation.



3. Training Time:

Deep Learning: Deep learning models generally require longer training times compared to traditional machine learning models. This is primarily due to the complex architectures and large parameter spaces of deep neural networks. Training deep learning models on large datasets may involve multiple iterations (epochs) of forward and backward passes, which can be computationally intensive and time-consuming.

Machine Learning: Machine learning models often have shorter training times compared to deep learning models, especially for simpler algorithms and smaller datasets. While training times may vary depending on the complexity of the model and the size of the dataset, machine learning algorithms generally converge faster than deep learning algorithms.



4. Feature Selection:

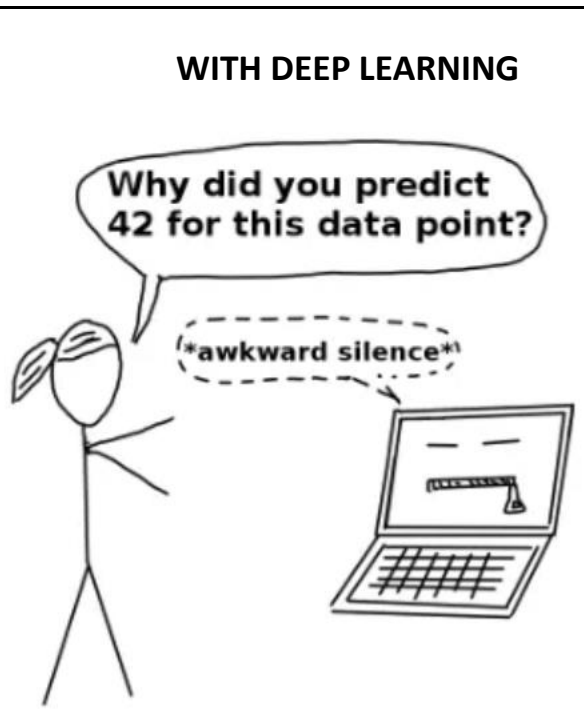
Deep Learning: Deep learning models automatically learn hierarchical features and representations from raw input data, eliminating the need for manual feature engineering. These models extract relevant features directly from the data during the training process, which can be advantageous for tasks involving high-dimensional and unstructured data.

Machine Learning: Traditional machine learning models typically require manual feature engineering, where domain experts identify and extract relevant features from the data. Feature selection involves choosing or engineering informative features that capture the underlying patterns and relationships in the data. While this approach offers interpretability and control over feature selection, it may be labor-intensive and prone to human biases.

5. Interpretability:

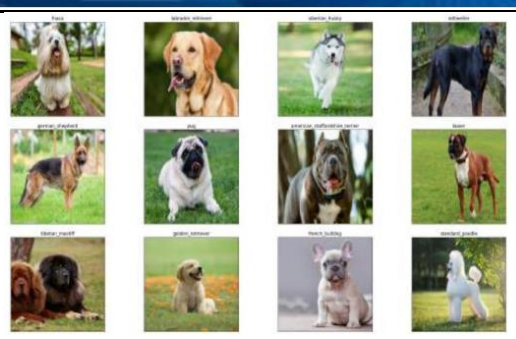
Deep Learning: Deep learning models are often criticized for their lack of interpretability due to their complex architectures and black-box nature. Understanding how deep neural networks arrive at their predictions can be challenging, especially for deep architectures with millions of parameters. While techniques such as visualization and feature attribution methods can provide insights into model behavior, deep learning models are generally less interpretable compared to traditional machine learning models.

Machine Learning: Traditional machine learning models are often more interpretable than deep learning models, especially for linear models and decision trees. These models provide transparency into the decision-making process, allowing users to understand the importance of individual features and the rationale behind predictions. Interpretable models are particularly valuable in domains where transparency, accountability, and regulatory compliance are critical factors.



DATA COLLECTION:
The first step is to gather a lot of data related to the problem you want your AI model to solve.

For example, imagine you want to train a model to recognize different types of dog breeds. You'll need a large collection of images labeled with the corresponding breeds.



DATA PREPROCESSING:
Prepare your data for training by cleaning and organizing it.
The raw data might have inconsistencies.

For instance, images may have different sizes or color formats. Preprocessing ensures uniformity. This could involve resizing images, converting colors to a standard format, or removing irrelevant information. This might also involve tasks like removing noise from images, converting text to a standard format, or normalizing numerical data.

In the dog breed example, you might resize all images to a specific size and convert them to RGB (Red, Green, Blue) format.
Clean up the images by removing any irrelevant background, resizing them to a standard size (e.g., 224x224 pixels), and normalizing the pixel values to be between 0 and 1.

DATA SPLITTING:
Divide your data into three sets: training data, validation data, and test data. The training data is used to teach the model, the validation data is used to fine-tune its parameters, and the test data is used to evaluate its performance.

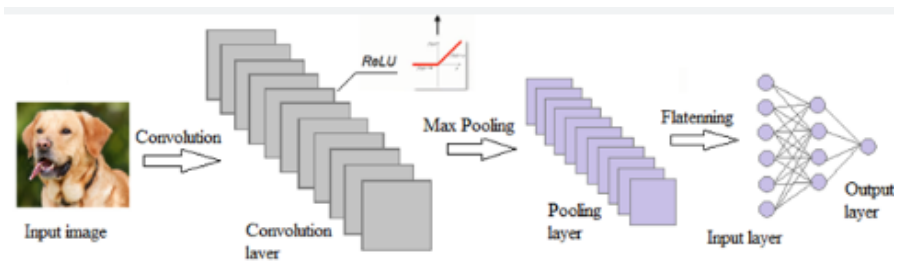
In the dog example, you might split your data into 80% for training and 20% for testing. You might also oversample rare dog breeds to create a more balanced dataset.

Training Data

Testing Data

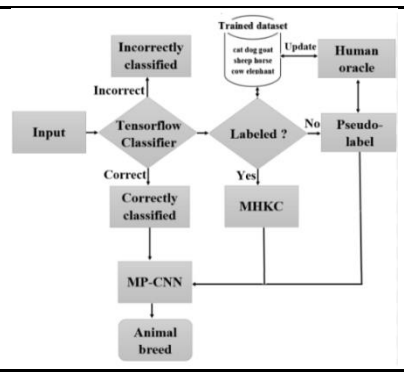
MODEL BUILDING:
Choose a suitable deep learning architecture for image classification tasks, such as a convolutional neural network (CNN). CNNs are well-suited for analyzing visual data and are commonly used in image recognition tasks.

MODEL COMPILATION: Compile your model by specifying the loss function, optimizer, and evaluation metrics. The loss function measures how well the model is performing, the optimizer adjusts the model's parameters to minimize the loss, and the evaluation metrics are used to assess its performance.

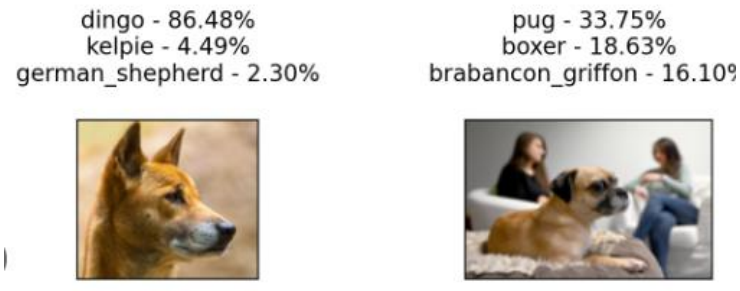
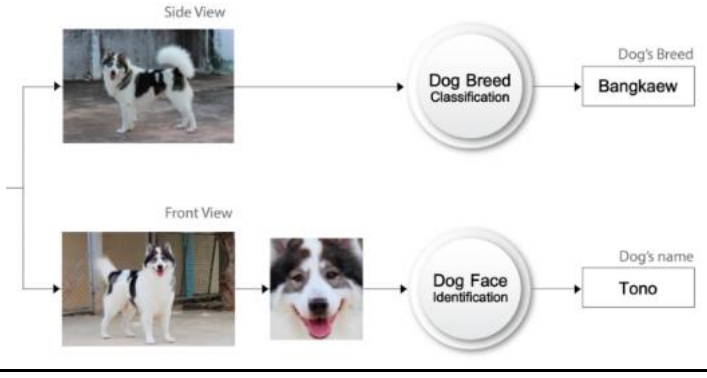


Construct the architecture of your deep learning model by defining its layers, activation functions, and other parameters. This is typically done using deep learning frameworks like **TensorFlow, PyTorch, or Keras.**

MODEL TRAINING:
During training, the model iterates through the training data, adjusting its internal parameters (weights and biases) to minimize the error between its predictions and the actual labels. This process is like the model constantly learning and refining its ability to recognize dog breeds.



EVALUATION & HYPERPARAMETER TUNING :
Once training is complete, evaluate your model's performance on the data to assess its real-world performance. This provides an unbiased estimate of how well your model will perform in production.

Model's performance is evaluated using the validation set. Fine-tune the hyperparameters of your model, such as the learning rate, batch size, and number of layers, to optimize its performance on the validation data. Here, the model makes predictions on unseen data, and its accuracy (how well it predicts the correct breed) is measured.	 In the dog example, you'd use the validation set to see how well the model identifies different dog breeds in images it hasn't seen before.
DEPLOYMENT: Once you're satisfied with the model's performance, you can deploy it for real-world use. This could involve integrating it into an app that allows users to upload dog images and get breed identifications.	

Deep learning's popularity stems from its exceptional performance in tasks like image and speech recognition, fueled by the abundance of big data and advances in computational power. Algorithmic breakthroughs, flexible architectures, and widespread industry adoption have further solidified its status as a dominant approach in machine learning.

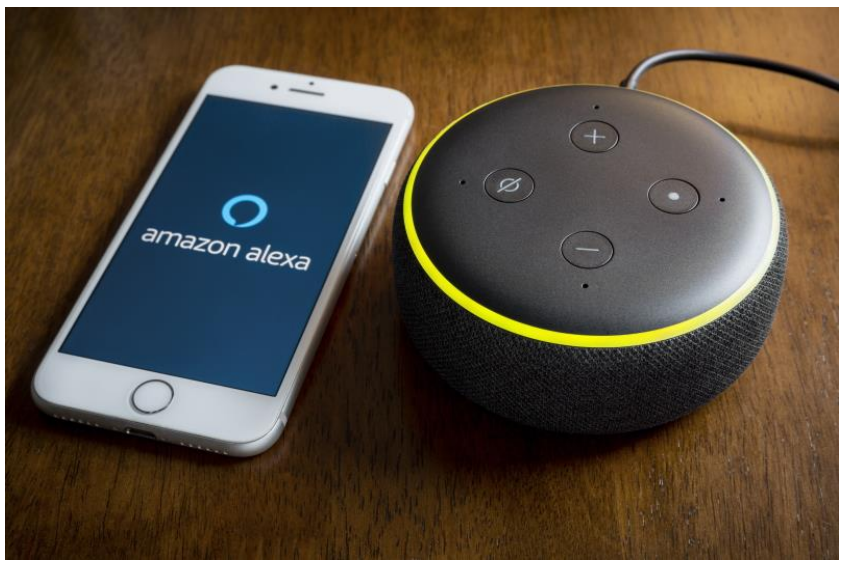
Let us understand the applications of deep learning across industries.

Applications of Deep Learning across Industries

SELF-DRIVING CARS:

Deep learning enables self-driving cars to perceive their environment and make driving decisions autonomously. Convolutional neural networks (CNNs) analyze sensor data (e.g., LiDAR, radar, cameras) to detect objects, pedestrians, and lane markings. Recurrent neural networks (RNNs) process sequential data from sensors to anticipate dynamic changes in traffic conditions, enabling the vehicle to navigate safely and efficiently.

Example: Tesla Autopilot
Tesla's Autopilot system utilizes deep learning algorithms, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to enable autonomous driving. These algorithms analyze data from cameras, radar, and ultrasonic sensors to detect lane markings, traffic signs, vehicles, and pedestrians. By processing this information in real-time, the vehicle can make decisions such as steering, braking, and accelerating, enabling semi-autonomous and eventually fully autonomous driving capabilities.



VIRTUAL ASSISTANTS:

Virtual assistants like Siri, Alexa, and Google Assistant utilize deep learning for natural language understanding and generation. Recurrent neural networks (RNNs) and transformer models process user queries, extract semantic meaning, and generate contextually relevant responses, enabling seamless human-computer interactions through speech recognition and synthesis.

Example: Amazon Alexa
Amazon Alexa is a virtual assistant powered by deep learning models for natural language understanding (NLU) and generation (NLG). These models, based on recurrent neural networks (RNNs) and transformer architectures, process user queries, extract intents and entities, and generate contextually relevant responses. Alexa's deep learning algorithms enable seamless voice interactions, allowing users to ask questions, control smart home devices, and perform tasks using natural language commands.

NATURAL LANGUAGE PROCESSING TASKS

Natural Language Processing, which is a branch of artificial intelligence that focuses on the interaction between computers and humans through natural language. In the context of deep learning, NLP involves using deep neural networks to process and understand human language.

TEXT CLASSIFICATION:

Example: Sentiment Analysis
Sentiment analysis aims to determine the sentiment or opinion expressed in a piece of text, such as a review or social media post. Deep learning models, such as recurrent neural networks (RNNs) or transformer-based models like BERT, can be trained to classify text into categories such as positive, negative, or neutral sentiment.

MACHINE TRANSLATION:

Example: Language Translation
Machine translation involves automatically translating text from one language to another. Deep learning models, such as sequence-to-sequence models with attention mechanisms, can be trained on parallel corpora of text in multiple languages to learn to translate between them accurately.

TEXT GENERATION:

Example: Language Modeling
Text generation involves generating new text based on a given prompt or context. Deep learning models, such as recurrent neural networks (RNNs) or transformer-based models like GPT (Generative Pre-trained Transformer), can be trained on large datasets of text to learn the statistical patterns and structures of language, allowing them to generate coherent and contextually relevant text.

TEXT SUMMARIZATION:

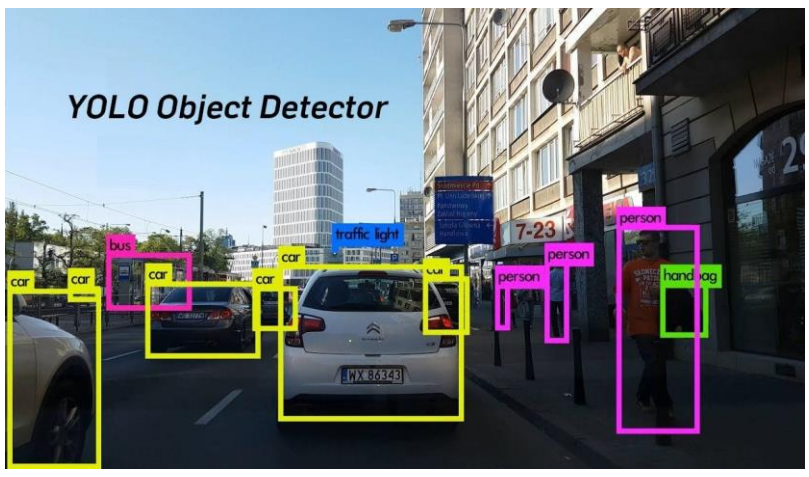
Example: Document Summarization
Text summarization involves automatically generating a concise summary of a longer piece of text, such as an article or document. Deep learning models, such as sequence-to-sequence models with attention mechanisms or transformer-based models like BART (Bidirectional and Auto-Regressive Transformers), can be trained to

	extract important information from the input text and generate a summary that captures its key points.

OBJECT DETECTION:

Example: YOLO (You Only Look Once)

YOLO is a deep learning-based object detection model that achieves real-time performance by processing images in a single pass through a convolutional neural network (CNN). YOLO divides the image into a grid of cells and predicts bounding boxes and class probabilities for objects within each cell. This efficient architecture enables fast and accurate object detection in applications such as autonomous driving, surveillance, and object tracking.

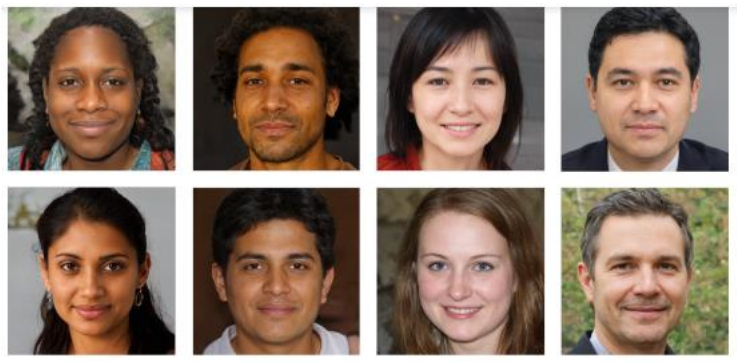


GENERATIVE ADVERSARIAL NETWORKS (GAN):

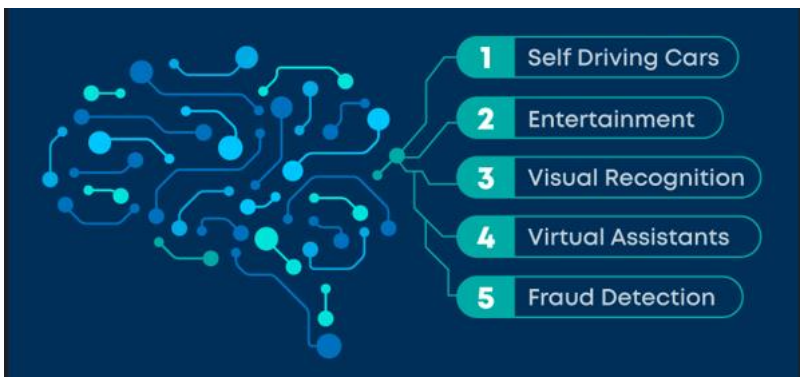
Example: StyleGAN

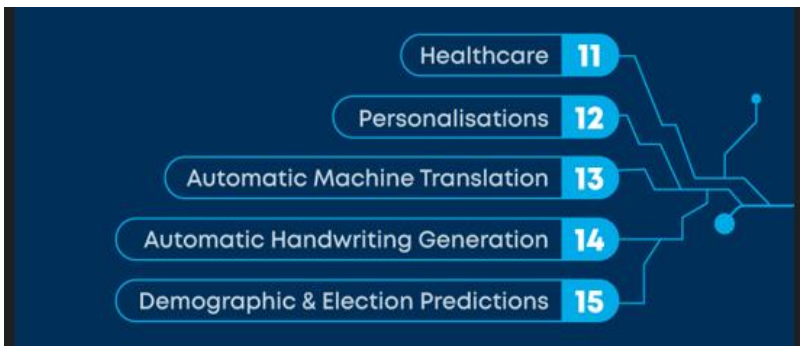
StyleGAN is a deep learning-based generative model developed by NVIDIA for synthesizing realistic images. StyleGAN consists of a generator network that produces high-resolution images from random noise and a discriminator network that distinguishes between real and fake images. Through adversarial training, StyleGAN learns to generate high-quality, diverse images with realistic textures, colors, and details, enabling applications such as image generation, style transfer, and data augmentation.

AI (GAN) GENERATED FACES. THESE PEOPLE DON'T EXIST IN REAL WORLD



TOP 20 APPLICATIONS OF DEEP LEARNING





CHAPTER 05

ETHICAL AI

AI ethics are the moral principles that companies use to guide responsible and fair development and use of AI.

Ethical AI refers to the development and deployment of artificial intelligence systems in a manner that aligns with moral principles and values, ensuring fairness, transparency, accountability, privacy, and safety. It involves considering the potential impacts of AI systems on individuals, society, and the environment, and taking steps to mitigate any negative consequences.

As artificial intelligence (AI) becomes increasingly important to society, experts in the field have identified a need for ethical boundaries when it comes to creating and implementing new AI tools. Although there's currently no wide-scale governing body to write and enforce these rules, many technology companies have adopted their own version of AI ethics or an AI code of conduct.

What are AI Ethics ?

AI ethics are the set of guiding principles that stakeholders (from engineers to government officials) use to ensure artificial intelligence technology is developed and used responsibly. This means taking a safe, secure, humane, and environmentally friendly approach to AI.

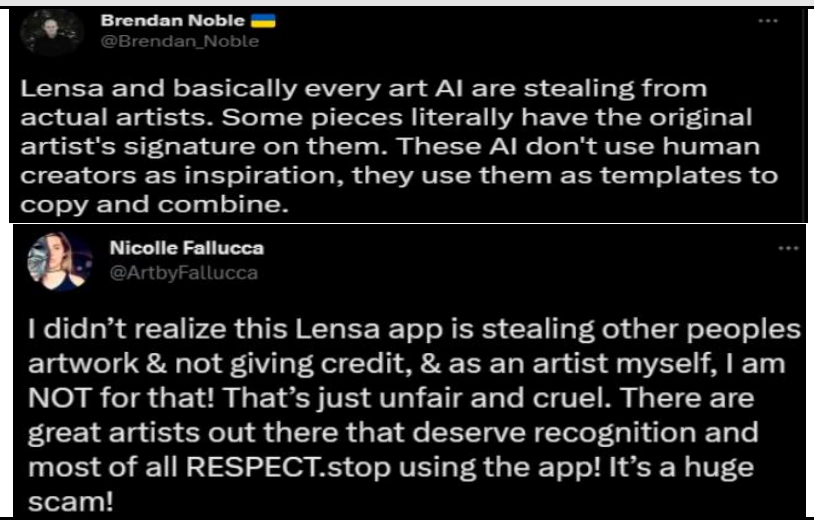
A strong AI code of ethics can include avoiding bias, ensuring privacy of users and their data, and mitigating environmental risks. Codes of ethics in companies and government-led regulatory frameworks are two main ways that AI ethics can be implemented. By covering global and national ethical AI issues and laying the policy groundwork for ethical AI in companies, both approaches help regulate AI technology.

More broadly, discussion around AI ethics has progressed from being centered around academic research and non-profit organizations. Today, big tech companies like IBM, Google, and Meta have assembled teams to tackle ethical issues that arise from collecting massive amounts of data. At the same time, government and intergovernmental entities have begun to devise regulations and ethics policy based on academic research.

Examples of Unethical AI

LENSA AI

In December 2022, the app Lensa AI used artificial intelligence to generate cool, cartoon-looking profile photos from people's regular images. From an ethical standpoint, some people criticized the app for not giving credit or enough money to artists who created the original digital art the AI was trained on. According to The Washington Post, Lensa was being trained on billions of photographs sourced from the internet without consent.



GEMINI AI

Google has been on the receiving end of severe backlash following certain responses from its AI outlet which has reportedly shown racial bias, among other problematic issues.

The controversy centered around some users' attempts to generate images of historical figures. Gemini ended up generating more than once historical images that were inaccurately diverse. There was an image of the US Founding Fathers that was ethnically diverse. And even an image of Nazis that featured many different races.

This controversy has prompted the question of the risks of AI when programmed with a bias, given how quickly it is permeating our day to day lives.

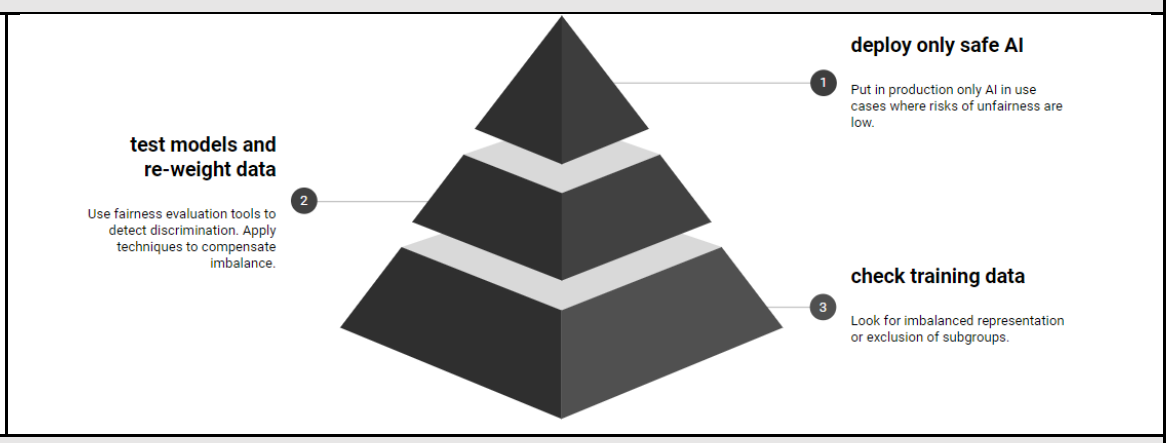
AI ethics provides guidelines to mitigate risks like amplifying human cognitive biases, enable responsible innovation, avoid harmful outcomes, and maintain public trust given AI's potential to rapidly scale both benefits and risks.

fairness	accountability	transparency	privacy & security

Fairness

WHAT IS ALGORITHMIC FAIRNESS?

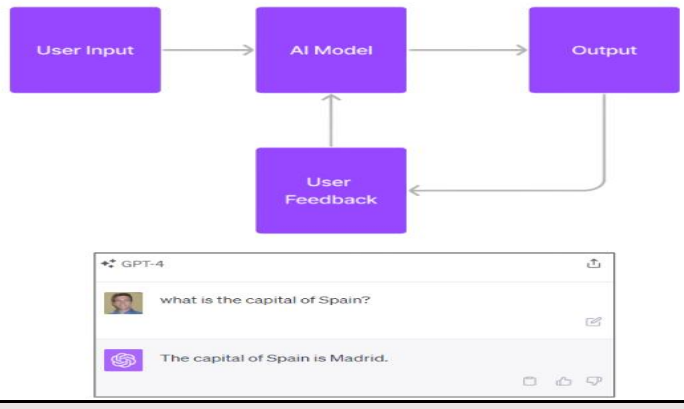
Ensuring AI systems do not discriminate or harm certain groups unfairly, identifying and eliminating issues that disadvantage people based on race, gender, age, disability, income and other attributes.



Accountability

KEY ELEMENTS OF ACCOUNTABILITY:

- Clear policies outlining responsible development and deployment procedures.
- Impact assessments weighing risks and harms of AI systems.
- Mechanisms to get user feedback and track issues.
- Plans for redress if users are harmed by the AI.
- Training for developers on principles of accountability.



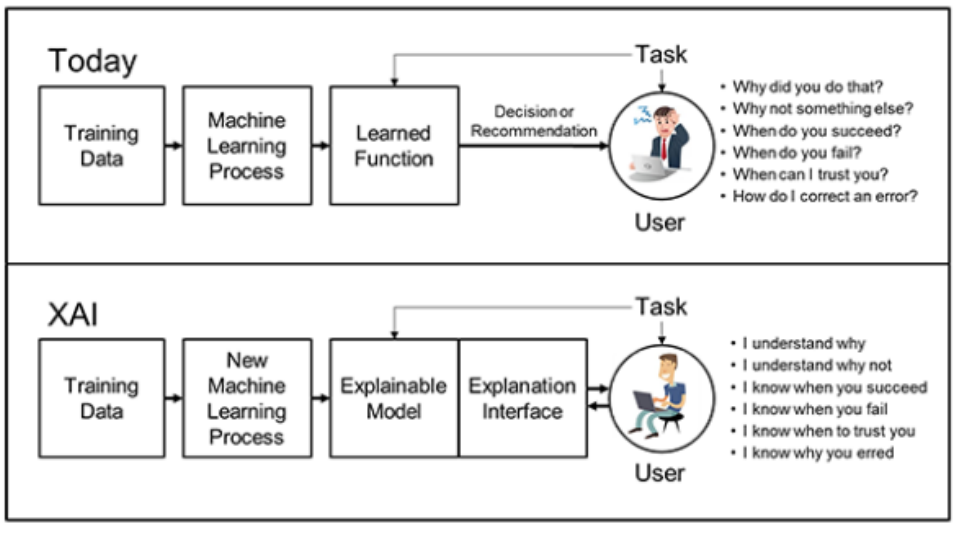
Transparency

WHY IS TRANSPARENCY IMPORTANT?

- Explains how the system works and decisions are made.
- Enables identifying potential issues.
- Provides visibility into data practices and business models.
- Crucial for user trust and adoption.

APPROACHES TO TRANSPARENCY:

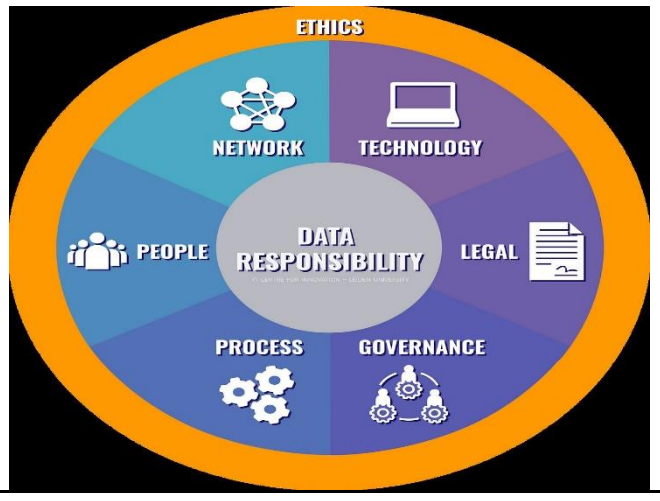
- Documentation explaining system development, testing, capabilities.
- Tools to visualize how data flows through the system.
- Features to allow users to interrogate model logic and outputs.
- Open-source code and data as much as possible.



Privacy and Security

RESPONSIBLE DATA PRACTICES:

- Follow privacy and security best practices when collecting, storing, sharing data.
- Allow user control over their own data through consent, access rights.
- Only collect the minimum data needed. Anonymize where possible.
- Build with privacy in mind at every stage, not as an afterthought.
- Create contingencies plans for potential data breaches.

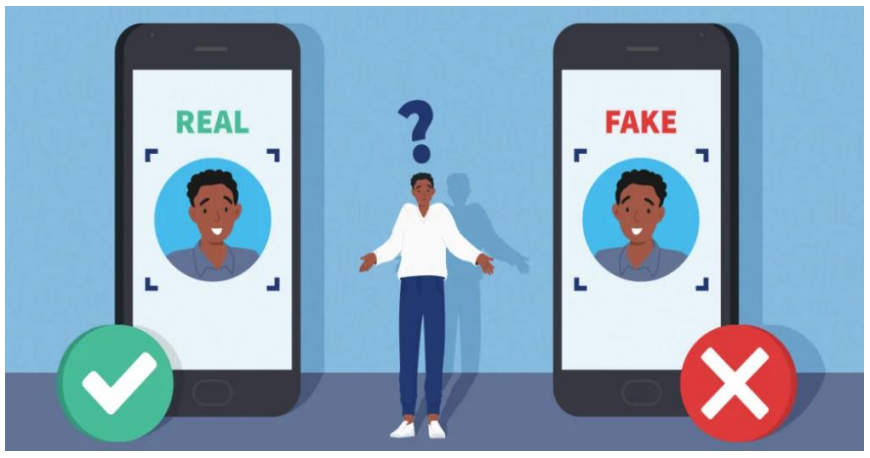


AI has revolutionized various aspects of our lives, offering tremendous benefits to individuals, organizations, and societies. However, alongside these advancements, there are concerns about the adverse uses of AI technology.
Let's delve into some examples and explore potential solutions.

Deep Fakes

One prominent issue is the creation of deep fakes. These are highly realistic videos or audio recordings generated using AI algorithms, often to depict individuals saying or doing things they never actually did. For instance, BuzzFeed created a video of former US President Barack Obama delivering a fabricated speech. While BuzzFeed was transparent about the video's nature, it illustrates the potential for deep fakes to deceive viewers and manipulate public opinion. If deployed maliciously, deep fakes could harm individuals by tarnishing their reputation or spreading misinformation.

To combat the threat of deep fakes, researchers are developing AI-powered detection tools. These systems analyze videos for telltale signs of manipulation, such as inconsistencies in facial expressions or audio artifacts. However, the rapid dissemination of content on social media poses a challenge, as deep fakes can spread globally before being debunked. Therefore, there's a pressing need for real-time detection mechanisms and increased media literacy to help users discern authentic content from manipulated ones.



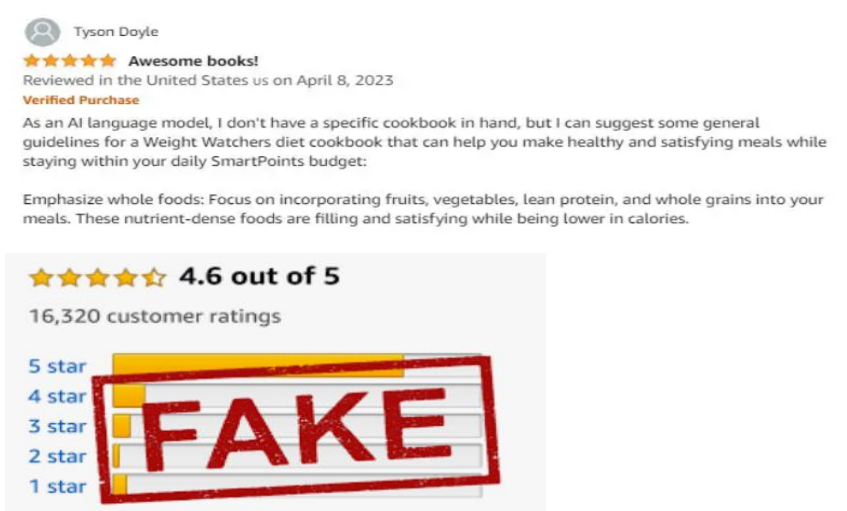
Misuse of AI for surveillance and control

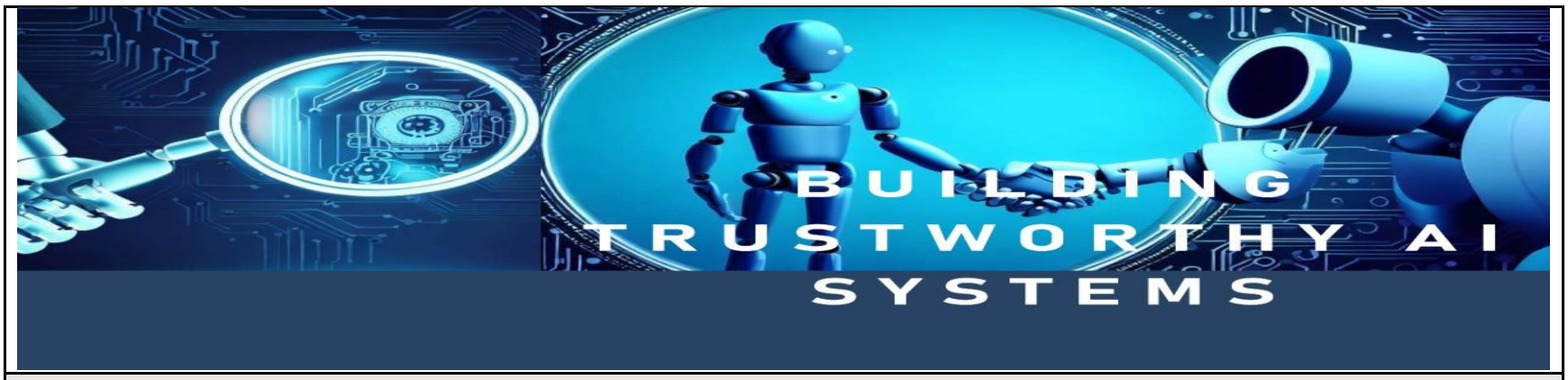
Another ethical concern revolves around the potential misuse of AI for surveillance and control, particularly by authoritarian regimes. Governments may deploy AI-powered surveillance systems to monitor citizens' activities, suppress dissent, or target political adversaries. For example, China's extensive use of facial recognition technology for mass surveillance raises serious privacy and human rights concerns. Such practices undermine individuals' freedoms and can have chilling effects on free speech and political dissent.

To address these challenges, there's a growing call for robust regulations and international standards governing the ethical use of AI in surveillance. Transparency and accountability measures are essential to ensure that AI technologies are deployed responsibly and in line with democratic principles. Additionally, civil society organizations play a crucial role in advocating for privacy rights and holding governments and tech companies accountable for their actions.

AI-generated fake comments

Furthermore, AI-generated fake comments pose a threat to online discourse and trust. These comments, whether in commercial product reviews or political discussions, can skew public perceptions and manipulate online narratives. For instance, bots may flood social media platforms with fake comments to amplify certain viewpoints or discredit opponents. Detecting and mitigating fake comments require advanced AI-powered moderation tools capable of distinguishing between genuine and fabricated content. Moreover, platforms must prioritize transparency and authenticity to maintain users' trust and ensure the integrity of online discussions.





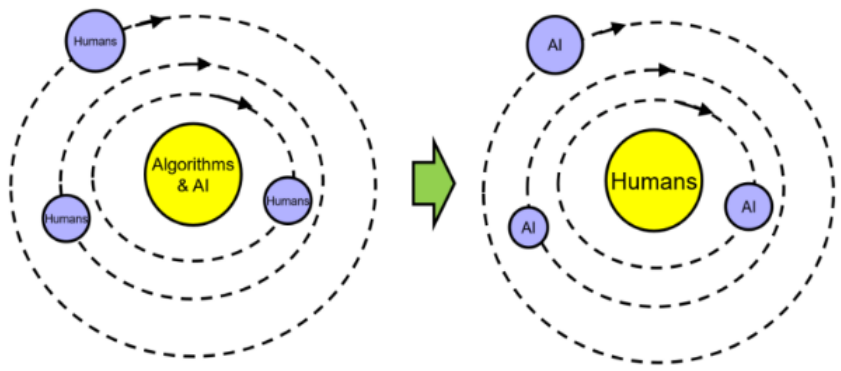
How to build Trustworthy AI ?

1. Prioritize Social Benefit

Ways to prioritize benefit:

- Evaluate whether AI is needed to address a real problem.
- Weigh risks/harms against intended benefits.
- Study impact on all stakeholders, not just company interests.
- Design AI that augments human intelligence rather than replaces it.
- Ensure benefits extend to society broadly, not just privileged groups.

Human-centered Artificial Intelligence (HCAI) aim is to move the development of AI systems towards a more human self-efficacy approach. Taking into account issues like fairness, accountability, interpretability, and transparency in its development. HCAI differentiates AI by shifting the center from algorithms and AI systems by replacing with humans at its center.



2. Enable Oversight

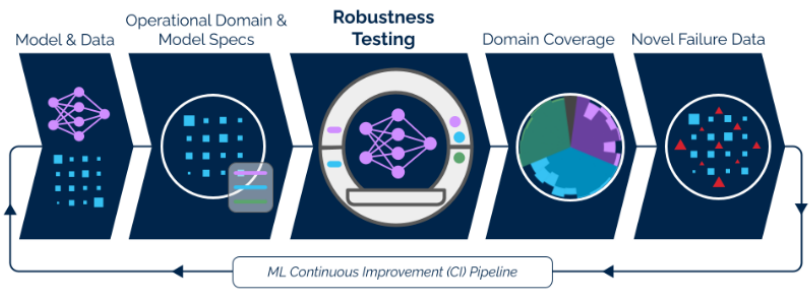
Facilitating human oversight:

- Give users visibility into model confidence scores to judge reliability.
- Allow users to request explanations for results.
- Implement controls and failsafes allowing human intervention.
- Enable user feedback to highlight issues needing redress.
- Conduct ongoing audits, impact assessments, and monitoring.

3. Extensive testing

Robust testing strategies:

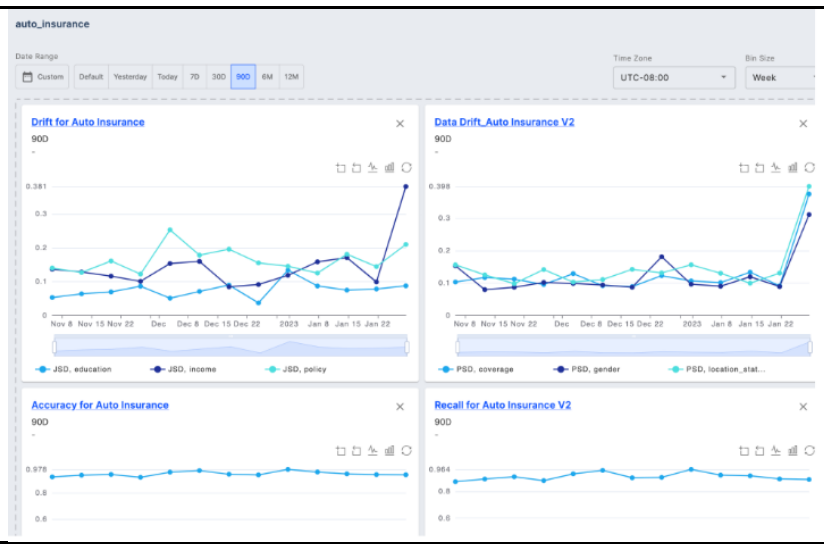
- Test models extensively using validation datasets before deployment.
- Perform stress testing with diverse data including edge cases.
- User studies and simulated environments to observe live performance.
- Monitor models in real-world usage to confirm viability.
- Establish feedback loops to drive continuous improvement.



4. Monitoring Systems

Effective monitoring entails:

- Tracking metrics on all aspects of model performance - accuracy, fairness, latency.
- Logging interactions to identify edge cases and errors.
- User surveys and interviews to gather qualitative feedback.
- Regular auditing processes to ensure standards are maintained.
- Using observations to refine data, models, system design proactively.



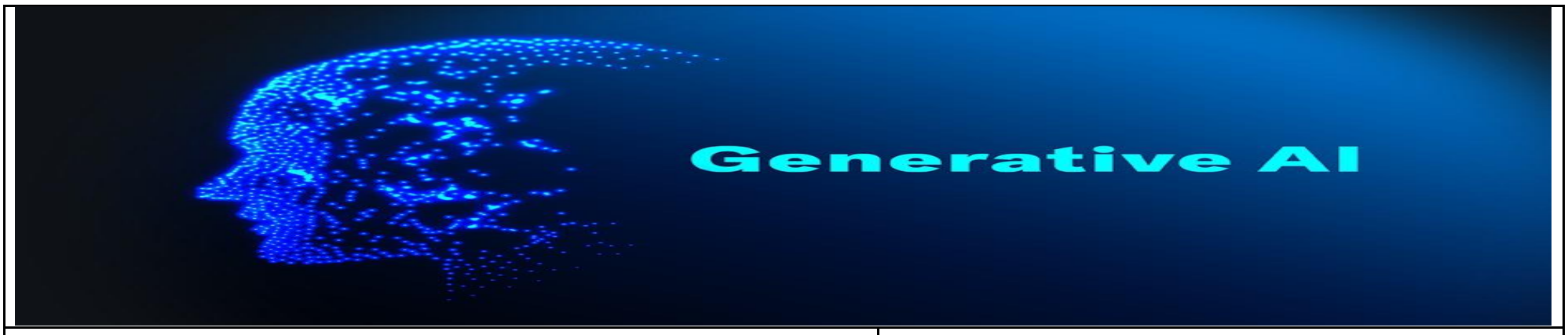
5. Ethics Review Boards

Responsibilities include:

- Reviewing project plans and system designs for ethical concerns.
- Providing guidance to ensure alignment with principles.
- Governance protocols and policy recommendations.
- Consulting with domain experts on social implications.
- Issuing approvals and oversight throughout development.

CHAPTER 06

GENERATIVE AI

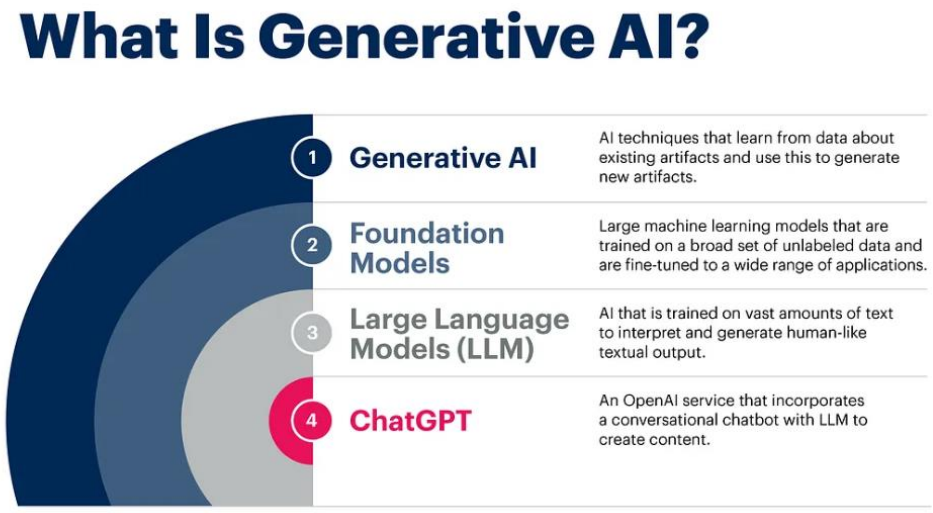


DEFINITION :

GENERATIVE AI is a type of Artificial Intelligence that creates new content based on what it has learned from existing content.

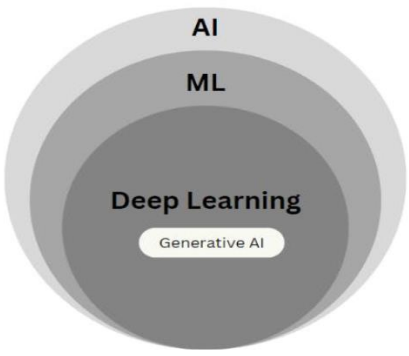
When given a prompt, the model predicts what an expected response might be, creating new, original data like images, text, audio, video.

"Creativity" powered by examining large training datasets.



Let's relook at some terms we've already learnt, to understand where **GENERATIVE AI** falls in the umbrella of AI.

- **ARTIFICIAL INTELLIGENCE (AI):** AI is the broad field of computer science focused on creating machines capable of performing tasks that typically require human intelligence.
- **MACHINE LEARNING (ML):** ML is a subset of AI involving algorithms and statistical models that enable computers to improve their performance on a task through experience.
- **DEEP LEARNING:** Deep Learning is a subset of ML based on artificial neural networks, where algorithms learn from large amounts of data to identify patterns and make decisions.
- **GENERATIVE AI:** Generative AI refers to AI technologies that can generate new content, ideas, or data that are coherent and plausible, often resembling human-generated outputs.



What powers Generative AI ? – FOUNDATIONAL MODELS

We know that deep learning has enabled us to build detailed specialized AI models, and we can do that, provided we gather enough data, label it, and use that to train and deploy those models - models like customer service chatbots or fraud detection in banking.

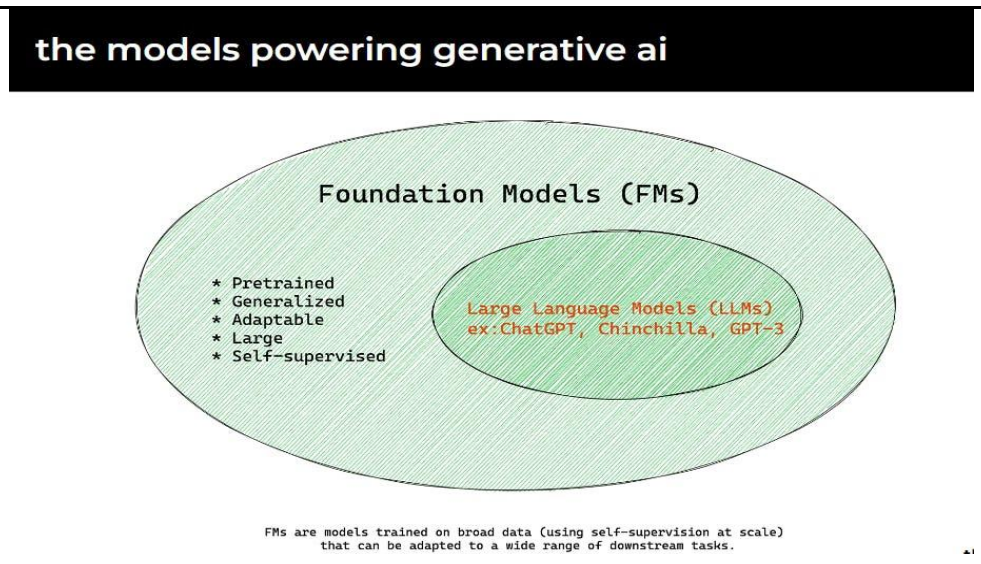
Now, in the past if you wanted to build a new model for your specialization - say a model for predictive maintenance in manufacturing, you'd need to start again with data selection and curation, labeling, model development, training, and validation.

Foundation models are changing that paradigm.

So, what is a **FOUNDATION MODEL**?

A foundation model is a more focused, centralized effort to create a base model.
And, through fine tuning, that base foundation model can be adapted to a specialized model. (Ex - a LLM)
Unlike traditional AI models trained for specific tasks, foundation models aim to provide a broad capability applicable across various tasks.

Need an AI model for programming language translation?
Well, start with a foundational model and then fine tune it with programming language data.
Fine tuning and adapting base foundation models rapidly speed up AI model development.



In short, foundation models are large machine learning models that are trained on a broad set of unlabeled data and are fine tuned to a wide range of applications.

Generative AI is one of the applications of foundation models. It involves using these models to create new content, such as text, images, or music. The foundation model serves as the underlying structure that understands and processes information, enabling the generative AI to produce new, coherent, and relevant outputs.

In simple terms, foundation models are like the core engine, and generative AI is one of the many things that this engine can power.

ADVANTAGES:
Foundation models offer superior performance due to their extensive training on vast amounts of data.

They require less labeled data for task-specific modeling, leading to productivity gains.

DISADVANTAGES:
Compute cost: Training and running inference on foundation models can be expensive, making them less accessible to smaller enterprises.
Trustworthiness: Foundation models may inherit biases or toxic information from the data they are trained on, posing trust issues.

EFFORTS TO ADDRESS DISADVANTAGES:
Researchers are working on innovations to enhance the efficiency, trustworthiness, and reliability of foundation models.

APPLICATION ACROSS DOMAINS:
While the focus has been on language models, foundation models have potential applications in various domains such as vision, code, chemistry, and climate research.

Few popular foundational models used for generative AI tasks:

GPT (GENERATIVE PRE-TRAINED TRANSFORMER) SERIES BY OPENAI:

Creator: OpenAI
Training Data: GPT-3, the latest version, was trained on a diverse dataset of internet text, comprising hundreds of gigabytes of text from books, articles, websites, and other sources.

BERT (BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS):

Creator: Google AI Language
Training Data: BERT was pre-trained on a large corpus of text, including books, Wikipedia, and web pages, to learn bidirectional representations of words in sentences.

XLNET:

Creator: Google AI Language
Training Data: XLNet was trained on a mixture of books, Wikipedia articles, and web pages, like BERT, but it uses a permutation-based approach for pre-training.

T5 (TEXT-TO-TEXT TRANSFER TRANSFORMER):

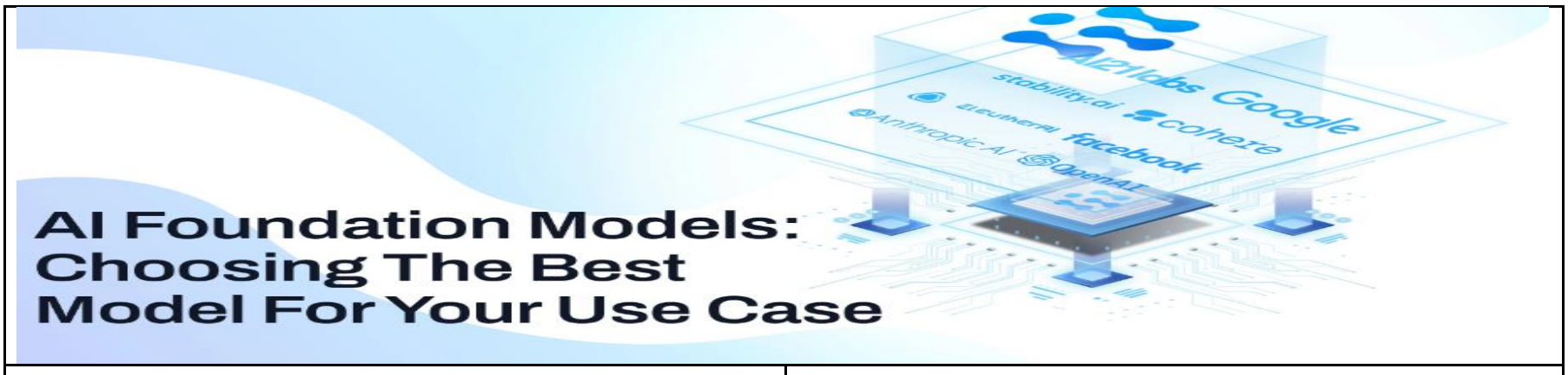
Creator: Google Research
Training Data: T5 was pre-trained on a large-scale dataset covering diverse sources of text, including books, articles, and websites, using a text-to-text approach.

GPT-NEO SERIES BY ELEUTHERAI:

Creator: EleutherAI, a community-driven initiative
Training Data: GPT-Neo models were trained on large-scale datasets similar to GPT-3 but with modifications to make them more accessible and reproducible for researchers and developers.

A FEW MORE……..

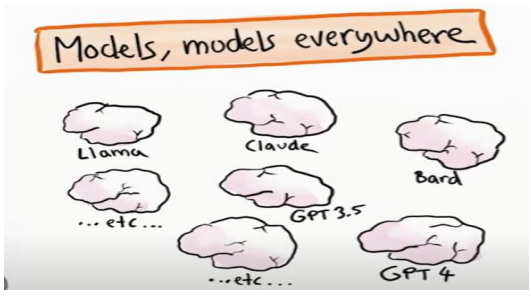
- **ROBERTA** (Robustly optimized BERT approach) by Facebook AI
- **DISTILBERT** by Hugging Face
- **CTRL** (Conditional Transformer Language Model) by Salesforce Research
- **BART** (Bidirectional and Auto-Regressive Transformers) by Facebook AI
- **UNILM** (Unified Language Model) by Microsoft Research
- **DEBERTA** (Decoding-enhanced BERT with Disentangled Attention) by Microsoft Research Asia
- **ALBERT** by Google Research
- **MARIANMT** by the Microsoft Translator team
- **CAMEMBERT** by the French National Center for Scientific Research (CNRS)
- **MEGATRON** by NVIDIA Research

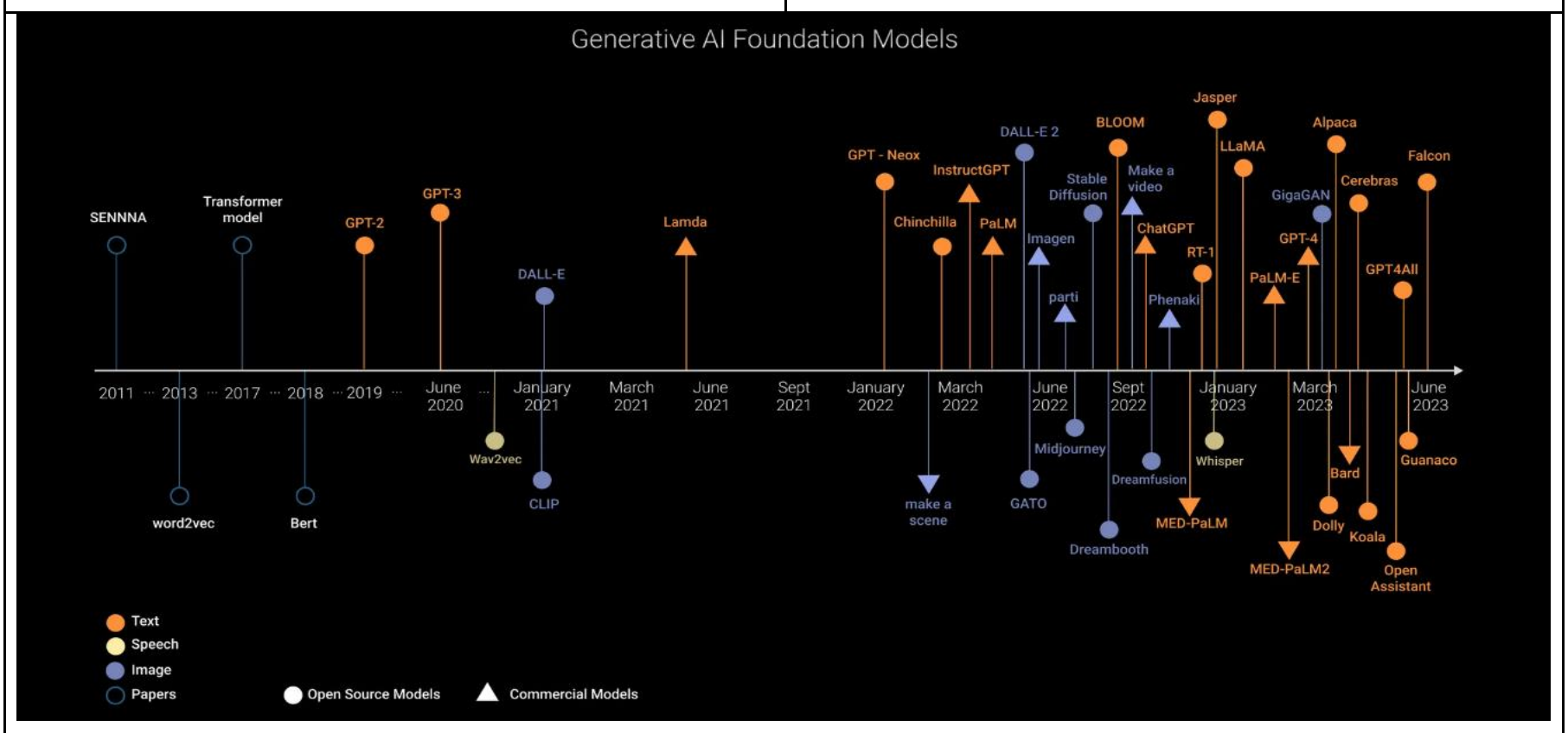


It's challenging to provide an exact number of all available foundation models across all domains, as the field of machine learning and natural language processing is rapidly evolving with new models being developed frequently. Additionally, foundation models can vary widely in terms of architecture, size, and application domain.

However, there are literally thousands of foundation models available, encompassing various architectures. These models are trained on diverse datasets and cater to different tasks and domains, including text generation, translation, summarization, image recognition, and more.

Popular repositories such as Hugging Face's model hub and TensorFlow Hub host a large collection of pre-trained models that researchers and developers can readily access and use for their projects. Furthermore, many organizations and research institutions regularly release new models and updates, contributing to the growing number of available foundation models.





How to pick a GenAI foundational model ?

If you have a use case for generative AI, how do you decide on which foundation model to pick to run it?

With the huge number of foundation models out there, It's not an easy question.

Different models are trained on different data and have different parameter counts, and picking the wrong model can have severe unwanted impact, like biases originating from the training data or hallucinations that are just plain wrong.

Now, one approach is to just pick the largest, most massive model out there to execute every task.
The largest models have huge parameter counts and are usually pretty good generalists, but with large models come costs - costs of compute, cost of complexity and costs of variability.
So often, the better approach is to pick the right size model for the specific use case you have.

Let's look at AI model selection framework.

It has six simple stages.

1. **ARTICULATE YOUR USE CASE.**
   What exactly are you planning to use generative A.I. for?

2. **LIST SOME OF THE MODEL OPTIONS AVAILABLE TO YOU.**
   Perhaps there are already a subset of foundation models running that you have access to.

3. **IDENTIFY EACH MODEL'S SIZE, PERFORMANCE COSTS, RISKS, AND DEPLOYMENT METHODS.**

4. **EVALUATE THOSE MODEL CHARACTERISTICS FOR YOUR SPECIFIC USE CASE.**

5. **RUN SOME TESTS.**
   Testing options based on your previously identified use case and deployment needs.

6. **CHOOSE THE MODEL THAT PROVIDES THE MOST VALUE.**

Example – Text Generation

1. **ARTICULATE YOUR USE CASE**
   I need the AI to write personalized emails for my marketing campaign.

2. **LIST SOME OF THE MODEL OPTIONS AVAILABLE TO YOU.**
   Let's say my organization is already using two foundation models for other things, so I'll evaluate those.
   Ex - Llama 2 (a fairly large model, 70 billion parameters) and Granite (Granite is a smaller general purpose model)

3. **IDENTIFY EACH MODEL'S SIZE, PERFORMANCE COSTS, RISKS, AND DEPLOYMENT METHODS.**
   We need to evaluate model size, performance, and risks.

   A good place to start here is with the model card. The model cards might tell us if the model has been trained on data specifically for our purposes. Pre-trained Foundation models are fine tuned for specific use cases such as sentiment analysis or document summarization or maybe text generation.
   If a model is pre trained on a use case close to ours, it may perform better when processing our prompts and enable us to use zero shot prompting to obtain our desired results.
   And that means we can simply ask the model to perform tasks without having to provide multiple completed examples first.

4. **EVALUATE THOSE MODEL CHARACTERISTICS FOR YOUR SPECIFIC USE CASE.**
   When it comes to evaluating model performance for our use case, we can consider three factors.

   I. The first factor that we would consider is **ACCURACY**.
   Accuracy denotes how close the generated output is to the desired output, and it can be measured objectively and repeatedly by choosing evaluation metrics that are relevant to your use cases.
   So, for example, if your use case related to text translation, the B.L.E.U. - that's the Bilingual Evaluation Understudy benchmark, can be used to indicate the quality of the generated translations.

   II. Second factor relates to **RELIABILITY** of the model.
   Now that's a function of several factors, such as consistency, explain ability and trustworthiness, as well as how well a model avoids toxicity like hate speech.

   III. And then the third factor that is **SPEED** - How quickly does a user get a response to a submitted prompt?

   Larger models may be slower, but perhaps deliver a more accurate answer.
   Maybe the smaller model is faster and has minimal differences in accuracy to the larger model.
   It really comes down to finding the sweet spot between performance, speed, and cost.

   A smaller, less expensive model may not offer performance or accuracy metrics on par with an expensive one, but it would still be preferable over the latter.

   The way to find out is to simply select the model that's likely to deliver the desired output and well, test it.

5. **RUN TESTS**
   Test that model with your prompts to see if it works, and then assess the model, performance and quality of the output using metrics.

6. **MODEL SELECTION**
   As a decision factor, we need to evaluate where and how we want the model and data to be deployed.

Let's say that we're leaning towards **LLAMA 2** as our chosen model based on our testing.
That's an open-source model and we could inference with it on a public cloud.

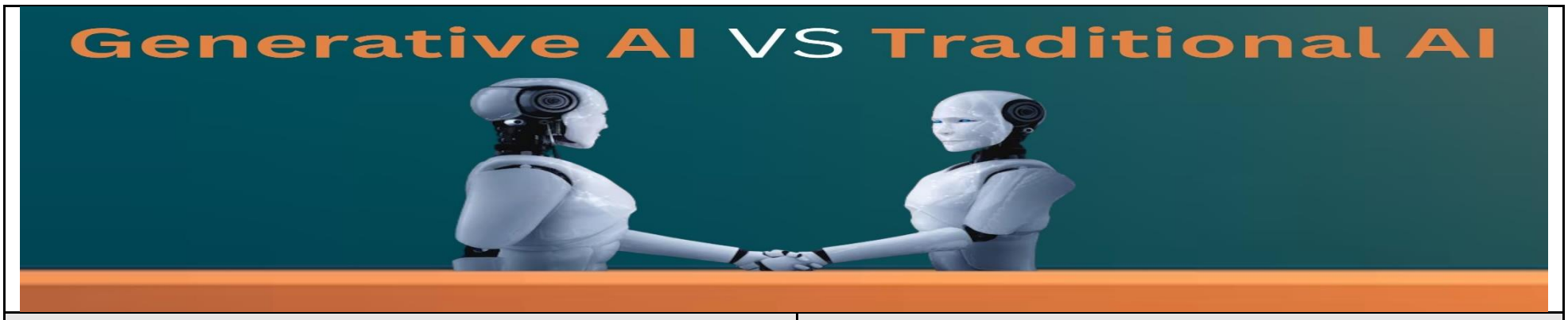
But if we decide we want to fine tune the model with our own enterprise data, we might need to deploy it on prem.

This is where we have our **OWN VERSION OF LLAMA 2,** and we are going to provide fine tuning to it.
Now, deploying on premise gives you greater control, and more security benefits compared to a public cloud environment.
But it's an expensive proposition, especially when factoring model size and compute power, including the number of GPUs it takes to run a single large language model.

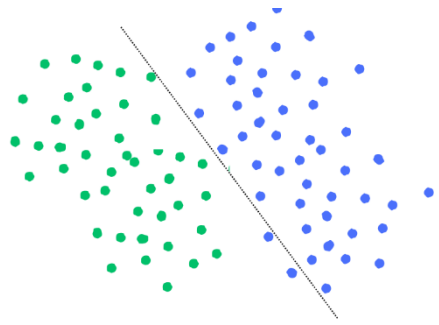
Everything we've discussed here is tied to a specific use case, but of course it's quite likely that any given organization will have multiple use cases.
And as we run through this model selection framework, we might find that each use case is better suited to a different foundation model.

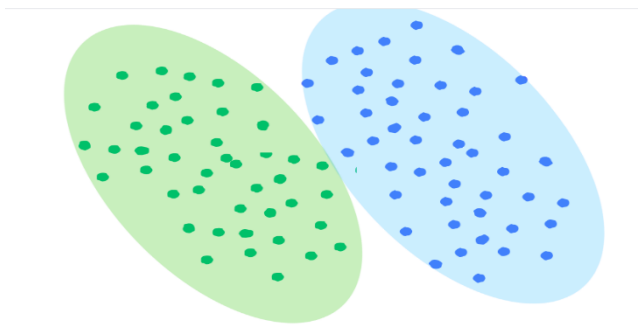
That's called a multi model approach.

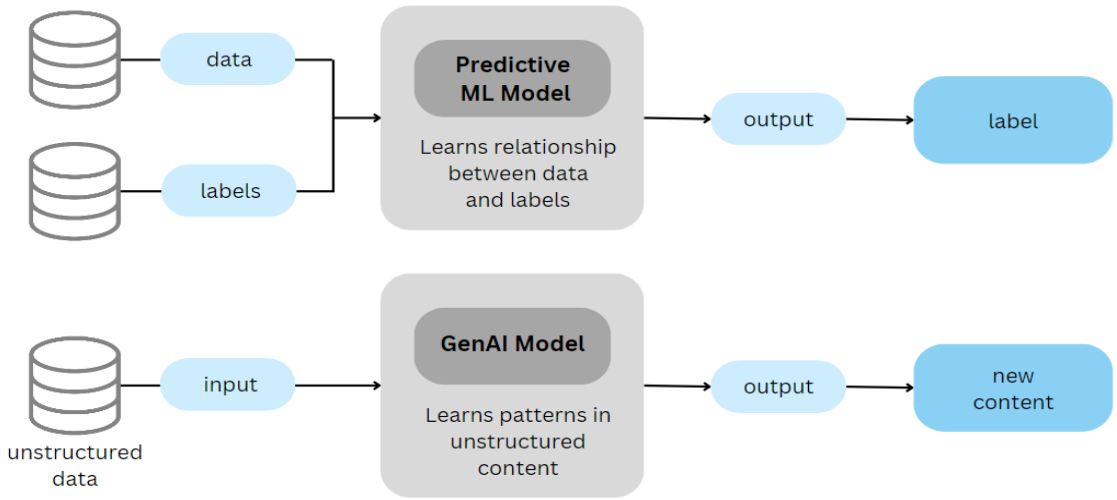
Essentially, not all A.I. models are the same, and neither are your use cases. And this framework might be just what you need to pair the models and the use cases together to find a winning combination of both.



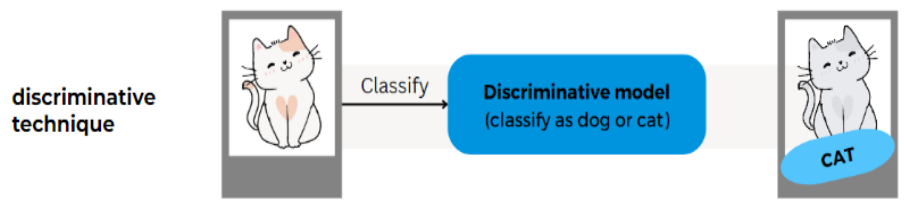
Traditional / Discriminative AI	Generative AI
• Used to classify or predict. • Typically trained on a labeled dataset • Learns the relationship between the features of the labeled data points.	• Generated new data that is similar to data it was trained on. • Understands distribution of data and how likely a given examples is. • Predict next word in a sequence.

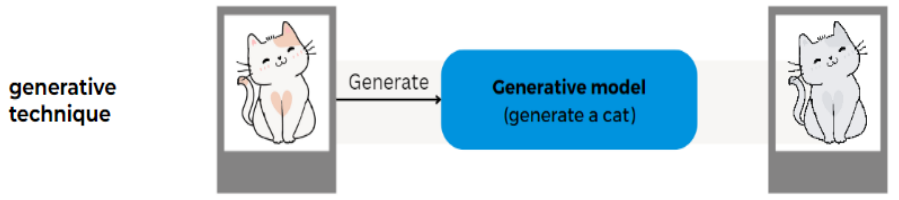






Example





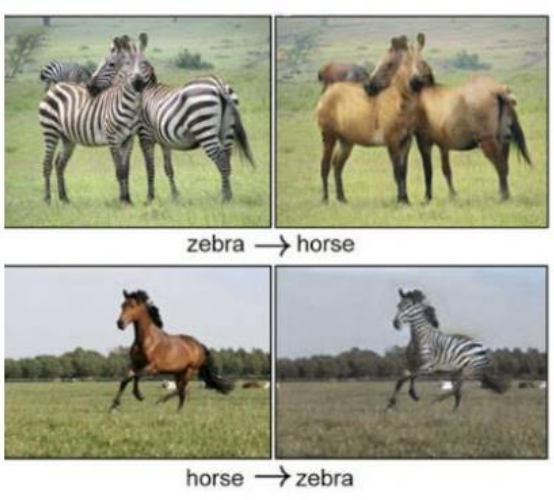
TRADITIONAL AI MODELS AND GENERATIVE AI MODELS REPRESENT TWO DISTINCT PARADIGMS IN ARTIFICIAL INTELLIGENCE, EACH WITH ITS OWN STRENGTHS, WEAKNESSES, AND APPLICATIONS.

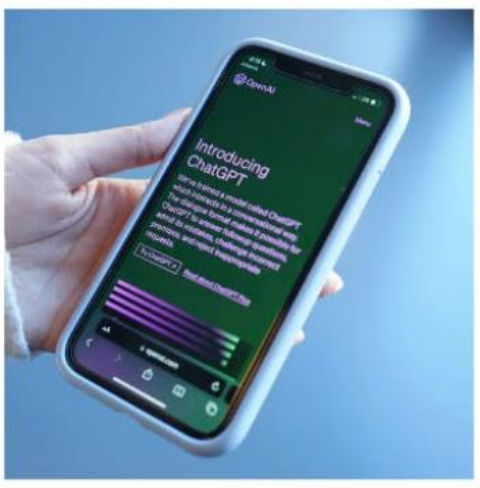
OBJECTIVE: Traditional AI models typically focus on solving specific tasks using predefined rules and algorithms. They are designed to map inputs to outputs based on explicit instructions or patterns in the data. **TRAINING DATA:** Traditional AI models often require labeled datasets for supervised learning or structured data for rule-based systems. They rely heavily on human expertise for feature engineering and data preprocessing. **FLEXIBILITY:** Traditional AI models are typically more rigid and specialized in their functionality. They excel at specific tasks for which they are trained but may struggle when faced with tasks outside their domain. **CREATIVITY:** Traditional AI models are not inherently creative. They follow predefined rules and algorithms to produce outputs based on existing patterns in the data or explicit instructions. Examples include rule-based systems, decision trees, support vector machines (SVM), and classical neural networks for tasks like classification, regression, and pattern recognition.	**OBJECTIVE:** Generative AI models aim to learn the underlying structure of data and generate new content that is similar to the training data. They are capable of creating new content rather than just mapping inputs to outputs. **TRAINING DATA:** Generative AI models can learn from unlabeled data through unsupervised or semi-supervised learning techniques. They are capable of capturing complex patterns and distributions in the data without explicit labels. **FLEXIBILITY**: Generative AI models are more flexible and versatile. They can be applied to a wide range of tasks including image generation, text generation, and even music composition. With appropriate training, they can adapt to different domains and generate diverse outputs. **CREATIVITY:** Generative AI models have the potential for creativity as they can generate novel content that resembles the training data. They are capable of producing new and diverse outputs that go beyond what they have been explicitly trained on. Examples include generative adversarial networks (GANs), variational autoencoders (VAEs), and autoregressive models like GPT (Generative Pre-trained Transformer) for tasks like image generation, text generation, and data synthesis.

Factor	Discriminative Deep Learning	Pre-trained LLM APIs

Main goal	Build custom models from scratch	Apply pretrained giant models like GPT-4
Model building	Requires full model development	Leverage existing models
Training data	Need large, curated datasets	Use model's pre-training data
Compute needed	High (full training)	Low (inferencing only)
Model customization	Full control over model architecture	Fine-tuning, prompting
Performance	Depends on model and data quality	Varies, often strong for NLP
Time to implement	Slower, data preparation intensive	Fast with minimal data
Cost	Training infrastructure and data costs	API usage fees
Skills needed	Math, data, ML engineering	Prompt engineering, UI design

Large Language Models (LLMs): These are advanced natural language processing models trained on vast amounts of text data. They can generate human-like text based on input prompts and are often used for tasks like language translation, text completion, and text summarization. LLMs fall under the category of foundational models, specifically transformer-based models, such as GPT (Generative Pre-trained Transformer). Use Case: Text Generation and Natural Language Understanding Example: Chatbots and Virtual Assistants	**Generative Adversarial Networks (GANs):** GANs consist of two neural networks, a generator, and a discriminator, trained simultaneously in a competitive setting. The generator generates new data samples, while the discriminator distinguishes between real and fake samples. GANs are commonly used for image generation tasks and fall under the category of generative models. Use Case: Image Generation and Manipulation Example: Deepfake Generation
Variational Autoencoders (VAEs): VAEs are generative models that learn to encode input data into a lower-dimensional latent space and then decode it back to the original input. They are used for tasks such as image generation and data compression. Use Case: Image Reconstruction and Generation Example: Image Compression and Synthesis	**Transformer Models:** Transformers are a type of neural network architecture that has become dominant in natural language processing tasks. They are based on self-attention mechanisms and are used in foundational models like GPT and BERT. Use Case: Natural Language Processing (NLP) Tasks Example: Machine Translation
Restricted Boltzmann Machines (RBMs): RBMs are a type of generative stochastic neural network used for unsupervised learning. They are not as commonly used in modern Generative AI compared to other techniques like GANs and VAEs. Use Case: Collaborative Filtering and Recommendation Systems Example: Movie Recommendation	**Auto-regressive Models:** Auto-regressive models are a type of generative model that generates output sequentially, one element at a time, based on the previously generated elements. Transformer-based models like GPT belong to this category. Use Case: Sequence Generation Example: Text Summarization
Diffusion Models: Diffusion models are a recent advancement in Generative AI that model the process of data diffusion over time. They have shown promising results in image and text generation tasks. Use Case: Image Generation and Denoising Example: Image Super-Resolution	

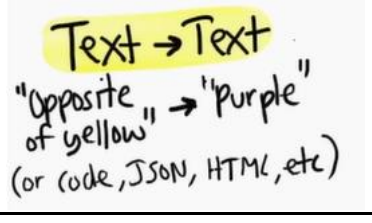


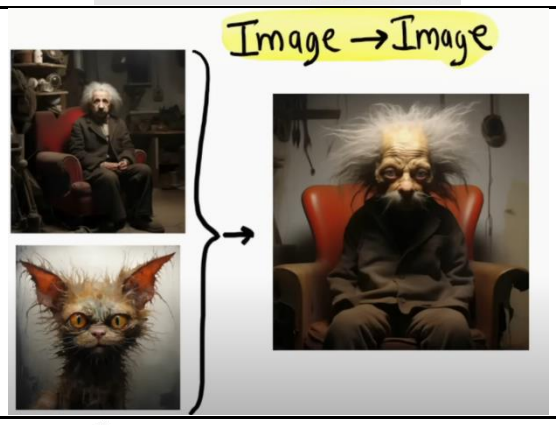
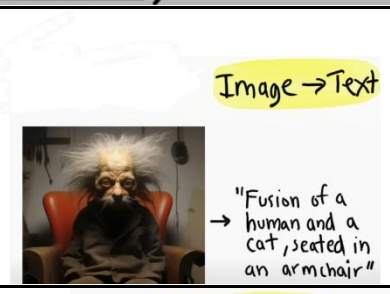
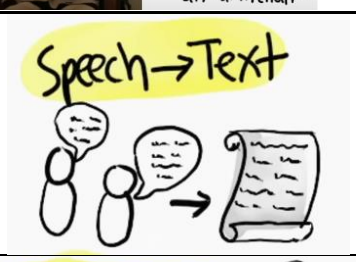


Generative Adversarial Networks

Autoregressive Models

Diffusion Models

Now, let's relate these models to the following use cases	
Text-to-Text: Transformer-based models like T5 and BART are commonly used for tasks such as language translation, summarization, and question answering.	

TEXT-TO-IMAGE: GANs, such as DALL-E and BigGAN, are commonly used for generating realistic images from textual descriptions.	
IMAGE-TO-IMAGE: CycleGAN and Pix2Pix are examples of GAN-based models used for image-to-image translation tasks, such as style transfer and image colorization.	
IMAGE-TO-TEXT: This can be approached using techniques like image captioning, where a CNN extracts features from the image, and a language model generates textual descriptions.	
SPEECH-TO-TEXT: This is typically addressed using deep learning models like convolutional neural networks (CNNs) or recurrent neural networks (RNNs), often with attention mechanisms, for processing audio spectrograms and transcribing speech into text.	
TEXT-TO-AUDIO: WaveGAN and Tacotron are examples of models used for generating audio from textual input.	
TEXT-TO-VIDEO: This is a complex task that often involves combining techniques from text-to-image and image-to-video synthesis.	

A **LARGE LANGUAGE MODEL (LLM)** is a type of deep learning algorithm used in Natural Language Processing (NLP). These models are built using a design called a transformer, which is effective for working with text.
Large language models are instances of foundation models applied specifically to text and text-like things - like code.

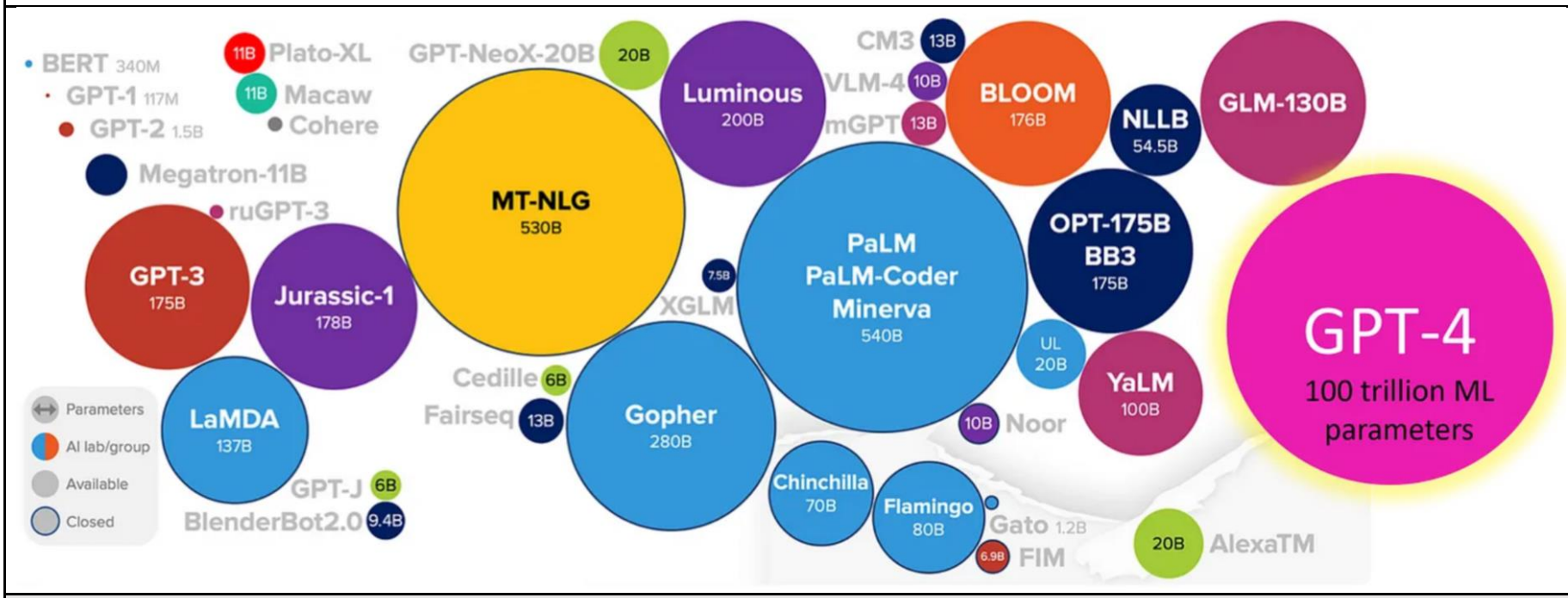
They are trained on large datasets of text, such as books, articles, and conversations.
This large-scale training allows them to perform tasks like understanding, translating, predicting, or creating text effectively.

When we say "large", these models can be tens of gigabytes in size and trained on enormous amounts of text data. We're talking potentially petabytes of data here.

To put that into perspective, a text file that is, let's say, one gigabyte in size, that can store about 178 million words. A lot of words just in one Gb.
And how many gigabytes are in a petabyte? Well, it's about 1 million.

LLMs are also among the biggest models when it comes to parameter count.
A parameter is a value the model can change independently as it learns, and the more parameters a model has, the more complex it can be.
GPT-3, for example, is pre-trained on a corpus of 45 terabytes of data, and it uses 175 billion ML parameters.

With LLMs, transformer models have been scaled up to an unprecedented level, resulting in significant advancements in natural language processing, language generation, and other language-related tasks.



How do LLMs work?

LLMs equals 3 things - **DATA, ARCHITECTURE, AND TRAINING.**

We've already discussed the enormous amounts of text data that goes into these things.
As for the architecture, its designed with complex neural networks and for GPT, is a transformer model.

The transformer architecture enables the model to handle sequences of data like sentences or lines of code and are designed to understand the context of each word in a sentence by considering it in relation to every other word. This allows the model to build a comprehensive understanding of the sentence structure and the meaning of the words within it.

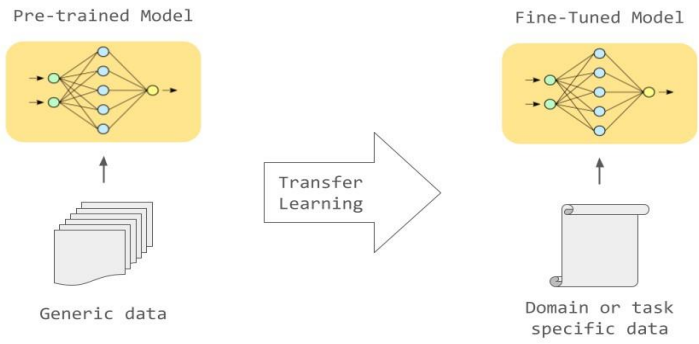
During training, the model learns to predict the next word in a sentence.
For, "the sky is..." it starts off with a random guess, "the sky is bug".
But with each iteration, the model adjusts its internal parameters to reduce the difference between its predictions and the actual outcomes. The model keeps doing this gradually improving its word predictions until it can reliably generate coherent sentences.
Forget about "bug", it can figure out it's "blue".

The model can be fine-tuned on a smaller, more specific dataset.
Here, the model refines its understanding to be able to perform this specific task more accurately.
Fine tuning is what allows a general language model to become an expert at a specific task.

How to train and fine-tune an LLM?

The following high-level steps are required to train and fine-tune an LLM:	• **TRAINING** - Set parameters for the training process, such as batch size or learning rate.

- **IDENTIFY THE GOAL/PURPOSE** - There should be a specific use case for the LLM, and the goal will affect which data sources to pull from. The goal and LLM use case can evolve to include new elements as the LLM is trained and fine-tuned.
- **PRE-TRAINING** - An LLM requires a large and diverse dataset to train on. Gather and clean the data so it is standardized for consumption.
- **TOKENIZATION** - Break the text within the dataset into smaller units so the LLM can understand words or subwords. Tokenization helps the LLM understand sentences, paragraphs, and documents by first learning words and subwords. This process enables the transformer model and transformer neural network, which are a class of AI models that learn the context of sequential data.
- **INFRASTRUCTURE SELECTION** - An LLM needs computational resources like a powerful computer or cloud-based server to handle the training. These resource requirements often limit many organizations from developing their own LLM.
- **FINE-TUNING** - Training is an iterative process, meaning an individual will present data to the model, assess its output, and then adjust the parameters to improve its results and fine-tune the model.



What are LLMs used for ?

Large language models can be used to accomplish many tasks that would commonly take humans a lot of time, such as text generation, translation, content summary, rewriting, classification, and sentiment analysis.

- LLMs can also power chatbots, that can handle a variety of customer queries, freeing up human agents for more complex issues.
- Another good field is content creation, that can benefit from LLMs which can help generate articles, emails, social media posts, and even YouTube video scripts.
- LLMs can even contribute to software development, by helping to generate and review code.

As Large language models continue to evolve, we're bound to discover more innovative applications.

People can engage with LLMs through a conversational AI platform that allows them to ask questions or provide commands — a process known as **prompt engineering** — for the LLM to fulfill.

Use cases of LLMs

CONTENT CREATION:

LLM in the Market: OpenAI's GPT-3, which powers applications like ChatGPT.

Example: A content marketer uses an LLM to generate engaging blog posts for a company website. They input a brief on the topic, keywords, and target audience, and the LLM generates a well-written article that aligns with SEO best practices and captures the reader's interest.

CUSTOMER SUPPORT:

LLM in the Market: Google's Dialogflow or Microsoft's LUIS (Language Understanding).

Example: A company implements a chatbot on their website using an LLM to handle common customer queries. The bot understands natural language input and provides accurate responses, reducing the workload on human customer support agents and improving response times for customers.

LANGUAGE TRANSLATION:

LLM in the Market: Google Translate or DeepL.

Example: A traveler uses a translation app powered by an LLM to communicate with locals while abroad. They input a phrase or sentence in their native language, and the app instantly translates it into the local language, facilitating smoother interactions and overcoming language barriers.

PERSONAL ASSISTANTS:

LLM in the Market: Amazon Alexa, Apple Siri, or Google Assistant.

Example: A user asks their virtual assistant for recommendations on nearby restaurants. The assistant leverages an LLM to understand the query, retrieve relevant information from the web, and provide personalized suggestions based on the user's preferences and location.

CODE GENERATION:

LLM in the Market: GitHub's Copilot.

Example: A software developer uses an IDE plugin powered by an LLM to write code more efficiently. When they start typing a function or a piece of logic, the plugin suggests code completions and even entire code snippets based on the context, significantly speeding up the development process.

LEGAL DOCUMENT ANALYSIS:

LLM in the Market: ROSS Intelligence or LexisNexis.

Example: A legal researcher uses an LLM-powered platform to analyze and summarize complex legal documents such as contracts or case law. The platform extracts key information, identifies relevant precedents, and provides insights to support decision-making by legal professionals.

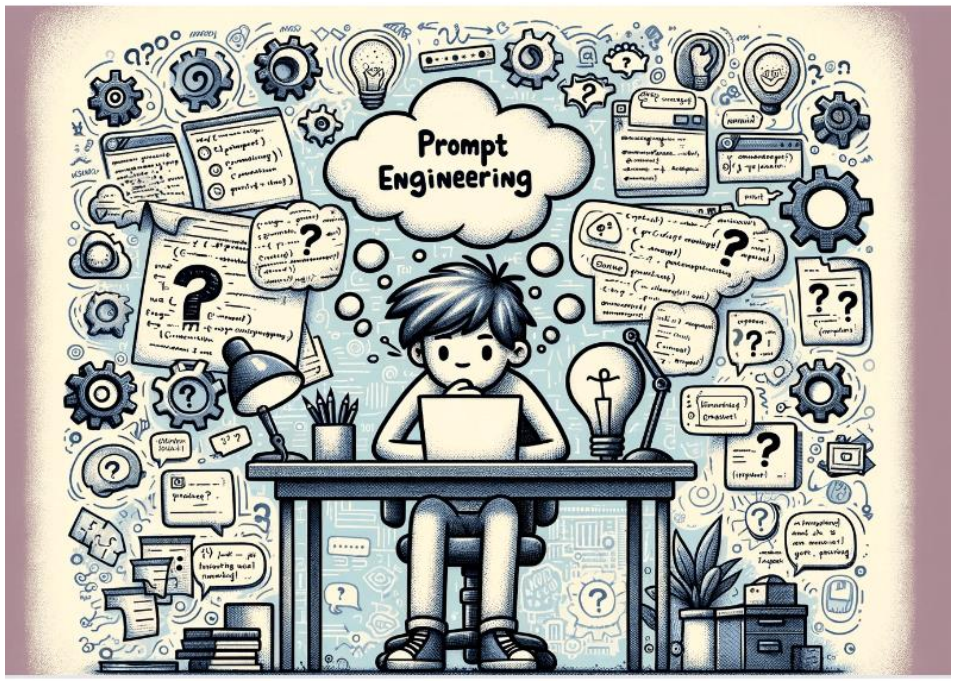
Prompt Engineering

What is Prompt Engineering ?

PROMPT ENGINEERING refers to the process of crafting precise and effective prompts or queries for large language models, such as GPT (Generative Pre-trained Transformer), to elicit desired responses. These prompts guide the behavior of language models by providing them with input data, influencing the quality and relevance of the generated output.

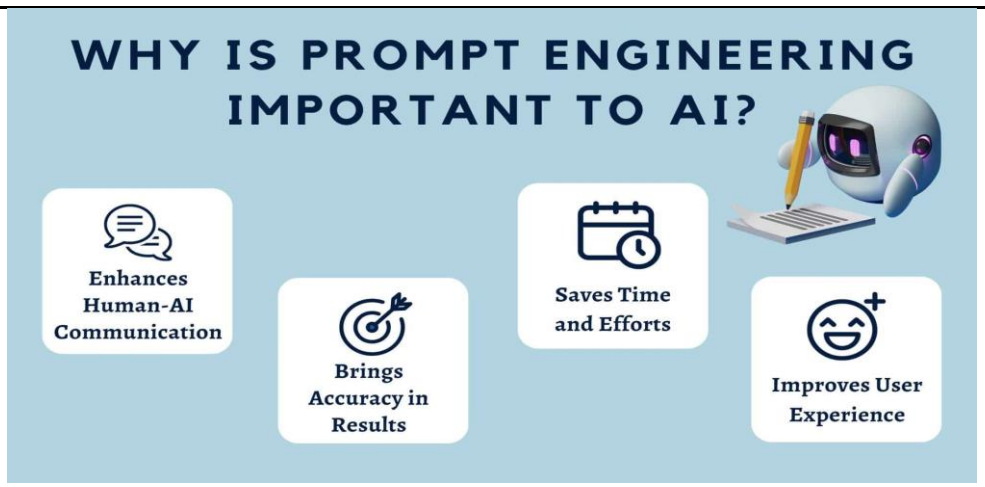
EVOLUTION OF PROMPT ENGINEERING

The evolution of prompt engineering can be traced back to the development of sophisticated language models like GPT. As these models grew in complexity and capability, it became crucial to optimize the way they interacted with users. Prompt engineering emerged as a response to this need, aiming to refine the input provided to language models to produce more accurate, contextually relevant, and coherent responses.
The term "prompt engineering" likely originated within the context of AI research and development, particularly as large language models gained prominence in the early 21st century. It reflects the growing recognition of the importance of fine-tuning inputs to optimize the performance of AI systems.



Prompt engineering is important for several reasons:

- ✓ **IMPROVING RESPONSE QUALITY:** Crafting well-designed prompts helps ensure that language models generate accurate and relevant responses to user queries, enhancing the overall user experience.
- ✓ **ENHANCING USER INTERACTION:** By tailoring prompts to specific contexts, tones, or intents, prompt engineering facilitates more natural and meaningful interactions between users and AI systems, promoting engagement and usability.
- ✓ **MITIGATING BIAS AND MISINFORMATION:** Thoughtfully constructed prompts can help mitigate biases and prevent the propagation of misinformation by steering language models toward producing more factual and unbiased responses.
- ✓ **INCREASING EFFICIENCY:** Effective prompt engineering can streamline the process of querying language models, reducing the need for manual intervention or post-processing of generated outputs.



In the era of artificial intelligence, prompt engineering plays a crucial role in leveraging the capabilities of large language models and other AI systems. It serves as a bridge between users and AI, enabling more effective communication and interaction. By providing AI systems with carefully crafted prompts, prompt engineers can harness the full potential of these technologies to solve complex problems, provide valuable insights, and enhance decision-making processes across various domains.

How to prompt ?

WRITE CLEAR INSTRUCTIONS

These models can't read your mind. If outputs are too long, ask for brief replies. If outputs are too simple, ask for expert-level writing. If you dislike the format, demonstrate the format you'd like to see. The less the model has to guess at what you want, the more likely you'll get it.

Tactics:

- Include details in your query to get more relevant answers
- Ask the model to adopt a persona
- Use delimiters to clearly indicate distinct parts of the input
- Specify the steps required to complete a task
- Provide examples
- Specify the desired length of the output

Tactic	Definition	Examples

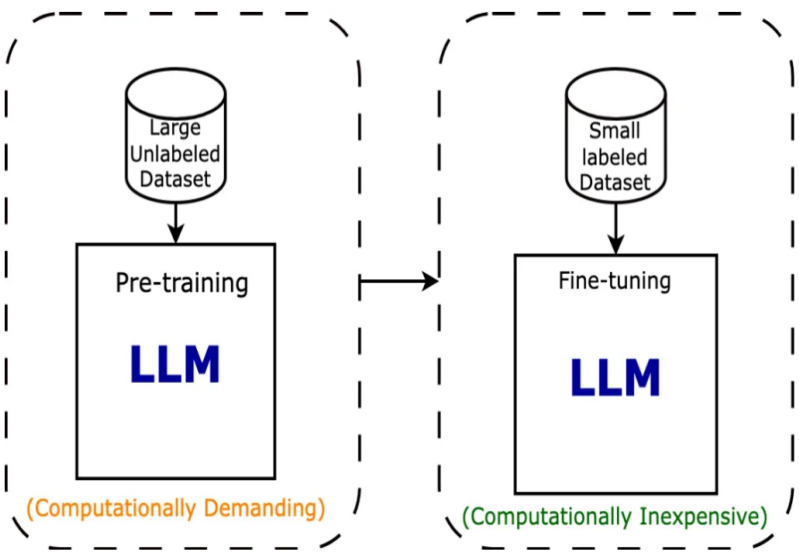
Include details in your query to get more relevant answers	In order to get a highly relevant response, make sure that requests provide any important details or context. Otherwise, you are leaving it up to the model to guess what you mean.	**Worse** How do I add numbers in Excel? Summarize the meeting notes.	**Better** How do I add up a row of dollar amounts in Excel? I want to do this automatically for a whole sheet of rows with all the totals ending up on the right in a column called "Total". Summarize the meeting notes in a single paragraph. Then write a markdown list of the speakers and each of their key points. Finally, list the next steps or action items suggested by the speakers, if any.
Ask the model to adopt a persona	The system message can be used to specify the persona used by the model in its replies.	"You are a University Professor writing a module descriptor for a new course, help me write the learning outcomes."	
Use delimiters to clearly indicate distinct parts of the input	Delimiters like triple quotation marks, XML tags, section titles, etc. can help demarcate sections of text to be treated differently.	Summarize the text delimited by triple quotes with a haiku. """insert text here"""	
Specify the steps required to complete a task	Some tasks are best specified as a sequence of steps. Writing the steps out explicitly can make it easier for the model to follow them.	Use the following step-by-step instructions to respond to user inputs. Step 1 - The user will provide you with text in triple quotes. Summarize this text in one sentence with a prefix that says "Summary:". Step 2 - Translate the summary from Step 1 into Spanish, with a prefix that says "Translation: ".	
Provide examples	Providing general instructions that apply to all examples is generally more efficient than demonstrating all permutations of a task by example, but in some cases providing examples may be easier. This is known as "few-shot" prompting.	**Prompt:** A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses the word whatpu is: We were traveling in Africa, and we saw these very cute whatpus. To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses the word farduddle is: **Output:** When we won the game, we all started to farduddle in celebration.	
Specify the desired length of the output	You can ask the model to produce outputs that are of a given target length. The targeted output length can be specified in terms of the count of words, sentences, paragraphs, bullet points, etc.	**Example 1:** Summarize the text delimited by triple quotes in about 50 words. """insert text here""" **Example 2:** Summarize the text delimited by triple quotes in 2 paragraphs. """insert text here"""	

FINE-TUNING LARGE LANGUAGE MODELS

FINE-TUNING a large language model (LLM) involves customizing a pre-trained model for a specific task or domain by training it on task-specific data. This process allows the model to adapt its parameters to better suit the target task, resulting in improved performance and more accurate outputs. Fine-tuning typically involves providing examples of inputs and their corresponding desired outputs to the model, allowing it to learn patterns and relationships specific to the task at hand. Fine-tuning enables the model to better understand the nuances of the target domain and produce more relevant and contextually appropriate responses.

Fine-tuning lets you get more out of pre-trained models:

- Higher quality results than prompting
- Ability to train on more examples than can fit in a prompt.
- Token savings due to shorter prompts
- Lower latency requests



Fine tuning LLMs examples

TEXT CLASSIFICATION: Suppose you have a large dataset of customer reviews labeled as positive or negative sentiment. You can fine-tune an LLM like GPT-3 by providing it with these reviews and their corresponding sentiment labels. The model learns to associate certain language patterns with positive or negative sentiment, allowing it to classify new reviews accurately.

LANGUAGE TRANSLATION: If you want to create a language translation system, you can fine-tune an LLM using parallel corpora, which are pairs of sentences in two different languages with their translations. By feeding these pairs into the model, it learns to generate accurate translations for input sentences in one language into another.

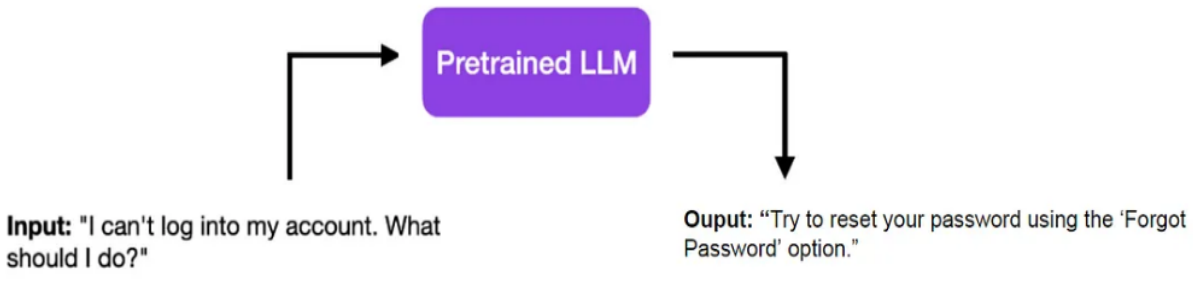
MEDICAL DIAGNOSIS: Consider an LLM used for diagnosing diseases based on medical transcripts. This LLM, fine-tuned with medical data, will offer far superior performance compared to the base model, which lacks the required medical knowledge. Therefore, fine-tuning becomes indispensable when dealing with specialized fields, sensitive data, or unique information that isn't well-represented in the general training data.

Fine-tuning allows you to leverage the power of pre-trained models while tailoring them to specific applications, resulting in more accurate and contextually relevant outputs.

Finetuning LLMs give them domain expertise. Let's say you're interested in a specific topic, like the stock market. Reading a general book on finance won't provide you with the expert perspective you need. You would benefit more from a book written by a stock market expert.

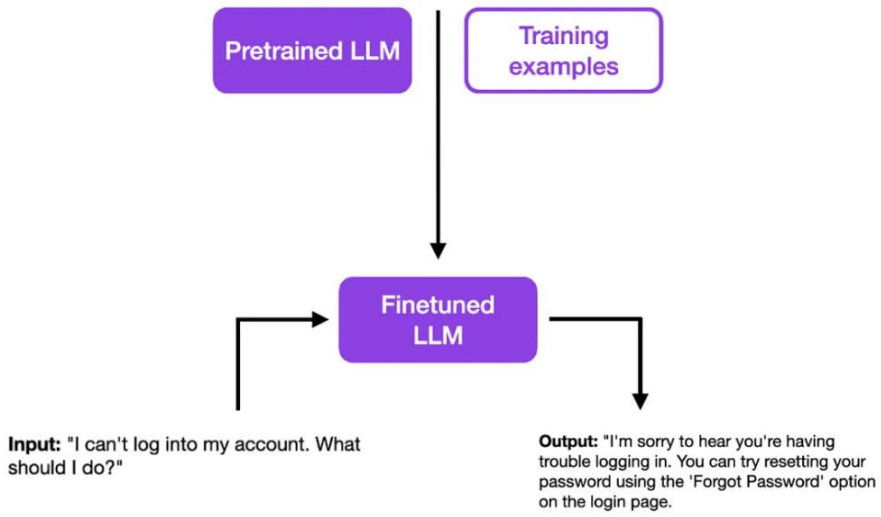
Another example - Suppose you are an accomplished linguist, capable of speaking several languages fluently. Now, you want to become a tour guide in Paris. While your overall language skills are impressive, you'll need specialized knowledge about French history, culture, and local slang to truly shine in your new role. This is where specialized training or 'fine-tuning' comes into play.

For example, let's suppose you have a pretrained LLM. To the input `I can't log into my account. What should I do?`
it answers with a simple `Try to reset your password using the "Forgot Password" option.`



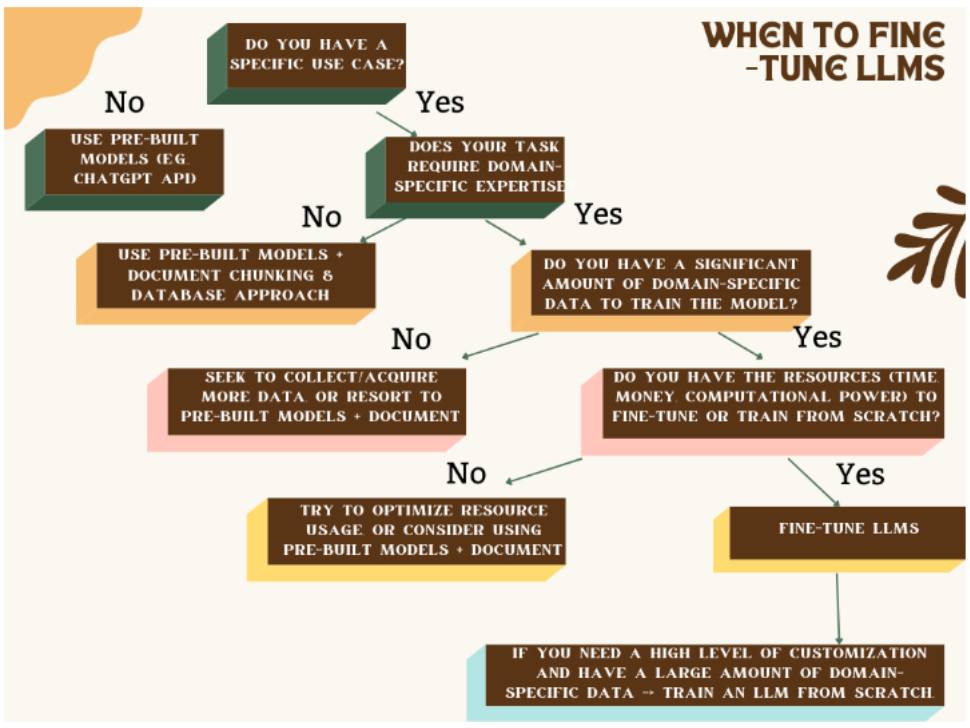
A dry and concise answer to a Customer Support question, using Pretrained LLMs.

And now imagine you want to build a chatbot for a Customer Support service. Although the answer below may be **correct**, it is not **adequate** as it is as a Customer Support answer, which would require more empathy, a different format, additional contact details, or whatever your guidelines are. This is when **Finetuning** comes into play.



A better answer to a Customer Support question matching some guidelines, after Finetuning.

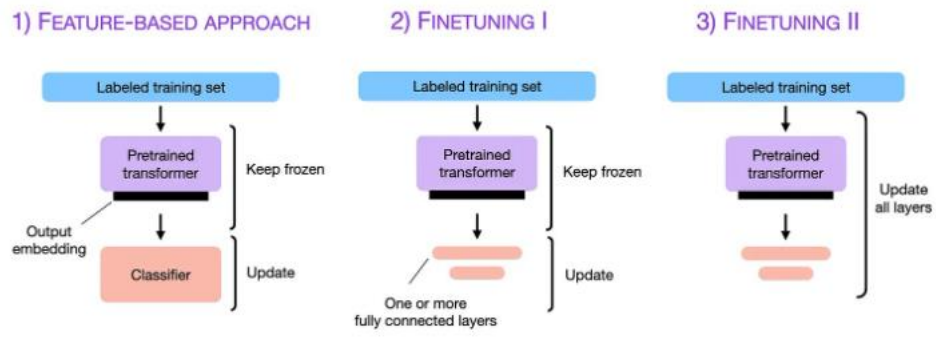
WHEN TO OPT FOR FINE TUNING ?

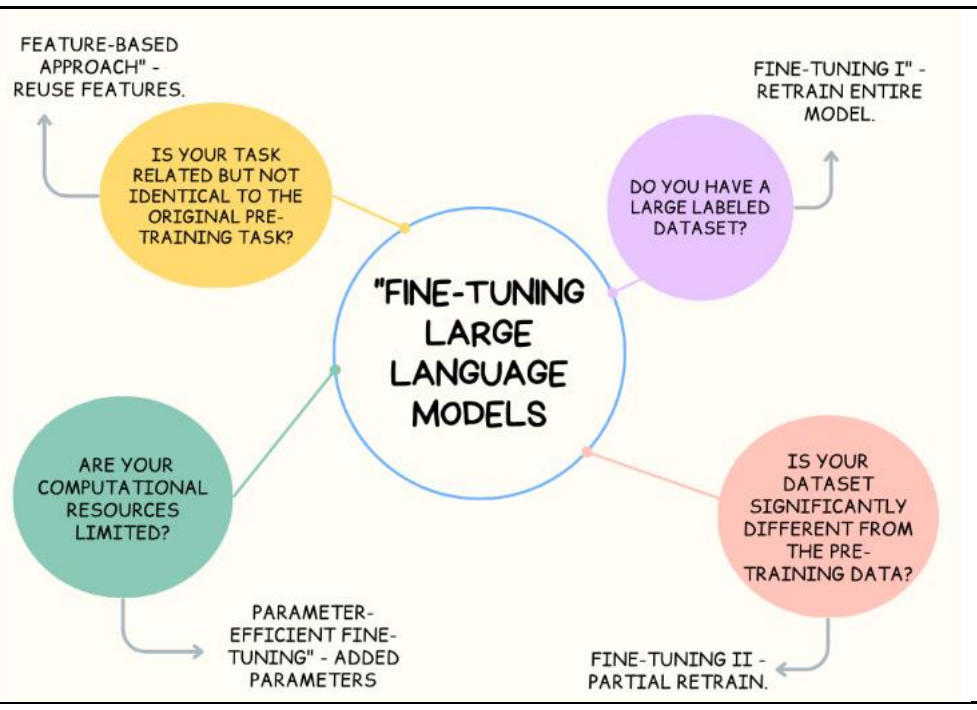


METHODS OF FINE TUNING

FINE-TUNING an LLM can be carried out in a variety of ways, depending on the task at hand and the resources at your disposal. At times, the base model is used as a 'fixed feature extractor,' wherein the model processes new input data and generates high-dimensional vector representations. These features are then utilized to train a new model that is task specific.

Recently, **'PARAMETER-EFFICIENT FINE-TUNING'** methods like Adapter Tuning have gained traction in the field. This approach introduces 'adapter layers,' a new set of parameters, to each transformer block of the model. This significantly enhances the efficiency of the fine-tuning process, making it a promising avenue for future LLMs applications.





CONCLUSION :

As models continue to grow larger and more powerful, effective fine-tuning will become even more essential to leverage these models' capabilities fully. It's conceivable that we'll see more fine-tuned models in a broad array of specialized tasks and languages, further democratizing AI and making it accessible to even more people and industries.

The road to perfect fine-tuning is an ongoing journey, paved with constant learning, experimentation, and innovation. As advancements continue in AI research, and as LLMs become even more powerful and sophisticated, the importance and impact of fine-tuning will only grow.

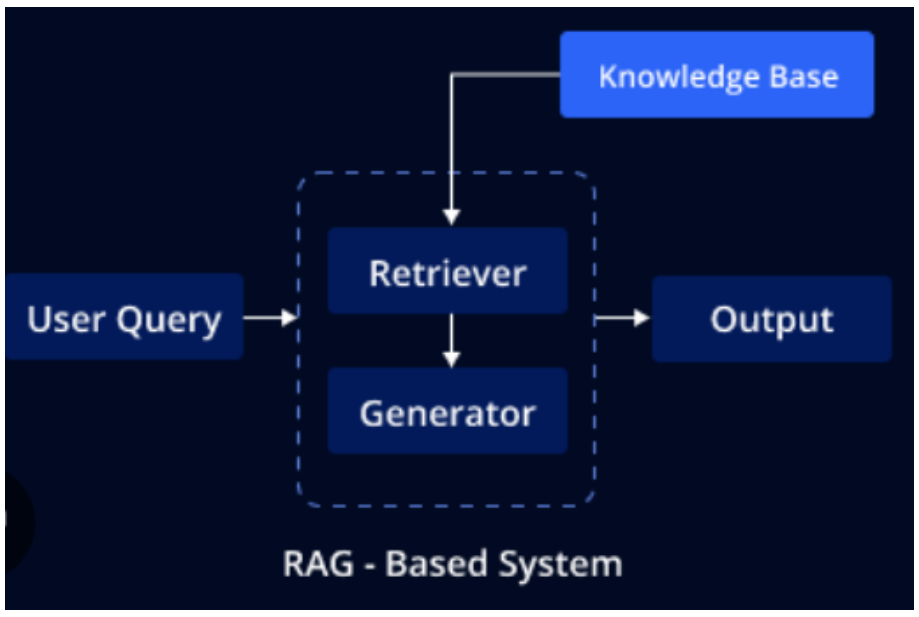
What is Retrieval Augmented Generation (RAG)?

RAG stands for **RETRIEVAL-AUGMENTED GENERATION.**

Retrieval-augmented generation is an AI framework that integrates LLMs with external knowledge sources. This synergy allows LLMs to supplement their internal information with facts retrieved from an external knowledge base, such as customer records, dialogue paragraphs, product specifications, and even audio content. By utilizing this external context, LLMs can generate more informed and accurate responses.

It combines two main processes:

RETRIEVAL: Before generating a response, the model retrieves relevant information from a large external database or corpus. This could be a collection of documents, articles, or even the internet.
GENERATION: Using the retrieved information, the model generates a coherent and contextually relevant response to a given query or prompt.



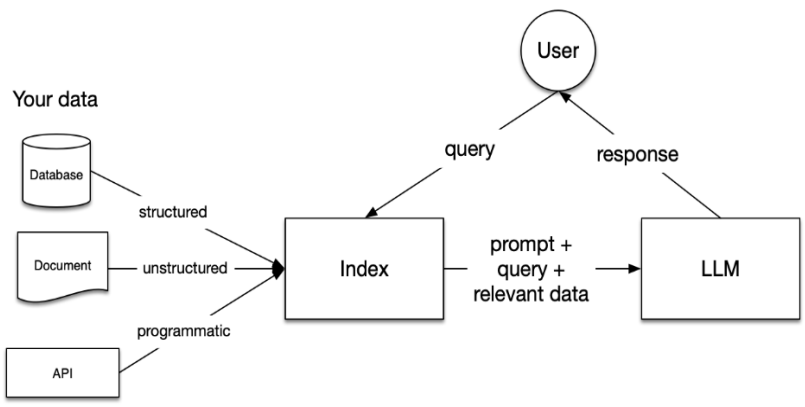
RAG architecture

The RAG architecture is composed of the Data Source, the Index and the Language Model.

DATA SOURCE: This is where all the knowledge begins. It includes both structured data, like databases where information is organized in a clear format, and unstructured data, such as documents or web pages where information is more free form. This varied data is processed into what we call 'vector embeddings', which are essentially numerical representations that a computer can understand and search through efficiently.

INDEX: Think of the Index as a vast, high-tech library catalog. It takes all those vector embeddings from the Data Source and organizes them so that the most relevant pieces of information can be retrieved when needed. This Index can be stored in various ways—like in-memory for quick access or in a Vector Database for more extensive, complex searches.

LANGUAGE MODEL (LLM): When a user comes with a query or prompt, instead of the LLM guessing the answer using only what it has learned before, the RAG system first consults the Index. It identifies the information that is 'closest'—meaning most relevant and up to date—to the user's query. This selected information is then fed to the LLM, which uses it to craft a precise and relevant response.



Benefits of RAG in LLMs

ACCURACY: By incorporating information retrieval, RAG helps LLMs produce more accurate responses by grounding them in existing, verified information from reliable sources.
UP-TO-DATE INFORMATION: RAG enables LLMs to access and incorporate the latest information from the external database, ensuring that the responses are current and relevant.
CREDIBILITY: By citing the retrieved sources, RAG enhances the credibility of the generated responses, making them more trustworthy and reliable.
FLEXIBILITY: RAG allows for easy updating of the external database with new information, without the need to retrain the entire model. This makes it scalable and adaptable to changing data.

RAG Challenges

RISK OF HALLUCINATIONS: RAG can still make errors if the database lacks certain information.

MANAGING SCALABILITY: Increasing the database size can complicate quick and efficient data retrieval.

POTENTIAL BIASES: Biases in the retrieval database can influence the responses, necessitating tools to identify and mitigate these biases.

The Generative AI Maturity curve

Before LLMs, ML development was linear. Teams needed months of tedious data collection, feature engineering, and multiple training runs, as well as a team of PhDs before the system could be produced as a customer-facing end product.

The table below summarizes key methods in LLM development, outlining their definitions, use cases, data requirements, advantages, and considerations for your business applications. It covers Prompt Engineering, Retrieval Augmented Generation (RAG), Fine-tuning, and Pretraining.

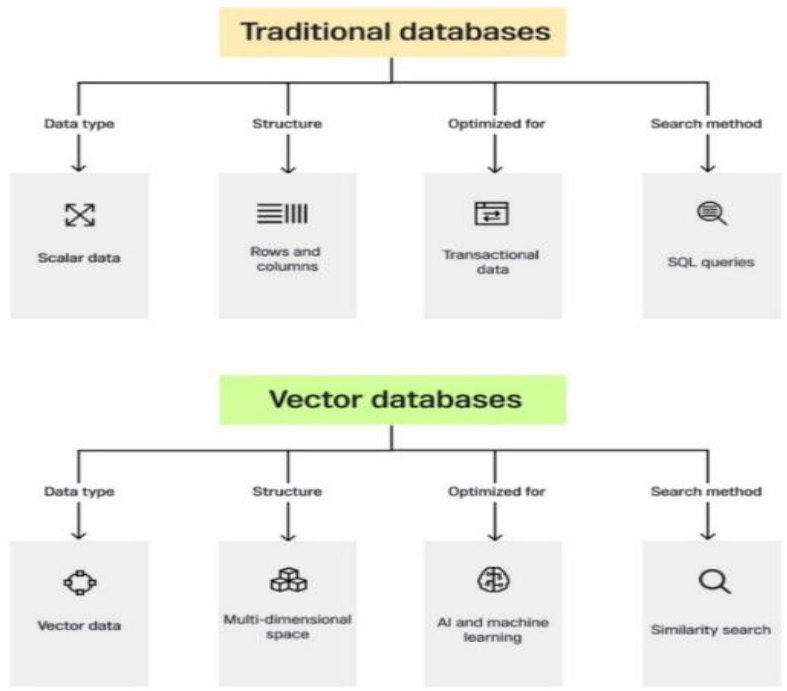
Method	Definition	Primary Use Case	Data Requirements	Advantages	Considerations
Prompt Engineering	Crafting specialized prompts to guide LLM behavior	Quick, on-the-fly model guidance	None	Fast, cost-effective, no training required	Less control than fine-tuning
Retrieval Augmented Generation (RAG)	Combining an LLM with external knowledge retrieval	Dynamic datasets and external knowledge	External knowledge base or database (e.g., vector database)	Dynamically updated context, enhanced accuracy	Increases prompt length and inference computation
Fine-tuning	Adapting a pretrained LLM to specific datasets or domains	Domain or task specialization	Thousands of domain-specific or instruction examples	Granular control, high specialization	Requires labeled data, computational cost
Pretraining	Training an LLM from scratch	Unique tasks or domain-specific corpora	Large datasets (billions to trillions of tokens)	Maximum control, tailored for specific needs	Extremely resource-intensive

What is a Vector Database ?

VECTOR DATABASES are a new type of database designed to store and manage vectors, which are arrays of numerical values representing complex objects like images, text, and documents.
These databases are essential for AI applications, especially large language models, as they enable efficient storage, retrieval, and comparison of high-dimensional data.

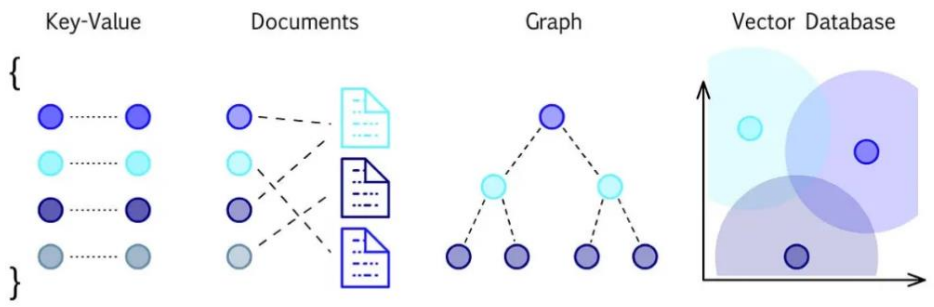
Vector databases are specialized systems that store and manage high-dimensional vector data efficiently. They enable efficient storage, retrieval, and similarity search for complex data types like text, images, and audio. Vector databases have gained popularity with the rise of machine learning and AI, which rely heavily on vector data representations. A vector database can organize large volumes of vector data and perform operations like similarity search, clustering, and dimensionality reduction.

In the simplest terms, vector databases use an embedding model that converts text data into vectors, which are a numerical representation of the text data in a n-dimensional space. Then, in the next step, when you are trying to search for something, this embedding model is used again to convert your query into a vector.



MEMORY LANE - EVOLUTION OF DATABASES:

- **SQL**: Traditional database storing structured data in tables.
- **NOSQL**: Designed for unstructured data like documents, suitable for real-time web applications and big data.
- **GRAPH DATABASES**: Store data in nodes and represent relationships, ideal for federated APIs.
- **VECTOR DATABASES**: The latest evolution, tailored for AI applications, offering efficient vector storage and retrieval.



How do Vector databases work ?

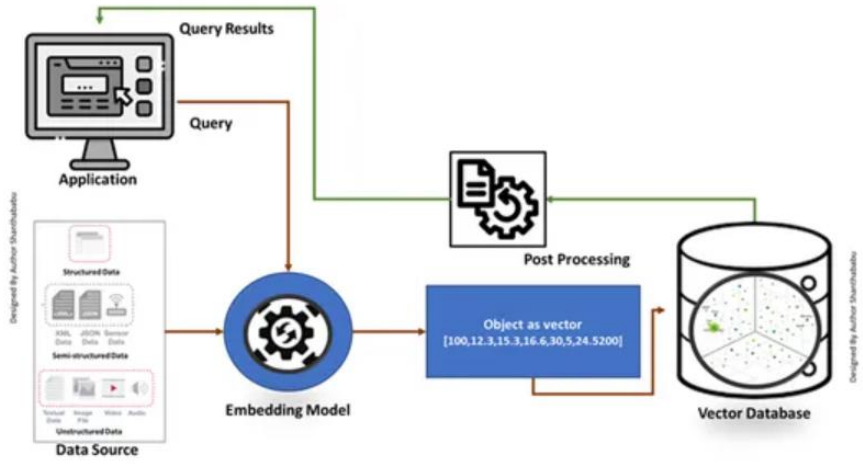
Vector databases are designed to efficiently store, retrieve, and manipulate high-dimensional vectors. They are tailored for handling complex data types like images, text embeddings, and other numerical representations commonly used in AI applications.
Here's an overview of how vector databases work:

DATA REPRESENTATION	INDEXING AND RETRIEVAL:
Vector Storage: Vectors, representing various data types such as images, text, and numerical values, are stored in the database. Each vector is associated with metadata and can be indexed for efficient retrieval. *Embeddings:* Groups of vectors, known as embeddings, are stored in a multidimensional format. Embeddings facilitate grouping and clustering of similar vectors, enabling efficient data organization and comparison.	*Similarity Search:* Vector databases support similarity search algorithms to identify vectors that are close or similar to a given query vector. This enables applications like image similarity, semantic search, and recommendation systems. *Indexing:* Efficient indexing techniques, such as approximate nearest neighbor (ANN) search, are employed to speed up the retrieval of vectors. Index structures like k-d trees, ball trees, or FAISS (Facebook AI Similarity Search) index can be used to organize vectors for quick querying.
SCALABILITY AND PERFORMANCE	**INTEGRATION WITH AI MODELS**
Horizontal Scalability: Vector databases are designed to scale horizontally, allowing them to handle large-scale datasets with millions or billions of vectors. Distributed storage and parallel processing enable efficient data management and high-throughput operations. *Query Optimization:*	*AI Model Interoperability:* Vector databases can be seamlessly integrated with AI models, such as large language models, to store and retrieve embeddings for various tasks. This integration enables AI applications to leverage the extensive vector databases for enhanced performance and scalability.

Advanced optimization techniques, including caching, prefetching, and query planning, are used to enhance query performance and reduce latency.
Batch processing and vector quantization can also be employed to optimize computational efficiency.

Working Mechanism

- Vector databases first convert data into embedding vectors, store it in vector databases, and create indexing for quicker searching.
- A query from the application will interact with the embedding vector, searching for the nearest neighbor or similar data in the vector database using an index and retrieving the results passed to the application.
- Basis the business requirements, the retrieved data would be fine-tuned, formatted, and displayed to the end user side or query or action(s) feed.



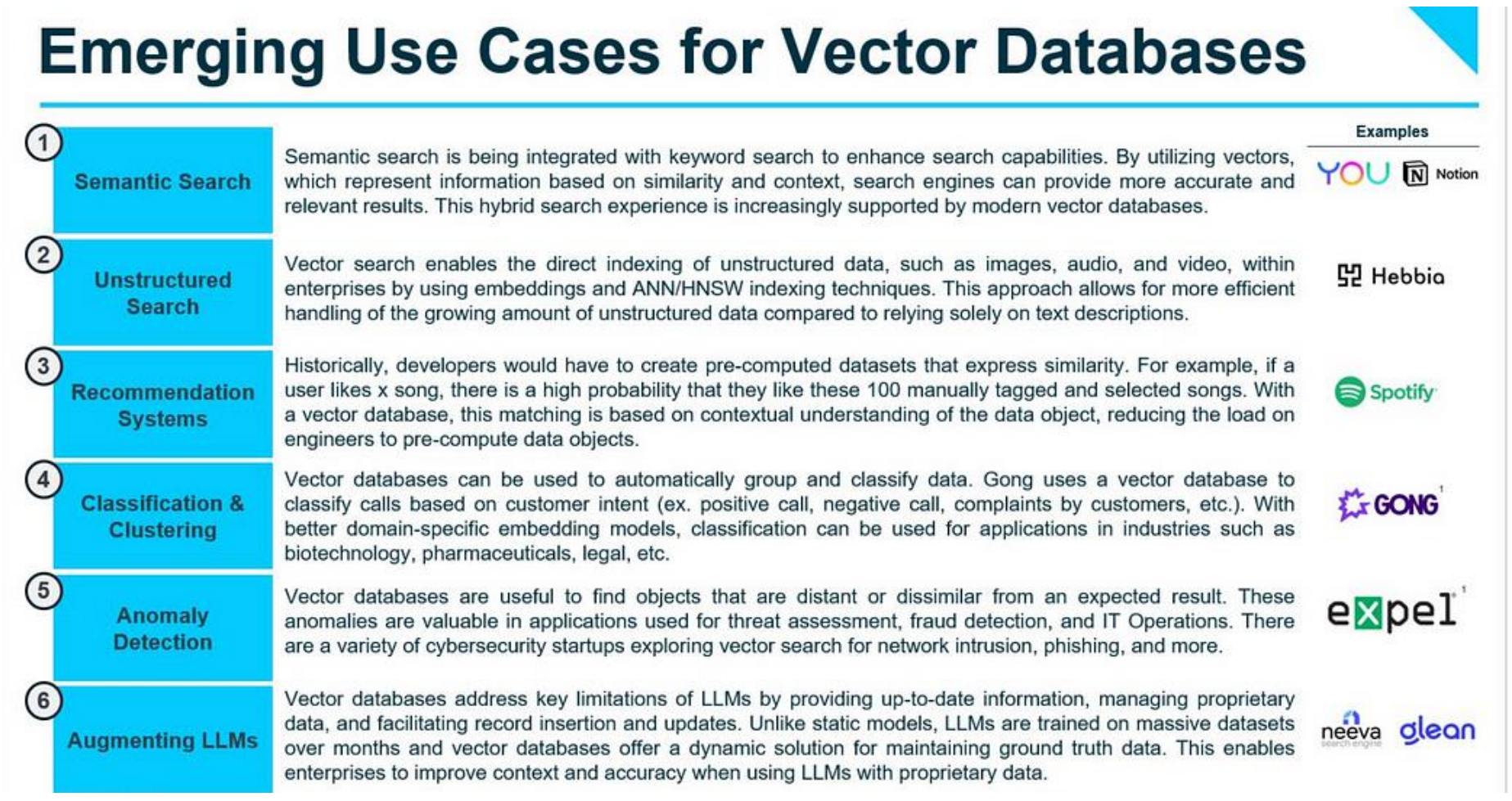
Use cases of Vector Database

Vector databases have many use cases across different domains and applications that involve natural language processing (NLP), computer vision (CV), recommendation systems (RS), and other areas that require semantic understanding and matching of data.

One use case for storing information in a vector database is to enable large language models (LLMs) to generate more relevant and coherent text based on an AI plugin.

However, large language models often face challenges such as generating inaccurate or irrelevant information; lacking factual consistency or common sense; repeating or contradicting themselves; being biased or offensive. To overcome these challenges, you can use a vector database to store information about different topics, keywords, facts, opinions, and/or sources related to your desired domain or genre. Then, you can use a large language model and pass information from the vector database with your AI plugin to generate more informative and engaging content that matches your intent and style.

For example, if you want to write a blog post about the latest trends in AI, you can use a vector database to store the latest information about that topic and pass the information along with the ask to a LLM in order to generate a blog post that leverages the latest information.



What is LangChain?

LangChain is an open-source orchestration framework for the development of applications using large language models (LLMs). Available in both Python- and JavaScript-based libraries, LangChain's tools and APIs simplify the process of building LLM-driven applications like chatbots and virtual agents.

LangChain serves as a generic interface for nearly any LLM, providing a centralized development environment to build LLM applications and integrate them with external data sources and software workflows. LangChain's module-based approach allows developers and data scientists to dynamically compare different prompts and even different foundation models with minimal need to rewrite code. This modular environment also allows for programs that use multiple LLMs: for example, an application that uses one LLM to interpret user queries and another LLM to author a response.

In simplest terms, LangChain is like a toolbox that helps developers easily build applications using large language models (LLMs) like GPT-4 or Llama 2.
It provides a set of tools that make it easier to work with LLMs, reducing the amount of code needed to create applications.
It offers a single place to develop, test, and integrate LLM applications with other software and data sources.
LangChain can work with various LLMs, allowing developers to choose the best model for their needs.

Integrations with LLMs

LLMs are not standalone applications: they are pre-trained statistical models that must be paired with an application (and, in some cases, specific data sources) in order to meet their purpose.

For example, Chat-GPT is not an LLM: it is a chatbot application that, depending on the version you've chosen, uses the GPT-3.5 or GPT-4 language model. While it's the GPT model that interprets the user's input and composes a natural language response, it's the application that (among other things) provides an interface for the user to type and read and a UX design that governs the chatbot experience. Even at the enterprise level, Chat-GPT is not the only application using the GPT model: Microsoft uses GPT-4 to power Bing Chat.

If a given generative AI task requires access to external software workflows—for example, if you wanted your virtual agent to integrate with Slack—then you will need a way to integrate the LLM with the API for that software.

While these integrations can generally be achieved with fully manual code, orchestration frameworks like LangChain greatly simplify the process. They also make it much easier to experiment with different LLMs to compare results, as different models can be swapped in and out with minimal changes to code.

LangChain – Key Features

ABSTRACTIONS:
LangChain offers a library of abstractions for Python and JavaScript, serving as building blocks for generative AI programs.
These abstractions can be chained together to create applications, reducing the coding complexity required for complex Natural Language Processing (NLP) tasks.

IMPORTING LANGUAGE MODELS:
LangChain supports nearly any LLM and allows for easy importing with an API key.
It supports both open-source models (e.g., BigScience's BLOOM, Meta AI's LLaMa, Google's Flan-T5) and custom LLM wrappers.

PROMPT TEMPLATES:
Helps in crafting effective prompts for LLMs, facilitating context-aware responses.
The PromptTemplate class enables structured prompt composition without manual coding.

CHAINS:
Core of LangChain's workflows, combining LLMs with other components to execute sequences of functions.
Supports basic chains like LLMChain and advanced ones like SimpleSequentialChain for more complex workflows.

Additional Components:

VECTOR DATABASES:
Represents data points using vector embeddings, enabling efficient retrieval and low-latency queries.

TEXT SPLITTERS:

INDEXES:
LangChain provides access to external data sources termed as "indexes" for specific tasks.
Offers document loaders, vector databases, and text splitters to facilitate data integration and retrieval.

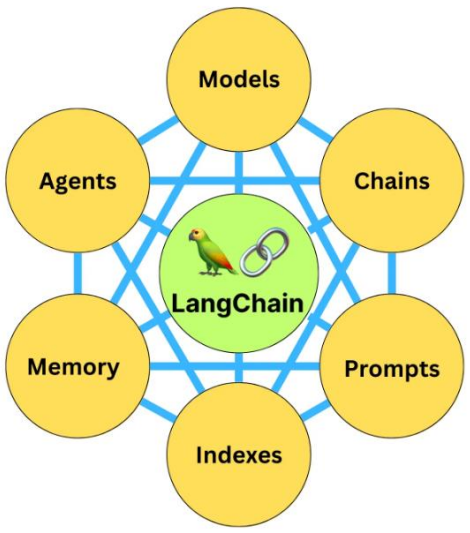
MEMORY UTILITIES:
Enables retention of conversation history to improve LLM performance and context retention.

AGENTS:
Utilizes LLMs as reasoning engines to determine actions, taking inputs like available tools, user input, and previous steps.

TOOLS:
LangChain tools enable interaction with real-world information to enhance the services provided by LLMs.
Includes functions like Wolfram Alpha for mathematical capabilities, Google Search for real-time information, OpenWeatherMap for weather data, and Wikipedia for access to articles.

Divides large text documents into smaller, meaningful chunks to improve processing speed.

RETRIEVAL:
Offers retrieval augmented generation (RAG) for efficient information retrieval from connected external sources.



Uses Cases of Langchain

Applications made with LangChain provide great utility for a variety of use cases, from straightforward question-answering and text generation tasks to more complex solutions that use an LLM as a "reasoning engine."

- **CHATBOTS:** Chatbots are among the most intuitive uses of LLMs. LangChain can be used to provide proper context for the specific use of a chatbot, and to integrate chatbots into existing communication channels and workflows with their own APIs.
- **SUMMARIZATION:** Language models can be tasked with summarizing many types of text, from breaking down complex academic articles and transcripts to providing a digest of incoming emails.
- **QUESTION ANSWERING:** Using specific documents or specialized knowledge bases (like Wolfram, arXiv or PubMed), LLMs can retrieve relevant information from storage and articulate helpful answers). If fine-tuned or properly prompted, some LLMs can answer many questions even without external information.
- **DATA AUGMENTATION:** LLMs can be used to generate synthetic data for use in machine learning. For example, an LLM can be trained to generate additional data samples that closely resemble the data points in a training dataset.

- **VIRTUAL AGENTS:** Integrated with the right workflows, LangChain's Agent modules can use an LLM to autonomously determine next steps and take action using robotic process automation (RPA).